



CARRERA DE INGENIERÍA EN COMPUTACIÓN

INFORME DEL TRABAJO DE GRADUACIÓN

Diseño, Implementación y Despliegue de un Sistema de Firma Electrónica de Documentos On-Premise para la FACET

Autor:

Blasich, Fabricio Lucas

Tutor:

Mag. Ing. Carlos Albaca Paraván

Co-Tutor:

Mag. Ing. Esteban Daniel Volentini

Jurados:

Ing. Mariana Sanchez

Ing. Federico Lutz

Ing. Javier Bilbao

*Este trabajo fue realizado en la Ciudad de San Miguel de Tucumán,
entre noviembre de 2025 y abril de 2026.*

Resumen

Este trabajo describe el diseño, la implementación y el despliegue de un sistema de firma electrónica de documentos sobre la infraestructura propia de la Facultad de Ciencias Exactas y Tecnología de la Universidad Nacional de Tucumán. Ante la necesidad de centralizar y agilizar los procesos de firma, hasta entonces manuales y distribuidos, se seleccionó una plataforma de código abierto y se desplegó sobre el entorno de virtualización institucional. La solución integra almacenamiento de objetos, control de acceso restringido a las cuentas de la facultad y notificaciones automáticas por correo electrónico, garantizando el control total de la información sin depender de servicios externos con costos recurrentes de licenciamiento.

El desarrollo requirió conocimientos de redes de computadoras, virtualización, contenerización de servicios, seguridad informática y administración de sistemas. Como resultado, se entregó una solución funcional que incluye manuales técnicos y procedimientos de mantenimiento orientados a asegurar la trazabilidad documental y la autonomía operativa del personal de la facultad.

Agradecimientos

A mis padres y a mi hermana, pilares fundamentales de mi vida, por brindarme la oportunidad, la confianza y el apoyo necesarios para recorrer este camino. A mi familia, tanto a quienes me acompañaron incondicionalmente a la distancia como a quienes me recibieron y me hicieron sentir en casa aquí en Tucumán.

A mis tutores, Carlos Albaca Paraván y Esteban Daniel Volentini, por su guía, paciencia y acompañamiento constante para llevar este Trabajo a su cierre. Al Ing. Javier Bilbao, por su enorme predisposición y generosidad al resolver cada obstáculo de infraestructura de red que este proyecto demandó: desde la configuración del espacio de trabajo institucional en Google hasta la gestión del dominio propio del sistema. Sin ese respaldo, la puesta en producción no habría sido posible. A mis docentes, compañeros y amigos, por todo lo compartido a lo largo de la carrera y por haber contribuido, cada uno a su manera, a mi formación personal y profesional. A la cátedra de Informática/Programación 1, en la que tuve la oportunidad de trabajar durante dos años y obtener mi primer trabajo oficial, por haber sido una experiencia formativa clave en mi recorrido. A la Universidad Nacional de Tucumán, por brindarme la posibilidad de acceder a una educación pública, gratuita y de excelencia, a la altura de cualquier institución de referencia en el mundo.

Índice general

Resumen	II
1. Introducción General	1
1.1. Descripción del Problema	1
1.2. Motivación y Justificación	2
1.3. Objetivos y Alcance del Trabajo	3
1.3.1. Objetivo General	3
1.3.2. Objetivos Específicos	3
1.3.3. Alcance y Limitaciones	3
1.4. Estado del Arte	4
1.4.1. Alternativas de Código Abierto (<i>Open Source</i>)	5
2. Marco Teórico y Tecnológico	7
2.1. Modelos de Despliegue: <i>On-Premise</i> y <i>Self-Hosted</i>	7
2.1.1. Software de Código Abierto y Licencia AGPL-3.0	7
2.2. Fundamentos de Seguridad de la Información	8
2.3. Mecanismos Técnicos de Firma Digital	8
2.3.1. Proceso de Generación de Firma	8
2.3.2. Certificados de Clave Pública y CA Interna	9
2.4. Marco Legal Argentino: Ley 25.506	9
2.5. Infraestructura de Virtualización y Despliegue	10
2.5.1. Plataforma de Virtualización y Orquestación	10
2.5.2. <i>Reverse Proxy</i> y Enrutamiento Dinámico (Traefik)	10
2.6. Arquitectura de Almacenamiento de Objetos	10
2.6.1. Gestión Física y Resiliencia (TrueNAS y ZFS)	10
2.6.2. MinIO: Almacenamiento de Objetos S3	11
2.6.3. Principios de Respaldo y Retención de Datos	11
2.7. Gestión de Identidad y Autorización: Google OAuth 2.0	11
3. Diseño e Implementación de la Infraestructura	13
3.1. Topología de Red y Arquitectura General	13
3.2. Infraestructura Base y Entorno PaaS	15
3.2.1. Auditoría del Entorno LXC y Configuración Base	15
3.2.2. Justificación del Uso de Coolify como PaaS	15
3.3. Configuración de Servicios y Despliegue de Documenso	17
3.3.1. Persistencia de Estado (PostgreSQL)	18
3.3.2. Motor de Renderizado Auxiliar (Browserless)	18
3.3.3. Servicio Principal de Documenso	19

3.4.	Conectividad con el Almacenamiento de Objetos (Proxy Socat)	21
3.5.	Estrategia de Respaldo y Retención de Datos	24
3.5.1.	Respaldo lógico de la base de datos PostgreSQL	25
3.5.2.	Retención e inmutabilidad del repositorio documental	25
3.5.3.	Resguardo de la configuración del despliegue	26
3.6.	Integración del Servicio de Correo Transaccional	27
3.6.1.	Configuración del Servicio de Correo Transaccional (Brevo SMTP) . .	27
3.6.2.	Autenticación Institucional con Google OAuth 2.0	28
4.	Ensayos y Resultados	29
4.1.	Metodología de Validación	29
4.2.	Validación de Despliegue de Servicios	30
4.2.1.	Estado de los contenedores en Coolify	30
4.2.2.	Verificación del estado interno de la aplicación	30
4.3.	Verificación de la Comunicación entre Servicios	31
4.3.1.	Conectividad con el almacenamiento de objetos (MinIO)	31
4.3.2.	Comunicación con Browserless (renderizado del certificado de firma) .	33
4.3.3.	Entregabilidad del correo transaccional (Brevo SMTP)	34
4.4.	Validación de Acceso y Control de Seguridad	35
4.4.1.	Restricción del inicio de sesión con Google OAuth 2.0	35
4.4.2.	Deshabilitación del registro local	35
4.5.	Validación del Flujo Funcional de Extremo a Extremo	36
4.5.1.	Ciclo completo de firma documental	36
4.5.2.	Verificación de la firma electrónica en el documento	37
4.6.	Métricas de Rendimiento del Sistema	38
4.6.1.	Consumo de recursos en operación normal (baseline)	38
4.6.2.	Consumo de recursos bajo carga controlada	40
4.7.	Validación del Respaldo y Recuperación	43
4.7.1.	Respaldo de la base de datos	44
4.7.2.	Prueba de restauración de la base de datos	44
4.7.3.	Respaldo de la configuración del <i>stack</i>	45
4.7.4.	Validación de las políticas de ciclo de vida	46
4.8.	Trazabilidad y Cumplimiento de Requisitos	48
4.8.1.	Matriz de trazabilidad	49
4.8.2.	Comparativa con el estado del arte	49
5.	Conclusiones	51
5.1.	Conclusiones Generales	51
5.2.	Trabajo Futuro	52
A.	Diagramas y Topologías de Red	54
A.1.	Topología General de la Infraestructura	54
A.2.	Arquitectura Interna de la PaaS y Servicios Contenerizados	55
A.3.	Topología Interna de la Red Docker del Stack Documenso	56
A.4.	Flujos de Acceso a MinIO a Través del Proxy Socat	57

B. Composición de Servicios y Scripts de Despliegue	58
B.1. Composición Docker Completa del Despliegue	58
B.2. Asignación Inicial del Rol Administrador	60
B.3. Automatización del Resguardo de Configuración	60
B.4. Variables Mágicas de Coolify (SERVICE_###)	61
B.5. Generación de Material Criptográfico Institucional	62
B.6. Configuración de Almacenamiento S3 (MinIO)	63
B.7. Configuración de Transporte de Correo (SMTP)	63
B.8. Autenticación con Google OAuth 2.0	64
C. Manual de Mantenimiento del Sistema	65
C.1. Restauración de la Base de Datos	65
C.2. Restauración de la Configuración del Stack	67
C.2.1. Acceso a la terminal del servidor	67
C.2.2. Recuperación y extracción de archivos	68
C.2.3. Identificación de servicios	68
C.2.4. Restauración desde la interfaz de Coolify	69
C.3. Renovación del Certificado de Firma Institucional	70
C.4. Actualización de la Versión de Documenso	71
C.4.1. Prerrequisito crítico de hardware	71
C.4.2. Preparación	71
C.4.3. Procedimiento de actualización en Coolify	72
C.4.4. Verificación	72
C.4.5. Procedimiento de rollback	73
C.5. Diagnóstico de Fallas y Referencia Rápida	73
C.5.1. Monitoreo y registros	73
C.5.2. Tabla de resolución de problemas	73
C.6. Migración a Certificado Emitido por Autoridad Certificante (CA)	74
D. Guía de Inicio Rápido para Usuarios	76
D.1. Acceso al Sistema	76
D.2. Selección de Organización y Equipo	76
D.3. Carga y Preparación de un Documento	77
D.3.1. Carga del Archivo	77
D.3.2. Destinatarios y Roles	78
D.3.3. Colocación de Campos de Firma	78
D.4. Envío del Documento	79
D.5. Seguimiento y Finalización	80
D.6. Experiencia del Destinatario	81
D.7. Referencia Rápida	81
Bibliografía	82

Índice de figuras

1.1. Flujo actual del proceso de firma	2
2.1. Proceso de generación de firma	8
3.1. Arquitectura Global del Sistema	14
3.2. Cabeceras HTTP del dominio institucional	14
3.3. Certificado TLS del dominio institucional	15
3.4. Topología interna de la PaaS Coolify	17
3.5. Topología interna de la red Docker del despliegue Documenso	21
3.6. Evidencia del flujo directo entre la interfaz web de Documenso y MinIO durante la carga de recursos gráficos.	22
3.7. Evidencia del flujo indirecto de descarga de PDF firmado	23
3.8. Flujos de acceso a MinIO a través del <i>proxy</i> Socat	24
3.9. Restricción del registro local en Documenso	28
4.1. Panel de Coolify: servicios principales en estado Healthy	30
4.2. Panel de Coolify: servicio Socat en estado Healthy	30
4.3. Registro de acceso HTTP de MinIO: flujo directo desde el navegador	32
4.4. Registro de acceso HTTP de MinIO: flujo indirecto desde el <i>backend</i>	32
4.5. Encabezados de autenticación SMTP de un correo transaccional de Documenso	34
4.6. Error de acceso con cuenta externa en Google OAuth	35
4.7. Respuesta HTTP 302 al acceder a la ruta <i>/signup</i>	36
4.8. Cierre del ciclo de firma: notificación de completado y persistencia en MinIO	37
4.9. Panel de firmas en Adobe Acrobat sobre el documento sellado	37
4.10. Detalle del certificado institucional FACET en el visor de Microsoft Edge	38
4.11. Consumo de RAM en baseline: cuatro contenedores del <i>stack</i>	39
4.12. Consumo de CPU en baseline: cuatro contenedores del <i>stack</i>	40
4.13. CPU y RAM durante el sellado de un documento (1 firma)	41
4.14. CPU y RAM durante la ráfaga de cinco documentos concurrentes	42
4.15. <i>Bucket</i> documenso-backup-databases: listado de volcados diarios de PostgreSQL	44
4.16. <i>Bucket</i> backups-coolify: listado de respaldos diarios de configuración	45
4.17. Políticas ILM activas en los tres <i>buckets</i> del sistema	47
4.18. Interfaz web de MinIO: configuración de los tres <i>buckets</i> del sistema	48
A.1. Topología general de la infraestructura (versión ampliada)	54
A.2. Arquitectura interna de la PaaS y servicios contenerizados (versión ampliada)	55
A.3. Topología interna de la red Docker del <i>stack</i> Documenso (versión ampliada)	56
A.4. Flujos de acceso a MinIO a través del proxy Socat (versión ampliada)	57

C.1. Historial de respaldos de PostgreSQL en Coolify	66
C.2. Formulario de importación de respaldo S3 en Coolify	67
C.3. Ejecución de rclone en la terminal de Coolify	68
C.4. Editor de Docker Compose en Coolify	69
C.5. Variable de entorno del certificado institucional en Coolify	70
C.6. Verificación de instrucciones AVX en el servidor actual	71
D.1. Selector de organización	77
D.2. Selector de equipo dentro de FACET	77
D.3. Opciones de carga de documento	78
D.4. Editor de campos de firma	79
D.5. Modalidades de envío	80

Índice de Tablas

1.	Comparativa de costos de soluciones SaaS	4
2.	Comparativa de plataformas de código abierto	5
3.	Comparación entre almacenamiento en base de datos y almacenamiento compatible con S3 en Documenso	25
4.	Métricas de rendimiento: baseline vs. pico bajo carga	43
5.	Matriz de trazabilidad de requisitos	49
6.	Comparativa final: plataformas <i>SaaS</i> versus solución implementada	50
7.	Referencia rápida de resolución de problemas	74
8.	Roles de destinatario disponibles en Documenso.	78
9.	Estados del ciclo de vida de un documento.	80

Capítulo 1

Introducción General

El presente trabajo de graduación se desarrolla en el marco de la Facultad de Ciencias Exactas y Tecnología (FACET) de la Universidad Nacional de Tucumán (UNT). La FACET procesa un volumen constante de documentación administrativa y académica que requiere de mecanismos de validación y firma por parte de sus autoridades y personal docente.

1.1. Descripción del Problema

Históricamente, el flujo de trabajo para la firma de documentos en la institución ha sido manual y descentralizado, con ineficiencias que dificultan el trabajo diario del personal académico.

El proceso actual comienza cuando un usuario necesita firmar un documento. Típicamente, debe descargar el archivo, firmarlo con alguna herramienta externa, guardar una copia y enviarla por correo electrónico al siguiente responsable. Este ciclo se repite hasta conseguir todas las firmas. El proceso se describe mediante un diagrama de flujo en la Figura 1.1, y genera los siguientes problemas:

- **Tiempo Administrativo:** el usuario invierte tiempo en tareas repetitivas (descargar, buscar una herramienta de firma, procesar el archivo y volver a enviarlo) en lugar de realizar la acción de manera inmediata y centralizada.
- **Errores Humanos:** es común que se pierdan hilos de correos, se olviden archivos adjuntos o que los usuarios firmen en lugares incorrectos por no tener una guía visual clara en el documento.
- **Falta de Trazabilidad:** cada firmante posee una versión intermedia del documento en su dispositivo. No existe un registro centralizado que indique cuándo firmó cada persona (fecha y hora), ni un mecanismo que asegure el orden secuencial de las firmas, lo que dificulta saber en qué etapa exacta se encuentra el trámite.
- **Uso Ineficiente del Almacenamiento:** al no existir un repositorio único, ese archivo se replica innecesariamente y ocupa espacio tanto en los servidores de correo como en el almacenamiento local de cada usuario.

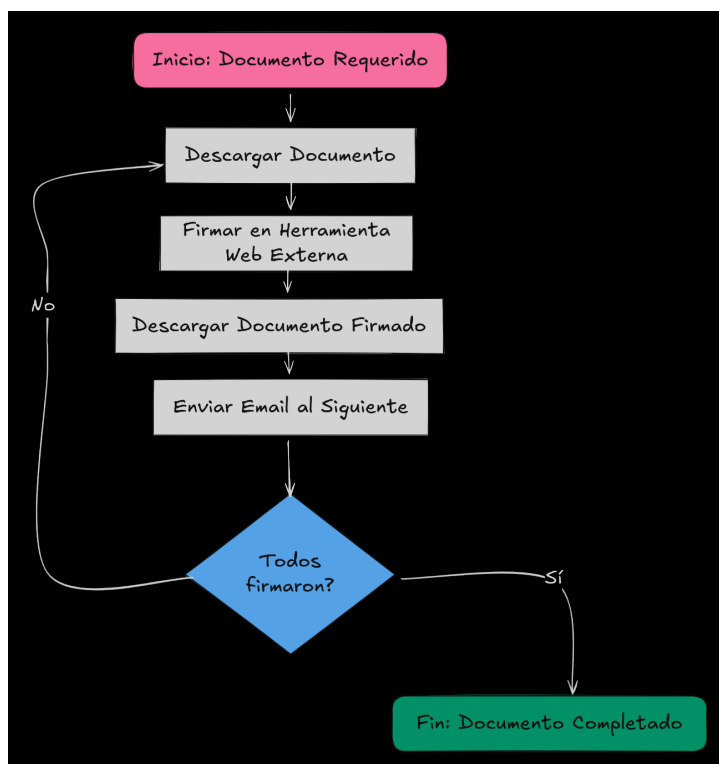


FIGURA 1.1: Diagrama de flujo del proceso actual de firma de documentos en la facultad.

1.2. Motivación y Justificación

Este trabajo se propone transformar el proceso manual y fragmentado descrito en la sección anterior en un flujo automatizado, aprovechando la infraestructura tecnológica con la que ya cuenta la facultad.

Frente a servicios externos con costos recurrentes elevados, una solución interna permite optimizar el uso del hardware, los sistemas de almacenamiento y la red institucional. El modelo de gestión propuesto organiza el ciclo de vida del documento en cuatro etapas:

- Carga única: el usuario (docente, alumno, personal de la facultad o tercero) sube el archivo PDF al sistema una sola vez.
- Definición de parámetros: se establecen los firmantes, el orden de las firmas y la ubicación visual de cada una en el documento.
- Orquestación automática: el sistema asume la gestión del proceso, enviando notificaciones, administrando los turnos y recolectando las firmas de forma ordenada.
- Resultado trazable: se genera un documento final único que contiene todas las firmas, almacenado de forma centralizada y con un registro de fecha y hora de cada intervención.

1.3. Objetivos y Alcance del Trabajo

1.3.1. Objetivo General

Diseñar, implementar y desplegar una solución “llave en mano” para la gestión y firma electrónica de documentos PDF en la facultad, definiendo la infraestructura y los procedimientos necesarios para su operación y mantenimiento.

1.3.2. Objetivos Específicos

- Evaluar las alternativas de código abierto mediante un estudio comparativo, a fin de seleccionar la solución que mejor se adapte a la organización de la facultad y a su integración con los servicios institucionales de identidad y almacenamiento.
- Diseñar la arquitectura de despliegue, configurar la infraestructura de virtualización, red y almacenamiento, y las políticas de respaldo y retención.
- Validar la solución técnica mediante una prueba piloto con usuarios reales, asegurando que la herramienta responda correctamente al flujo de trabajo propuesto.
- Elaborar el manual de mantenimiento y los procedimientos que garanticen la transferencia de conocimiento y la continuidad del servicio.

1.3.3. Alcance y Limitaciones

Este trabajo se centra en las tareas de infraestructura y administración de sistemas. El alcance comprende las siguientes áreas técnicas:

- Análisis de la topología de red existente de la facultad, desde el *reverse proxy* perimetral hasta el entorno de virtualización Proxmox y el contenedor LXC donde operan Coolify y Traefik.
- Despliegue contenerizado de la plataforma y de sus servicios auxiliares como contenedores Docker administrados por Coolify, ejecutados sobre un contenedor LXC provisionado por la facultad en Proxmox.
- Integración con el servidor de almacenamiento de objetos TrueNAS/MinIO, incluyendo la resolución de la conectividad necesaria entre la red privada de Docker y la red institucional.
- Configuración de la autenticación mediante Google OAuth 2.0, delegando la verificación de identidad al dominio institucional de Google Workspace.
- Integración con un servicio externo de *relay* SMTP para el envío de notificaciones transaccionales del sistema.
- Definición de políticas de respaldo y retención orientadas a la preservación de los datos transaccionales y los documentos firmados.
- Habilitación del acceso web cliente-servidor, garantizando la operabilidad de la plataforma desde navegadores estándar (escritorio y móviles) sin requerir instalación de software en los equipos.

Se establecen las siguientes limitaciones:

- El trabajo no incluye el desarrollo de software de aplicación desde cero, sino la implementación, integración y optimización de herramientas de código abierto existentes para cumplir con los requisitos de la facultad.
- El hardware disponible en el servidor (procesadores Xeon E5620 sin soporte para instrucciones AVX/AVX2) impone un límite a la versión de las imágenes Docker utilizables, lo que restringe la plataforma a Documenso v2.1.0. Este aspecto se documenta en el Capítulo 3.
- El certificado de firma utilizado es autofirmado y no posee validez ante terceros externos, por lo que el sistema se clasifica legalmente como firma electrónica conforme a la Ley 25.506.

1.4. Estado del Arte

El análisis inicial del sector permitió identificar una marcada diferencia entre las plataformas comerciales de tipo SaaS (*Software as a Service*) y las alternativas de código abierto. Si bien el mercado comercial ofrece herramientas robustas, los costos de licenciamiento y la dependencia respecto de servicios externos motivaron el descarte de soluciones propietarias para la facultad.

Las soluciones líderes en el mercado se comercializan mediante planes de suscripción con costo recurrente, generalmente estructurados por usuario o por nivel de servicio, lo que resulta económicamente inviable para una institución pública en el contexto actual. En la Tabla 1 se detallan los costos de los proveedores más relevantes.

Proveedor	Costo por Usuario (USD/mes)
DocuSign [1]	\$25.00
Dropbox Sign [2]	\$25.00
Adobe Sign [3]	\$22.19

TABLA 1: Comparativa de costos de soluciones SaaS de firma electrónica.

Considerando el escenario más económico entre los proveedores analizados (\$22,19 USD por licencia, Adobe Sign según la Tabla 1), la Ecuación (1.1) cuantifica el costo anual proyectado para un grupo de apenas 20 usuarios, resultado que sigue siendo significativo para el presupuesto de una institución pública:

$$(1.1) \text{ Costo Anual} = 20 \text{ usuarios} \times 22,19 \text{ USD} \times 12 \text{ meses} = 5,325,60 \text{ USD}$$

El ahorro en licencias que se logra con una solución de código abierto no es absoluto: al optar por un modelo *on-premise*, la facultad asume la responsabilidad de garantizar la disponibilidad del servicio, el mantenimiento de los servidores y la seguridad de la red y el almacenamiento

—tareas que en una solución comercial quedarían delegadas en el proveedor—. Como contrapartida, la adopción de código abierto permite reducir costos recurrentes, preservar la documentación dentro de la infraestructura de la UNT y desarrollar capacidades técnicas propias. La implementación realizada establece, además, una base operativa para futuras iniciativas de digitalización dentro de la facultad.

1.4.1. Alternativas de Código Abierto (*Open Source*)

Habiendo descartado las soluciones comerciales por su inviabilidad económica, la investigación se centró en plataformas de firma electrónica de código abierto que permitieran un despliegue *Self-Hosted*. El análisis del estado del arte arrojó dos alternativas principales con un nivel de madurez apto para producción: DocuSeal y Documenso.

Ambas plataformas se distribuyen bajo la licencia de código abierto AGPL-3.0 y ofrecen la funcionalidad base de firma electrónica. Sin embargo, el análisis de sus características técnicas y de licenciamiento reveló diferencias relevantes para el contexto universitario, resumidas en la Tabla 2.

Característica	Documenso (AGPL Core)	DocuSeal (AGPL Core)
Gestión de Roles (RBAC)	✓ Incluido	× Bloqueado (Solo PRO)
Inicio de sesión con Google OAuth	✓ Incluido	× Bloqueado (Solo PRO)
Personalización Visual (Branding)	✓ Incluido	× Bloqueado (Solo PRO)
Stack Tecnológico	Remix, Prisma, PostgreSQL	Ruby on Rails, PostgreSQL

TABLA 2: Comparativa de características en versiones de código abierto.

La versión comunitaria de DocuSeal restringe funcionalidades empresariales a la edición Pro [4]. La carencia de control de acceso basado en roles implica que, por defecto, todos los usuarios creados en la plataforma posean privilegios de administrador. Esta limitación compromete la seguridad de los documentos y hace inviable su adopción en un entorno institucional de múltiples usuarios.

Documenso cuenta, además, con una comunidad activa. Durante el despliegue, los intercambios mantenidos en su servidor de Discord resultaron útiles para identificar soluciones a problemas frecuentes y destrabar inconvenientes de instalación y operación.

Por estos motivos, se seleccionó Documenso como la plataforma definitiva para el trabajo. La decisión se apoya en tres criterios técnicos que resuelven las necesidades de la facultad:

1. Seguridad e Identidad: soporte nativo para la integración con Google OAuth, lo que permite utilizar las cuentas institucionales @herrera.unt.edu.ar dentro del esquema de autenticación ya administrado por la universidad.
2. Gestión Organizacional: la plataforma incluye una estructura de “Equipos” (*Teams*) que permite mapear fielmente el organigrama jerárquico de la institución. Es posible instanciar una organización central (FACET) y subdividirla en equipos departamentales (como el DEEC o el Departamento de Física), aislando los documentos y plantillas de cada área.

3. Ecosistema de Infraestructura: compatibilidad nativa con servidores de almacenamiento de objetos S3 (como MinIO) y con servicios de *relay* SMTP para el envío de notificaciones transaccionales del flujo de firma.

Documenso ofrece una solución sin costos de licenciamiento y con capacidad de crecimiento, sobre la cual se diseña la arquitectura que se detalla en los capítulos posteriores.

Capítulo 2

Marco Teórico y Tecnológico

Este capítulo presenta los fundamentos teóricos, legales y tecnológicos que sustentan la solución implementada: el marco criptográfico y normativo aplicable a la firma electrónica en la Argentina, y las capas de virtualización, orquestación y almacenamiento sobre las que se apoya el despliegue.

2.1. Modelos de Despliegue: *On-Premise* y *Self-Hosted*

Para la implementación de Documenso se adoptó una estrategia que combina los modelos *on-premise* y *self-hosted*. Aunque la industria suele emplear ambos términos de manera intercambiable, en este trabajo se utilizan con alcances distintos.

El carácter *on-premise* responde a que el software opera sobre hardware alojado físicamente en la facultad —clústeres de virtualización Proxmox y servidores TrueNAS ya disponibles—, lo que evita los costos recurrentes de una infraestructura en la nube (IaaS). La condición *self-hosted* implica que la propia institución aprovisiona, configura y mantiene la plataforma, en lugar de delegar esas responsabilidades a un proveedor de *Software as a Service* (SaaS). En conjunto, el modelo otorga control completo sobre las configuraciones del sistema, la arquitectura de red, las políticas de retención y el ciclo de vida de los documentos, a cambio de asumir la carga operativa del mantenimiento.

2.1.1. Software de Código Abierto y Licencia AGPL-3.0

Documenso se distribuye en su Community Edition bajo licencia AGPL-3.0 e incluye las funcionalidades centrales de firma documental y autoalojamiento [5, 6]. La AGPL-3.0 conserva las libertades habituales de la familia GPL —uso, estudio, modificación, distribución— y agrega la obligación de publicar las modificaciones también cuando el software se ofrece como servicio a través de una red [5, 7, 8].

Para este trabajo, la Community Edition resultó adecuada porque habilitó el despliegue sobre infraestructura propia, la revisión del comportamiento interno de la plataforma y la adaptación de la solución sin depender de una licencia comercial. La documentación de Documenso señala que las funciones empresariales y la posibilidad de mantener modificaciones privadas corresponden a la Enterprise Edition [5].

2.2. Fundamentos de Seguridad de la Información

El diseño de la plataforma se enmarca en la arquitectura de seguridad OSI X.800 [9]. Los servicios relevantes para un sistema de gestión documental sobre infraestructura institucional son: confidencialidad, integridad, disponibilidad, autenticación, control de acceso y no repudio.

La solución implementada materializa cada servicio mediante componentes concretos del *stack*: la autenticación se delega en Google OAuth 2.0 con cuentas institucionales; el control de acceso opera bajo un modelo Role-Based Access Control (RBAC) dentro de Documenso; la integridad y el no repudio se sustentan en el sellado criptográfico del documento; la confidencialidad se resguarda mediante cifrado TLS en tránsito y restricción del acceso al dominio institucional; y la disponibilidad se apoya en la redundancia RAID 6, los respaldos automatizados hacia MinIO y la separación entre cómputo y almacenamiento.

2.3. Mecanismos Técnicos de Firma Digital

La implementación de los servicios de integridad y no repudio se basa en criptografía de clave pública. Este esquema utiliza un par de claves vinculadas matemáticamente: una clave privada (KR), de conocimiento exclusivo de su propietario, y una clave pública (KU), distribuida libremente para verificación.

2.3.1. Proceso de Generación de Firma

El flujo técnico empleado por Documenso [10] para sellar un documento consiste en calcular un *hash* criptográfico del contenido, cifrarlo con la clave privada (KR) asociada al certificado de la plataforma e incrustar la firma resultante en la estructura del archivo PDF. La Figura 2.1 ilustra el procedimiento general de generación y verificación de firma digital.

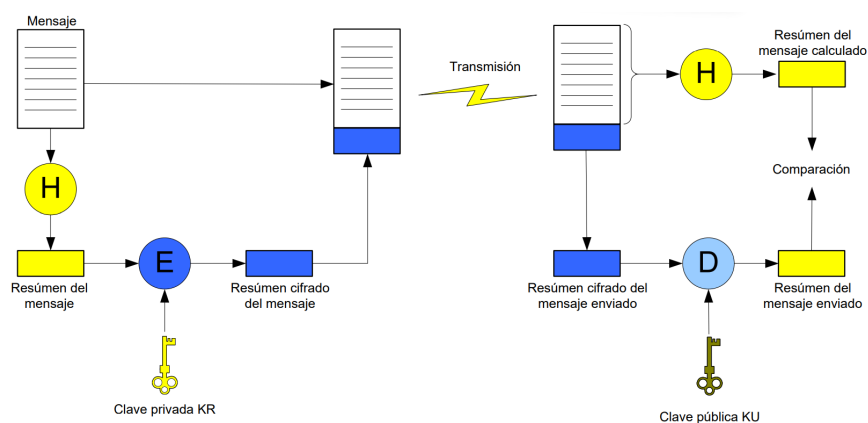


FIGURA 2.1: Proceso de generación y verificación de firma digital. Extraído de [9].

La firma se aplica a nivel de plataforma y no por cada firmante individual. Las acciones de cada usuario —trazos de firma, textos, casillas de verificación— quedan registradas en el sistema;

una vez que todos los participantes completan su intervención, el documento se sella criptográficamente bajo el certificado de la instancia institucional. Este sellado proporciona dos garantías: integridad —el documento no fue alterado desde el momento de la firma— y autenticidad —el PDF fue firmado por el titular del certificado—. Si el archivo se modifica con posterioridad, la firma deja de ser válida y los lectores PDF advierten sobre la alteración [10].

2.3.2. Certificados de Clave Pública y CA Interna

Debido a que la *KU* es de acceso general, se emplean certificados digitales que vinculan la clave con su titular mediante la firma de una Autoridad de Certificación (CA).

En esta implementación se utiliza un certificado autofirmado generado dentro del entorno institucional. La documentación de Documenso distingue entre confianza del certificado y validez criptográfica de la firma [10, 11]: al no pertenecer a una cadena de confianza comercial, los lectores PDF pueden advertir que el emisor no fue validado por una tercera parte, pero esa condición no invalida la integridad criptográfica del documento ni impide su uso en circuitos administrativos internos.

En este trabajo, el material criptográfico se administró como un contenedor PKCS#12 codificado en Base64 e inyectado mediante variables de entorno, tal como se documenta en el Apéndice B.5. La restricción legal depende del certificado y no de la plataforma: Documenso soporta certificados X.509 emitidos por autoridades certificadoras licenciadas [11], de modo que el certificado local podría reemplazarse sin modificar la infraestructura base si la institución requiriera validación ante terceros.

2.4. Marco Legal Argentino: Ley 25.506

La validez jurídica de la solución se encuadra en la Ley 25.506 de Firma Digital [12], que establece una distinción fundamental:

- Firma Digital (Art. 2 y Art. 9): el Artículo 2 define a la firma digital como el resultado de aplicar a un documento digital un procedimiento matemático que requiere información de exclusivo conocimiento del firmante, bajo su absoluto control. La verificación debe permitir identificar al firmante y detectar cualquier alteración del documento posterior a la firma. El Artículo 9 exige, entre otros requisitos, que el certificado haya sido emitido o reconocido por un certificador licenciado.
- Firma Electrónica (Art. 5): se entiende por firma electrónica al conjunto de datos electrónicos integrados, ligados o asociados de manera lógica a otros datos electrónicos, utilizado por el signatario como medio de identificación, que carezca de alguno de los requisitos legales para ser considerada firma digital. En caso de ser desconocida, corresponde a quien la invoca acreditar su validez.

Dado que el sistema utiliza una Autoridad Certificante propia sin licencia estatal —lo que incumple el requisito del Art. 9—, la solución se clasifica legalmente como firma electrónica. Su validez administrativa es, no obstante, plena dentro del ámbito interno de la organización, con fundamento en el acuerdo entre las partes y en la trazabilidad técnica que proporciona la plataforma.

2.5. Infraestructura de Virtualización y Despliegue

La arquitectura del despliegue se organizó en capas para desacoplar los servicios de la administración directa del hardware y simplificar las tareas de operación y recuperación.

2.5.1. Plataforma de Virtualización y Orquestación

La infraestructura de cómputo se estructura en tres niveles. En la base, Proxmox VE [13, 14] administra los recursos físicos del servidor y aloja un contenedor LXC. Se optó por LXC en lugar de una máquina virtual KVM porque comparte el kernel del anfitrión, lo que reduce la sobrecarga de CPU y memoria —condición relevante dado el procesador Xeon E5620 disponible—.

Dentro de ese contenedor se ejecuta Docker [15] y, sobre él, opera Coolify [16]: una PaaS autoalojable de código abierto que centraliza la gestión de despliegues, variables de entorno, dominios y respaldos en un panel web unificado, sin ceder el control de la infraestructura a un proveedor externo. Este conjunto —LXC, Docker y Coolify— ya se encontraba aprovisionado por el área de sistemas de la FACET como nodo de orquestación base de la facultad.

El aporte de este trabajo consistió en desplegar el sistema de firmas sobre esa plataforma pre-existente. Documenso distribuye su *stack* como un conjunto de servicios definidos en un archivo `docker-compose.yml` [6], lo que exige un entorno containerizado con Docker como requisito mínimo. La justificación de la adopción de Coolify y los detalles de configuración se desarrollan en el Capítulo 3.

2.5.2. Reverse Proxy y Enrutamiento Dinámico (Traefik)

Traefik actúa como *reverse proxy* dentro de la red de contenedores, enrutando dinámicamente el tráfico entrante (HTTP/HTTPS) hacia los contenedores internos correspondientes según las reglas de dominio declaradas [17]. Este desacoplamiento entre la dirección pública del servicio y su ubicación real dentro de la infraestructura permite publicar múltiples aplicaciones desde un único punto de entrada.

2.6. Arquitectura de Almacenamiento de Objetos

La persistencia de los documentos digitales se resuelve mediante una arquitectura que separa la gestión física de los discos del acceso lógico a los archivos. Esta capa se vincula directamente con la disponibilidad definida en la Sección 2.2, ya que de ella depende que los documentos permanezcan accesibles cuando se los requiere. Se utiliza TrueNAS Community Edition [18] como plataforma de almacenamiento base y MinIO [19] como motor de objetos compatible con el protocolo S3.

2.6.1. Gestión Física y Resiliencia (TrueNAS y ZFS)

TrueNAS Community Edition [18] gestiona la capa física mediante ZFS sobre un arreglo RAID 6 [20]. Esta combinación proporciona resiliencia ante fallas de disco y mitiga la degradación silenciosa de datos (*bit-rot*). No obstante, la redundancia de hardware no sustituye una política de

respaldos lógicos: protege frente a fallas físicas, pero no frente a errores de software, eliminaciones accidentales o manipulación indebida de los datos [20, 21].

2.6.2. MinIO: Almacenamiento de Objetos S3

Sobre la infraestructura de TrueNAS se utiliza MinIO para la gestión de los documentos mediante el protocolo S3. Este componente es clave en el diseño por dos motivos:

- Separación entre aplicación y almacenamiento: Documenso guarda y recupera documentos a través de una interfaz S3 sin depender de carpetas locales del servidor de aplicación. Esto permite trasladar o ampliar el almacenamiento sin modificar la lógica del sistema y habilita, a futuro, esquemas de respaldo híbrido entre infraestructura local y servicios externos compatibles.
- Retención e inmutabilidad: mediante Object Lock, MinIO aplica políticas de conservación que impiden la modificación o eliminación prematura de los documentos almacenados [22]. Para los documentos firmados, esta capacidad preserva evidencia de que el archivo no fue alterado después de su incorporación al repositorio.

Mientras ZFS se ocupa de la redundancia y la integridad física en los discos, MinIO proporciona el control necesario para garantizar la persistencia e inmutabilidad de los archivos firmados a nivel de software.

2.6.3. Principios de Respaldo y Retención de Datos

Un despliegue *on-premise* exige que la institución asuma la responsabilidad completa sobre la preservación de los datos, dado que no existe un proveedor externo que gestione copias de seguridad o garantice la recuperación ante desastres [21]. Una política de respaldo define qué información debe copiarse, con qué frecuencia, dónde se conserva y bajo qué procedimiento puede recuperarse. Una política de retención establece durante cuánto tiempo se mantienen esas copias y en qué momento pueden eliminarse de acuerdo con criterios operativos, normativos o de capacidad [21, 23].

En una plataforma documental, ambas políticas deben contemplar al menos dos dominios con necesidades distintas:

- Datos transaccionales: usuarios, permisos, estados de firma y registros de auditoría. Su pérdida comprometería la trazabilidad operativa del sistema.
- Repositorio documental: archivos cargados y sus versiones firmadas, cuya preservación requiere reglas claras de conservación e inmutabilidad.

La implementación concreta de estas políticas —frecuencias de ejecución, períodos de retención y procedimientos de restauración— se documenta en el Capítulo 3.

2.7. Gestión de Identidad y Autorización: Google OAuth 2.0

El control de acceso a la plataforma se delega en Google Workspace mediante el protocolo OAuth 2.0 [24]. Esto desacopla la autenticación de la lógica de la aplicación y evita que el

servidor local almacene credenciales. El flujo de redirección e intercambio de tokens asegura que Documenso solo reciba la información estrictamente necesaria para validar la identidad institucional del usuario.

Para completar la integración se registra la instancia en Google Cloud y se obtiene un Client ID y un Client Secret. El detalle de las variables de entorno y los parámetros concretos de configuración se presenta en el Apéndice B.

Capítulo 3

Diseño e Implementación de la Infraestructura

El capítulo documenta el diseño y la implementación de la solución de firma electrónica: la infraestructura base, la validación del contenedor LXC y la PaaS Coolify, los servicios contenerizados que integran la plataforma, su vinculación con PostgreSQL y MinIO dentro de la red institucional, y la integración con Google OAuth 2.0 y el *relay* SMTP de Brevo.

3.1. Topología de Red y Arquitectura General

La topología del sistema se organizó en capas para preservar el aislamiento entre componentes, controlar la exposición de servicios y sostener la disponibilidad operativa. En este esquema, el tráfico proveniente de Internet (nivel WAN) ingresa a la red de área local de la facultad (nivel LAN), donde se ubican los contenedores de Docker, entre ellos Documenso y PostgreSQL.

La Figura 3.1 resume el recorrido de una petición web externa a través de tres niveles:

1. Capa de Borde (*Reverse Proxy* Nginx): las peticiones dirigidas al dominio institucional `documentos.facet.unt.edu.ar` son recibidas inicialmente por el *reverse proxy* perimetral Nginx de la facultad. Este componente gestiona la terminación SSL mediante certificados de Let's Encrypt y reenvía el tráfico HTTP/HTTPS hacia la red interna, tal como puede apreciarse en las Figuras 3.2 y 3.3.
2. Capa PaaS (Coolify): el tráfico es dirigido a un contenedor Linux (LXC) dentro del hipervisor Proxmox VE. En esta capa opera la PaaS Coolify, que gestiona una instancia interna de Traefik que enruta dinámicamente las peticiones hacia el servicio correspondiente.
3. Capa de Servicios (Contenedores Docker): finalmente, la solicitud llega al contenedor de la aplicación Documenso. Este se comunica de forma interna (mediante redes privadas de Docker) con su base de datos PostgreSQL y el servicio contenerizado Browserless, y a través de la red local con el servidor de almacenamiento TrueNAS/MinIO.

En este esquema, ni la dirección IP del servidor ni los puertos internos de Docker quedan expuestos directamente a Internet. Todo el flujo de datos atraviesa el *reverse proxy* perimetral de la facultad, que actúa como único intermediario entre los usuarios externos y la red local.

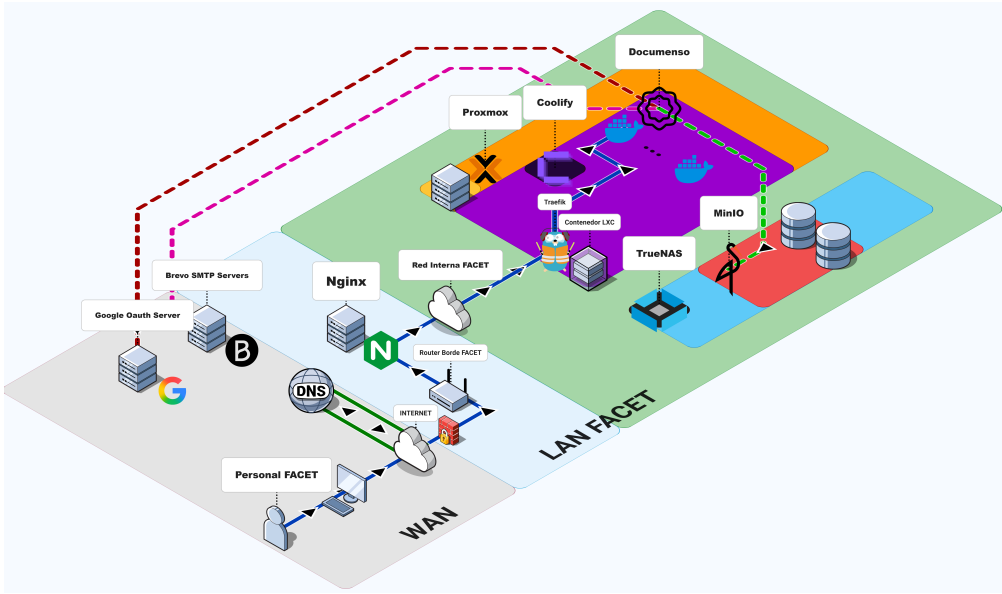


FIGURA 3.1: Diagrama de flujo de tráfico y arquitectura de capas del sistema de firma electrónica.

Una versión ampliada de este diagrama se encuentra disponible en la Sección A.1 del Apéndice A.

Name	×	Headers	Preview	Response	Initiator	Timing	>>
session-json							
__manifest?p=%2F&p=%2Fin							
document.search?input=%7B...							
document.inbox.getCount,org...							
organisation.internal.getOrgan...							
session-json							
organisation.internal.getOrgan...							
session-json							
organisation.internal.getOrgan...							
__manifest?p=%2Fo%2Ffacet&...							
envelope.item.getManyByToke...							
signed							
session-json							
organisation.internal.getOrgan...							
		Request URL	https://documentos.facet.unt.edu.ar/api/auth/session-json				
		Request Method	GET				
		Status Code	200 OK				
		Remote Address	200.45.169.100:443				
		Referrer Policy	strict-origin-when-cross-origin				
		Response headers					
		Alt-Svc	h3=":443"; ma=2592000				
		Content-Encoding	gzip				
		Content-Type	application/json				
		Date	Sat, 04 Apr 2026 13:37:13 GMT				
		Server	nginx				
		Vary	Accept-Encoding				
		X-Request-Id	3b455f85-3996-4ae7-ac6f-5b22a7ed88e0				

FIGURA 3.2: Cabeceras HTTP de la respuesta del dominio institucional, con Server: nginx.

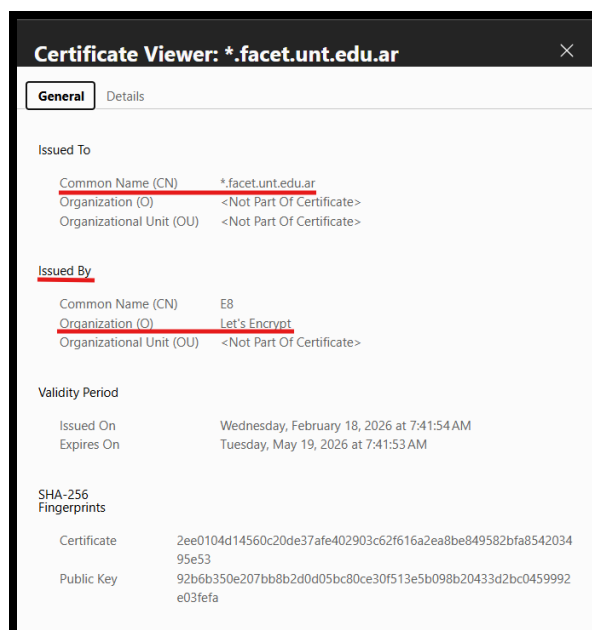


FIGURA 3.3: Certificado TLS presentado por el dominio institucional, emitido para *.facet.unt.edu.ar por Let's Encrypt.

3.2. Infraestructura Base y Entorno PaaS

La implementación no partió de un despliegue desde cero, sino de una infraestructura base previamente aprovisionada por el área de sistemas de la FACET. En esta etapa, el trabajo consistió en auditar ese entorno, comprobar que resultara suficiente y usarlo como base para desplegar Documenso sobre la PaaS ya disponible.

3.2.1. Auditoría del Entorno LXC y Configuración Base

La etapa de despliegue comenzó con la auditoría técnica de un contenedor LXC con Ubuntu 22.04 LTS, aprovisionado por la facultad sobre el hipervisor Proxmox VE. El contenedor disponía de 8 núcleos lógicos, 8 GB de memoria RAM, un disco raíz de 50 GB y un volumen de montaje adicional de 160 GB, recursos que superan holgadamente los requerimientos combinados de Coolify [25] y Documenso [26] (4 vCPUs, 4 GB de RAM y 50 GB de almacenamiento en conjunto).

El entorno se configuró como un contenedor no privilegiado para aislar los procesos del nivel de ejecución del servidor físico y mitigar el impacto ante un incidente de seguridad. A su vez, se habilitó el anidamiento (nesting=1), requisito para que el motor de Docker instancie redes y orqueste sus propios subcontenedores dentro del LXC.

3.2.2. Justificación del Uso de Coolify como PaaS

Inicialmente, se evaluó desplegar Documenso sobre una máquina virtual (VM) o un contenedor LXC dedicados en Proxmox. Esa alternativa se descartó porque implicaba montar y mantener

manualmente toda la infraestructura asociada: un *reverse proxy* adicional para publicar la aplicación, el enrutamiento hacia los servicios auxiliares y el despliegue separado de PostgreSQL, Browserless y el *proxy* hacia MinIO. En cambio, se aprovechó la instancia de Coolify ya disponible en la infraestructura. Esto permitió centralizar la operación sin incorporar una nueva VM ni sobredimensionar recursos.

En la práctica, la PaaS resolvió tareas que, en un despliegue manual, hubieran incrementado la carga operativa y el margen de error:

- Gestión de red: enrutamiento interno dinámico mediante Traefik, que recibe el tráfico del *reverse proxy* perimetral y lo dirige al puerto correspondiente de cada contenedor.
- Aislamiento: segmentación de servicios en redes virtuales de Docker para mantener separados los componentes.
- Mantenimiento: interfaz centralizada para inyectar variables de entorno, gestionar volúmenes y consultar logs en tiempo real.
- Respaldo: integración con almacenamiento S3 compatible, utilizada más adelante para automatizar los respaldos de PostgreSQL sobre MinIO.

En esta arquitectura conviene diferenciar el rol de cada capa. Proxmox VE corre sobre el servidor físico, cuya IP en la LAN es 192.168.X.Y. Su función consiste en asignar recursos de cómputo, memoria y almacenamiento a los contenedores y máquinas virtuales que administra. El hipervisor no participa del enrutamiento ni del procesamiento del tráfico HTTP/HTTPS de las aplicaciones.

El contenedor LXC, en cambio, se conecta a la LAN de la facultad mediante un puente virtual (bridge) configurado en Proxmox y obtiene su propia IP, 192.168.X.Z, como un nodo independiente de la red local. Para Docker y Traefik, ese LXC actúa como sistema anfitrión: allí se publican los puertos 80 y 443 y desde allí se resuelve el enrutamiento interno hacia los contenedores. Así, Proxmox queda en el plano de gestión de la infraestructura, mientras que el LXC concentra el plano de datos de la aplicación.

La Figura 3.4 muestra esta arquitectura anidada y el modo en que Coolify gestiona el enrutamiento interno mediante Traefik, manteniendo a Documenso separado de otros servicios desplegados sobre ese LXC.

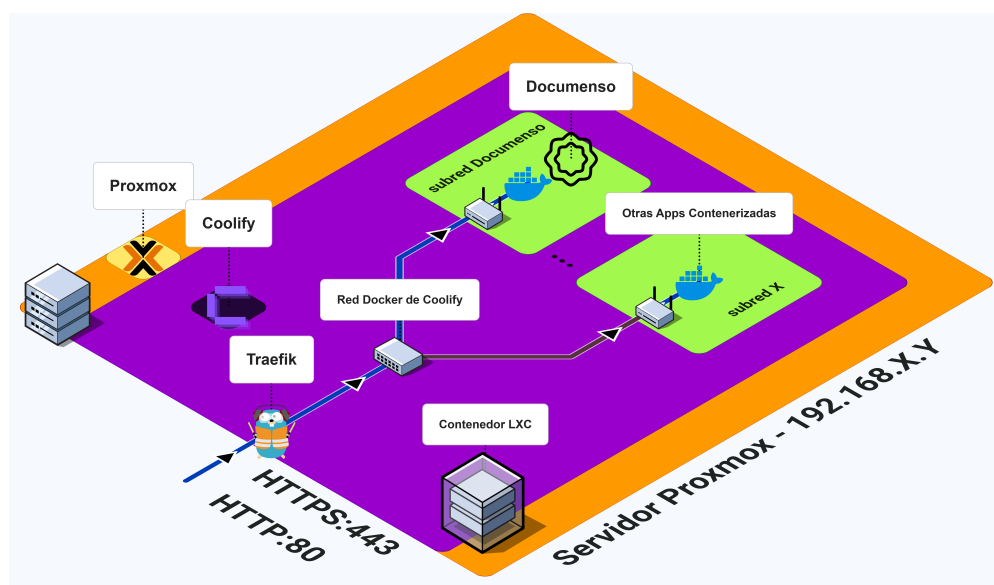


FIGURA 3.4: Arquitectura anidada y segmentación de redes virtuales Docker dentro del contenedor LXC base.

Una versión ampliada de este diagrama se encuentra disponible en la Sección A.2 del Apéndice A.

3.3. Configuración de Servicios y Despliegue de Documenso

Una vez validada la capa base, el trabajo avanzó hacia el despliegue de Documenso y sus servicios auxiliares. Para definir esta implementación se tomaron como punto de partida dos referencias: la plantilla oficial de Coolify para Documenso [27] y la guía de despliegue con Docker Compose publicada en la documentación de Documenso [28]. Ninguna de las dos resolvía por sí sola las necesidades del entorno institucional. A medida que avanzaron las pruebas aparecieron requisitos y problemas no cubiertos inicialmente, lo que obligó a investigar ajustes adicionales e incorporar servicios complementarios. La configuración final surgió de esa combinación entre referencias previas y decisiones tomadas durante la fase de prueba, que se describen en las subsecciones siguientes.

La arquitectura resultante quedó organizada en cuatro servicios contenerizados: PostgreSQL para la persistencia transaccional, Browserless como motor de renderizado auxiliar, Documenso como núcleo de aplicación y un *proxy* Socat para la conectividad con MinIO. Esta separación permitió distribuir responsabilidades entre persistencia, renderizado, lógica de aplicación y acceso al almacenamiento, lo que simplificó tanto el despliegue como el diagnóstico de fallas y la administración posterior. El detalle ampliado de los diagramas de topología y de la arquitectura interna de la PaaS se presenta en el Apéndice A. La composición Docker completa utilizada como base del despliegue se incluye en el Apéndice B.1.

3.3.1. Persistencia de Estado (PostgreSQL)

El primer servicio en inicializarse fue PostgreSQL, declarado en la composición bajo el identificador `database`. En este despliegue, PostgreSQL almacenó los datos de la aplicación necesarios para su funcionamiento diario: usuarios, equipos de trabajo, relaciones de permisos, metadatos de documentos, estados del flujo de firma y registros de auditoría [29].

Cuando Documenso se configura con almacenamiento S3 compatible, los archivos PDF no se guardan en PostgreSQL, sino en MinIO [30, 29]. La base de datos conserva la información asociada a esos archivos, como identificadores, estados del circuito de firma, marcas temporales y referencias a su ubicación dentro del *bucket*. Esta separación resulta conveniente en producción porque evita que la base crezca por almacenar documentos y deja a PostgreSQL dedicado a los datos propios de la aplicación [30].

PostgreSQL también almacenó la estructura de usuarios y permisos sobre la que Documenso aplica su modelo RBAC (Role-Based Access Control). Por eso, su arranque se priorizó antes que el del resto de los servicios y se definió un volumen persistente nominal, `documensoprod-postgresql-data`, de modo que la información se mantuviera aun cuando el contenedor fuera reiniciado, recreado o actualizado desde la PaaS.

Para establecer el esquema de permisos, Documenso exige promover al primer administrador ejecutando una sentencia SQL directamente sobre PostgreSQL, ya que las cuentas creadas desde la interfaz se registran como usuarios comunes [31]. A partir de ese punto, el administrador puede otorgar el rol a otros usuarios desde la interfaz web. El procedimiento de conexión y ejecución se detalla en el Apéndice B.2.

Para garantizar la estabilidad del arranque, el servicio incorporó un *healthcheck* con `pg_isready`, de modo que Documenso solo iniciara cuando PostgreSQL estuviera listo para aceptar conexiones. La composición completa del servicio, con el detalle de estas directivas, se incluye en el Apéndice B.1.

3.3.2. Motor de Renderizado Auxiliar (Browserless)

Browserless no formó parte del diseño inicial del despliegue. Se incorporó después de detectar un defecto funcional durante las pruebas de integración. Una vez completado el primer ciclo de firma de extremo a extremo, se observó que los documentos recorrían correctamente todas las etapas del flujo —carga, asignación de firmantes, notificación por correo y aplicación de la firma—, pero quedaban en estado `PENDING` en lugar de pasar a `COMPLETED`. El análisis de los registros del contenedor mostró que el job interno `internal.seal-document` fallaba al intentar generar el certificado de firma en PDF, con el siguiente mensaje de error:

```
Failed to get certificate PDF
browserType.launch: Executable doesn't exist at
/home/nodejs/.cache/ms-playwright/chromium_headless_shell-1169/
chrome-linux/headless_shell
```

Browserless provee un entorno Chromium remoto accesible por *WebSocket*, lo que permite a la aplicación delegar el renderizado sin depender de binarios locales [32]. La causa del problema ya había sido documentada en el repositorio oficial de Documenso. El issue #2060, identificado

durante el intercambio con la comunidad en su servidor de Discord, reportó que la imagen Docker no incluye Chromium, aun cuando la generación del certificado de firma depende de ese binario [33]. A partir de ese antecedente, se revisaron los hilos de la comunidad hasta llegar al issue #1634, donde se discutía una solución aplicable a este despliegue [34].

Un análisis del código fuente compartido en esas discusiones mostró que el módulo encargado de generar el certificado PDF sigue dos caminos: si la variable de entorno `NEXT_PRIVATE_BROWSERLESS_URL` está definida, la aplicación envía el renderizado a una instancia externa accesible por *WebSocket*; en caso contrario, intenta usar una instalación local de Chromium. Como ese ejecutable no está disponible en la imagen oficial, Documenso no puede completar el sellado solo con su contenedor.

La solución adoptada fue desplegar la imagen `browserless/chrome` [35] como servicio auxiliar dentro de la composición Docker. En este esquema, Documenso se conecta por *WebSocket* al servicio Browserless mediante la variable `NEXT_PRIVATE_BROWSERLESS_URL`, configurada con el endpoint `ws://browserless:3000`. De ese modo, el certificado de firma pasó a renderizarse en el contenedor Browserless, sin modificar la imagen oficial de Documenso.

La configuración del servicio —cuya composición completa se incluye en el Apéndice B.1— incorporó los siguientes parámetros de control:

- `MAX_CONCURRENT_SESSIONS=5`: cinco sesiones de Chrome proyectan un consumo pico de aproximadamente 1,3 GB, lo que deja un margen de seguridad del 30 % dentro del límite de 2 GB asignado al contenedor.
- `MAX_QUEUE_LENGTH=20`: cantidad máxima de solicitudes en espera cuando las sesiones concurrentes están ocupadas.
- `TIMEOUT=60000`: cada sesión se limita a 60 segundos para evitar procesos de renderizado bloqueados.
- `deploy.resources.limits`: restricción a 2 CPU virtuales y 2 GB de memoria, de modo que el renderizado no desplace recursos del resto del *stack*.
- `extra_hosts` con valor `host-gateway`: permite que Browserless resuelva el dominio de Documenso dentro de la red del despliegue, sin depender de una ruta externa.

Si bien esta solución garantizó la generación del certificado PDF sin alterar la imagen oficial, requirió aprovisionar recursos dedicados de memoria, CPU y supervisión para un servicio adicional dentro del despliegue.

3.3.3. Servicio Principal de Documenso

El contenedor principal de Documenso expuso la interfaz web, concentró la lógica de negocio y coordinó el flujo de firma. También reunió la mayor parte de la parametrización del despliegue, al depender de credenciales, rutas internas, dominios públicos y parámetros de integración con los servicios auxiliares.

La configuración se resolvió mediante inyección de variables de entorno desde Coolify, práctica que evita codificar de forma rígida credenciales y parámetros dentro del archivo `docker-compose.yml`. La composición completa del servicio y el inventario de variables se documentan en el Apéndice B.

La imagen se fijó en `v2.1.0` en lugar de usar la etiqueta `latest`. Al evaluar una actualización a versiones posteriores (`v2.2.0` y superiores), el contenedor terminaba inmediatamente con el error:

```
Illegal instruction (core dumped)
```

Desde la versión 2.2.0, Documenso incorpora dependencias internas (en particular Sharp/libvips) que requieren instrucciones AVX/AVX2 del procesador [36]. El servidor físico de la FACET utiliza procesadores Intel Xeon E5620, de la familia Westmere, que carecen de soporte AVX. Esta ausencia se verificó con `lscpu` en el contenedor LXC, donde las flags `avx` y `avx2` no estaban presentes. En consecuencia, el despliegue quedó anclado en `v2.1.0`, última versión operable sobre el hardware disponible. El requerimiento de AVX/AVX2 para ramas posteriores quedó además documentado en el issue #2292 del repositorio oficial [36]. Esta restricción impone una deuda técnica temporal, ya que impide adoptar parches de seguridad y mejoras funcionales de versiones superiores; en el corto plazo, debe priorizarse la migración a un servidor con soporte AVX.

Dentro del despliegue, Documenso resolvió sus dependencias internas mediante los nombres de servicio `database` y `browserless`, mientras que Traefik expuso únicamente la interfaz web. En la configuración del dominio de la aplicación, la forma `https://documentos.facet.unt.edu.ar:3000` no representó la apertura del puerto 3000 hacia el exterior, sino una regla declarativa consumida por el proxy interno para enrutar el tráfico HTTPS entrante hacia ese puerto dentro del contenedor. La Figura 3.5 resume esta organización y justifica la nomenclatura adoptada en el diagrama de red.

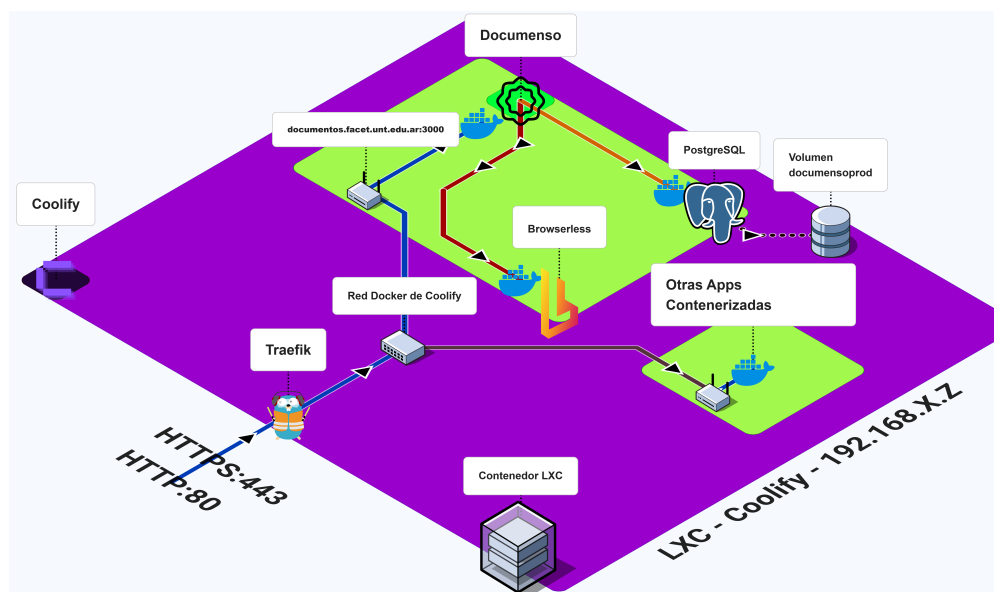


FIGURA 3.5: Comunicación entre servicios contenerizados y enrutamiento de Traefik dentro de la red privada del despliegue administrado por Coolify.

Una versión ampliada de este diagrama se encuentra disponible en la Sección A.3 del Apéndice A.

3.4. Conectividad con el Almacenamiento de Objetos (Proxy Socat)

La conectividad entre Documentoso y MinIO estuvo condicionada por una restricción de diseño de la aplicación: el sistema centraliza la configuración de su almacenamiento S3 a través de una única variable de entorno denominada `NEXT_PRIVATE_UPLOAD_ENDPOINT`. Al disponer de un único parámetro de configuración, la arquitectura obligó a que tanto el navegador web del usuario (tráfico externo) como el propio servidor interno de la aplicación (tráfico interno) utilizaran una única ruta para acceder al almacenamiento. La solución adoptada fue establecer `minio.facet.unt.edu.ar` como el dominio público unificado.

Bajo esta configuración, la plataforma opera con dos flujos de datos diferenciados. El primer flujo corresponde a los recursos gráficos de la interfaz, como las imágenes de branding. En este caso, Documentoso genera URLs prefirmadas que el navegador consume directamente contra MinIO [30, 29]. Si la variable de entorno hubiera contenido una dirección privada de la LAN, como `192.168.X.W:9000`, este flujo habría fallado porque esa dirección no es enrutable desde Internet y, por lo tanto, no puede ser alcanzada por clientes externos. A esto se suma que las políticas de seguridad del navegador bloquean peticiones desde un origen HTTPS hacia un destino HTTP plano (Mixed Content).

La publicación del dominio `minio.facet.unt.edu.ar` a través del *reverse proxy* perimetral resolvió este impedimento, pero exigió dos ajustes. Primero, fue necesario configurar el parámetro `client_max_body_size` en Nginx. Documentoso admite cargas de hasta 50 MB por archivo (valor por defecto de la aplicación), por lo que se fijó ese mismo límite para documentos.facet.

unt.edu.ar; para minio.facet.unt.edu.ar se estableció 75 MB como margen de seguridad para el peor caso operativo: durante el sellado, la incrustación de firma criptográfica, certificados y metadatos incrementa el tamaño del PDF. De este modo, si un usuario carga un archivo cercano a 50 MB, el sistema conserva holgura suficiente para almacenar la versión firmada sin rechazar la petición. Segundo, en el flujo directo del navegador, fue obligatorio autorizar en MinIO el origen institucional <https://documentos.facet.unt.edu.ar>. Esto permitió satisfacer las políticas CORS (Cross-Origin Resource Sharing) de MinIO para las peticiones iniciadas desde el navegador. La Figura 3.6 evidencia esta secuencia.

▼ General	
Request URL	https://minio.facet.unt.edu.ar/documento-prod/thm=AWS4-HMAC-SHA256&X-Amz-Content-Sha256=DhzphlaLgVpctk8P36Sp%2F20260424%2Fus-east-1%2Fs3%2F424T005414Z&X-Amz-Expires=3600&X-Amz-SignedHeaders=dfc26a7984c6d211065612ac6dec38&X-Amz-Signature=AAA%3D%3D&x-amz-sdk-checksum-algorithm=SHA256
Request Method	OPTIONS
Status Code	204 No Content
Remote Address	200.45.169.100:443
Referrer Policy	strict-origin-when-cross-origin
▼ Response headers	
Access-Control-Allow-Credentials	true
Access-Control-Allow-Headers	content-type
Access-Control-Allow-Methods	PUT
Access-Control-Allow-Origin	https://documentos.facet.unt.edu.ar

(A) Solicitud preflight OPTIONS del protocolo CORS hacia MinIO.

▼ General	
Request URL	https://minio.facet.unt.edu.ar/documento-prod/eoux4wrm9d5thm=AWS4-HMAC-SHA256&X-Amz-Content-Sha256=UNSIGNED-Payload&X-Amz-Expires=3600&X-Amz-SignedHeaders=dfc26a7984c6d211065612ac6dec38&X-Amz-Signature=AAA%3D%3D&x-amz-sdk-checksum-algorithm=CRC32&x-id=id-1
Request Method	PUT
Status Code	200 OK
Remote Address	200.45.169.100:443
Referrer Policy	strict-origin-when-cross-origin
▼ Response headers	
Accept-Ranges	bytes
Access-Control-Allow-Credentials	true
Access-Control-Allow-Origin	https://documentos.facet.unt.edu.ar
Access-Control-Expose-Headers	Date, Etag, Server, Connection, Accept-Ranges, Content-Range, Content-Length, Content-Type, Content-Disposition, Last-Modified

(B) Petición PUT de carga del recurso gráfico hacia el *bucket* S3.

FIGURA 3.6: Evidencia del flujo directo entre la interfaz web de Documenso y MinIO durante la carga de recursos gráficos.

A diferencia del caso anterior, el segundo flujo es indirecto y abarca la carga, descarga y firma de documentos PDF. En este caso, el navegador no interactúa directamente con MinIO, sino que

se comunica con la API del servidor interno de Documenso, y es este *backend* quien realiza las operaciones correspondientes sobre el almacenamiento S3.

La Figura 3.7 presenta la evidencia de este comportamiento desde la perspectiva del cliente. Durante la descarga de un documento PDF firmado, el navegador emite una petición GET hacia un *endpoint* de la API de la aplicación, en lugar de consumir una URL prefirmada del almacenamiento de objetos. Esto demuestra que el servidor interno actúa como intermediario entre el cliente y MinIO.

session-json	▼ General	
limits		
organisation.int...	Request URL	https://documentos.facet.unt.edu.ar/api/files/envelope-item/envelope_item_tzyrvwycdfkeeioh/download/sign...
signed	Request Method	GET
document.searc...	Status Code	200 OK
document.inbo...	Remote Address	200.45.169.100:443
document.findD...	Referrer Policy	strict-origin-when-cross-origin
document.searc...		
document.inbo...	▼ Response headers	
document.findD...	Alt-Svc	h3=":443"; ma=2592000
10 requests 1.4 MB	Cache-Control	no-cache, no-store, must-revalidate

FIGURA 3.7: Descarga indirecta de un documento PDF firmado a través de la API de Documenso.

Para enlazar la red virtual de contenedores con el servidor de almacenamiento TrueNAS, se implementó un contenedor basado en socat dentro de Coolify. El contenedor expone el puerto 9000 dentro de la red interna de Docker, donde la utilidad queda a la escucha mediante TCP4-LISTEN:9000, y redirige el tráfico TCP hacia la IP del servidor de almacenamiento TrueNAS (TCP4:192.168.X.W:9000). En esta topología, Traefik recibe las peticiones dirigidas a minio.facet.unt.edu.ar y deriva el tráfico entrante hacia ese puerto del contenedor socat, que actúa como puente de red hacia TrueNAS. La composición de este servicio se incluye en el Apéndice B.1. El servicio se desplegó de forma independiente del *stack* principal de Documenso para mantener un túnel reutilizable hacia almacenamiento S3 en futuros proyectos de la facultad.

El diseño final hizo que ambos flujos convergieran en un mismo dominio público, de modo que todo el tráfico dirigido al almacenamiento —ya se originara externamente en el navegador o fuera reinyectado internamente por el servidor de la aplicación— atravesara el *reverse proxy* perimetral de la facultad.

La Figura 3.8 ilustra esta convergencia de los dos flujos hacia el almacenamiento físico. La línea discontinua verde representa el flujo directo originado en el navegador del usuario para las cargas de branding, mientras que la línea discontinua naranja representa el flujo indirecto originado en el *backend* de Documenso para la gestión de documentos PDF. Ambas líneas discontinuas convergen en el *reverse proxy* perimetral Nginx. A partir de ese punto, el tráfico unificado se representa mediante una línea continua rosa que atraviesa Traefik hacia el contenedor socat, y finalmente se transforma en una línea continua azul que enlaza el túnel TCP con el servidor de almacenamiento TrueNAS/MinIO.

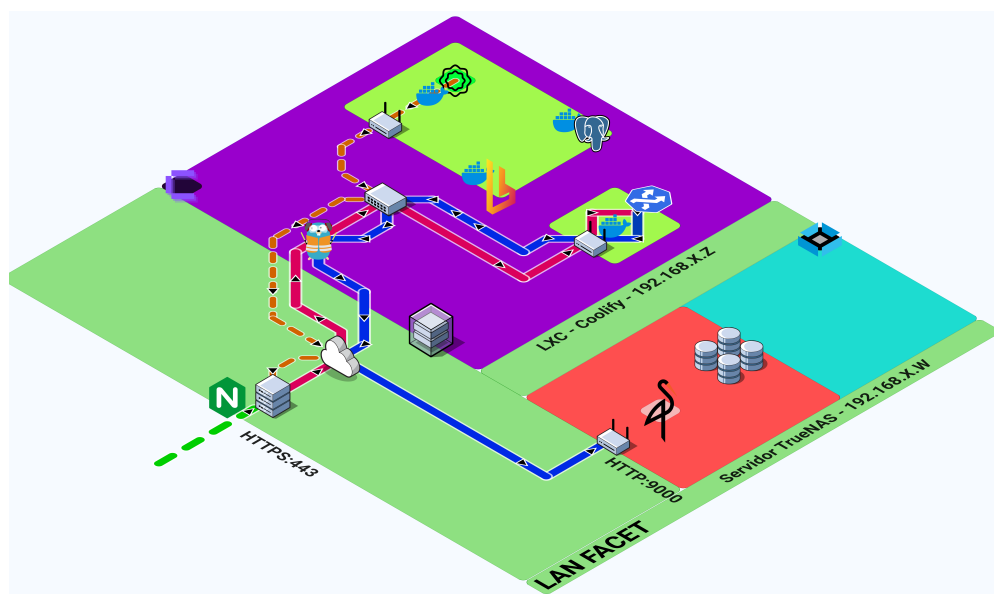


FIGURA 3.8: Convergencia del flujo directo del navegador y del flujo indirecto del *backend* hacia el servidor de almacenamiento TrueNAS/MinIO a través del *reverse proxy* perimetral, Traefik y el contenedor Socat.

Una versión ampliada de este diagrama se encuentra disponible en la Sección A.4 del Apéndice A.

3.5. Estrategia de Respaldo y Retención de Datos

La estrategia de preservación de datos se organizó en tres dominios con necesidades de protección diferentes: la base transaccional de PostgreSQL, el repositorio documental almacenado en MinIO y la configuración exportable de la plataforma. Esta separación respondió a un criterio de criticidad operativa: no todos los componentes del sistema se recuperan del mismo modo ni tienen el mismo impacto ante una falla. La base de datos admite respaldos lógicos periódicos, los documentos PDF requieren políticas de retención e inmutabilidad sobre el almacenamiento de objetos, y la reconstrucción del entorno exige conservar las variables de entorno junto con los archivos `docker-compose.yml` utilizados en producción.

En los tres dominios se aplicó la misma base técnica de almacenamiento S3. La política ILM (Information Lifecycle Management) define el plazo de retención de cada *bucket*, y *Object Lock* en modo *governance* impide modificar o eliminar objetos antes de que venza ese plazo. Para habilitar *Object Lock*, MinIO exige versionado de *buckets*; la operación diaria de la plataforma no depende de ese versionado.

La documentación de Documenso distingue dos alternativas de almacenamiento documental: el almacenamiento en base de datos y el almacenamiento S3 compatible. La Tabla 3 resume las diferencias operativas más relevantes entre ambas, incluida la recomendación explícita de utilizar almacenamiento S3 en despliegues de producción [30, 29].

TABLA 3: Comparación entre las alternativas almacenamiento en base de datos y almacenamiento compatible con S3 en Documenso, a partir de la documentación oficial.

Aspecto	almacenamiento en base de datos	almacenamiento compatible con S3
Costo de respaldo	Los respaldos de la base crecen junto con el volumen documental.	Requiere respaldar por separado la base y el <i>bucket</i> S3.
Rendimiento con archivos grandes	El rendimiento se degrada a medida que aumenta el tamaño del repositorio.	Mantiene un comportamiento más consistente para cargas documentales mayores.
Recomendación de uso	Adecuado para despliegues pequeños o simples.	La documentación lo recomienda para despliegues de producción.

3.5.1. Respaldo lógico de la base de datos PostgreSQL

En este despliegue, el respaldo lógico de PostgreSQL cubre exclusivamente los datos transaccionales descritos en la Subsección 3.3.1. Bajo este criterio, la continuidad operativa no se apoyó exclusivamente en el volumen persistente local del servicio, sino en una política de exportación periódica del contenido de la base hacia un destino independiente, alineada con las estrategias diarias de respaldo automatizado que la documentación de Documenso propone para recuperación a un punto reciente [29]. En Coolify se deshabilitó el almacenamiento local de respaldos, los volcados se enviaron al *bucket* `documenso-backup-databases` en MinIO y se fijó una cuota de 5 GB para ese destino; además, se utilizan marcas de tiempo para garantizar la unicidad de cada respaldo.

El job se programó con la expresión `cron 0 3 * * *`, de modo que la exportación se ejecuta diariamente a las 03:00 hacia el destino S3 configurado.

Sobre este *bucket* se aplicó ILM con una retención de 30 días para la versión vigente del respaldo. Esta ventana, junto con la cuota de 5 GB, ofrece un margen operativo prudente para revertir corrupciones o eliminaciones accidentales de la base. Además, se configuró *Object Lock* en modo *governance* por 30 días [22]. Esta frecuencia diaria fija un objetivo práctico de punto de recuperación (*RPO*) de 24 horas para el dominio transaccional. El objetivo de tiempo de recuperación (*RTO*) se mantiene acotado gracias a que Coolify integra importación y restauración directa de volcados desde el almacenamiento S3, lo que reduce la contingencia a una operación asistida desde la interfaz gráfica. El detalle operativo de este procedimiento se documenta en el Apéndice C.1.

3.5.2. Retención e inmutabilidad del repositorio documental

Los documentos originales cargados por los usuarios y las versiones firmadas se almacenan en el *bucket* `documenso-prod` del servidor MinIO. Los documentos constituyen el activo *core* del sistema. Como el almacenamiento físico es finito, no pueden conservarse de forma indefinida.

Por ello se definió una política ILM de 5 años y, para sostener esa retención, se proyectó y asignó una cuota de 2 TB para el repositorio. El cálculo se apoya en las siguientes restricciones:

- Bajo un límite de 300 correos diarios (proveedor SMTP externo) y estimando un promedio de 5 firmantes por trámite —lo que consume 6 notificaciones por documento (invitaciones y aviso de finalización)—, la plataforma alcanza un techo operativo de 50 trámites por día.
- Con un peso promedio estimado de 10 MB por PDF, y considerando que MinIO almacena tanto el archivo original como la versión final firmada (20 MB totales por trámite), el incremento máximo de datos es de 1 GB diario.
- Al aplicar la política de retención ILM de 5 años (1825 días), el volumen de datos proyectado es de 1825 GB.

En consecuencia, la cuota de 2 TB cubre la demanda máxima teórica proyectada a cinco años, con un margen aproximado del 10 %.

Este límite de 2 TB se estableció considerando la capacidad física del servidor. El volumen de almacenamiento virtual de TrueNAS expone 18,19 TiB al sistema operativo, por lo que la cuota asignada representa aproximadamente el 11 % del espacio total.

Sobre esa infraestructura se configuraron en MinIO las políticas de retención del *bucket*. Con el versionado ya habilitado a nivel de plataforma, se aplicó una política ILM de cinco años [37] y *Object Lock* en modo *governance* por el mismo plazo [22].

La limitación principal de este dominio de datos aparece en la transición entre ambas capas. La combinación de controles de retención e inmutabilidad protege frente a eliminaciones accidentales, sobrescrituras indebidas y parte de las alteraciones lógicas dentro de MinIO, pero no elimina el punto único de falla representado por el servidor TrueNAS que hospeda el almacenamiento físico. Una falla catastrófica del NAS, una corrupción severa del *pool* o la pérdida total del servidor implicarían la indisponibilidad o pérdida definitiva de los PDF almacenados. Por ello, la replicación o el respaldo *off-site* del repositorio documental hacia un segundo destino S3 compatible se identifica como la mejora de mayor prioridad para una etapa posterior.

3.5.3. Resguardo de la configuración del despliegue

El tercer dominio de protección se orientó a preservar la reconstruibilidad de la plataforma a partir de su configuración exportable. Para ello, se resguardaron las variables de entorno y los archivos `docker-compose.yml` de cada servicio desplegado, incluida la instancia de Documenso, en el *bucket* `backups-coolify`. Este resguardo se automatizó mediante un *script* en bash, documentado en el Apéndice B.3, y ejecutado diariamente con *cron* (0 4 * * *). En lugar de copiar toda la estructura de Coolify, el procedimiento conserva solo los archivos necesarios para reconstruir el entorno. La periodicidad diaria no respondió a cambios frecuentes en la configuración de Documenso, sino a que ese proceso resguarda, de forma unificada, la configuración exportable del conjunto de servicios administrados por Coolify. Sobre dicho *bucket* se definió una cuota de 100 MB y una política ILM con una retención de 180 días para la versión vigente. Al tratarse exclusivamente de archivos de texto plano comprimidos, esta ventana extendida consume un espacio de almacenamiento marginal, pero aporta un valor operativo crítico: garantiza la capacidad de auditar el historial de infraestructura y recuperar configuraciones antiguas

(.env) de cualquier proyecto de la facultad ante modificaciones accidentales, eliminaciones o migraciones fallidas.

Este resguardo no sustituye los respaldos de la base de datos ni del repositorio documental, puesto que no conserva ni los datos transaccionales de la aplicación ni los archivos PDF. Su aporte es otro: reduce el tiempo y el esfuerzo requeridos para reconstruir el entorno ante una contingencia severa sobre el servidor. Al preservar la definición de los servicios, las referencias de red y los parámetros de configuración, se evita depender de la reconstrucción manual de secretos, variables y descriptores de despliegue dispersos entre la interfaz de la PaaS y la memoria operativa del administrador.

La guía de mantenimiento de Documenso señala además que, junto con la base de datos y el almacenamiento documental, el certificado de firma y su frase de paso forman parte de los elementos críticos que deben resguardarse [29]. En este despliegue, el certificado no se administra como un archivo montado en un volumen, sino como una variable de entorno que contiene el archivo .p12 codificado en Base64, junto con la frase de paso inyectada por Coolify. Por ello, el respaldo diario de la configuración también preserva los valores necesarios para restablecer la capacidad de emitir nuevas firmas con la identidad institucional ya configurada.

La estrategia de preservación ofrece puntos de recuperación independientes para el dominio transaccional y para la configuración declarativa del entorno. El riesgo residual ya señalado — la ausencia de un respaldo externo del repositorio documental— permanece como la limitación de mayor prioridad para una etapa posterior, dado que una falla física del servidor de almacenamiento dejaría sin recuperación los PDF alojados en MinIO.

3.6. Integración del Servicio de Correo Transaccional

3.6.1. Configuración del Servicio de Correo Transaccional (Brevo SMTP)

La operación de Documenso requiere un servicio SMTP para emitir invitaciones de firma, recordatorios y notificaciones transaccionales asociadas al avance de cada flujo de firma. Aunque desde el punto de vista técnico hubiera sido posible desplegar un servidor de correo local dentro de la infraestructura institucional, esa alternativa quedó fuera del alcance por su carga operativa. El problema no radica en instalar el servicio, sino en sostener su entregabilidad en producción: reputación de la IP emisora, prevención de bloqueos en listas negras y cumplimiento de políticas de filtrado cada vez más estrictas por parte de proveedores como Gmail u Outlook.

Por ese motivo, el correo saliente se resolvió con Brevo y no con un servidor SMTP propio. La plataforma continuó alojada en la infraestructura de la facultad, pero el envío de notificaciones se delegó a un proveedor externo ya preparado para operar con tasas de entrega adecuadas.

La configuración de Brevo se realizó siguiendo el procedimiento habitual de alta del servicio: creación de la cuenta, verificación del remitente o dominio y obtención de las credenciales SMTP que luego fueron cargadas en Documenso. Durante este trabajo se utilizó el plan gratuito de Brevo, que permite hasta 300 correos diarios. Ese valor se tomó como restricción de dimensionamiento en la estimación presentada en la Sección 3.5. Bajo esa hipótesis, se obtiene un promedio estimado de 50 documentos por día, pero no un límite fijo del sistema, ya que

la cantidad real depende del número de firmantes, recordatorios y notificaciones efectivamente enviadas en cada flujo de firma.

Una vez obtenidos en Brevo los registros DNS requeridos para verificar el dominio, se solicitó al área de redes de la facultad su publicación. Se publicaron los registros DNS (TXT) requeridos para la verificación y autorización del dominio.

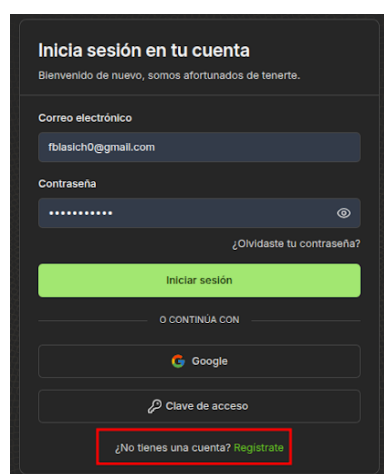
3.6.2. Autenticación Institucional con Google OAuth 2.0

Para el acceso a Documenso se descartó el registro local de usuarios y contraseñas. El motivo principal fue restringir el uso del sistema al personal de la facultad, de modo que ninguna persona ajena pudiera crear una cuenta ni operar la plataforma de firmas. Bajo este criterio, solo los usuarios con cuenta institucional pueden iniciar sesión, cargar documentos y definir los firmantes de cada trámite. Esta restricción alcanza a quienes crean y administran procesos dentro de la plataforma, pero no a los destinatarios de firma: los firmantes pueden pertenecer a cualquier dominio de correo electrónico, ya que el sistema los incorpora como participantes del flujo documental a través de notificaciones transaccionales enviadas por correo.

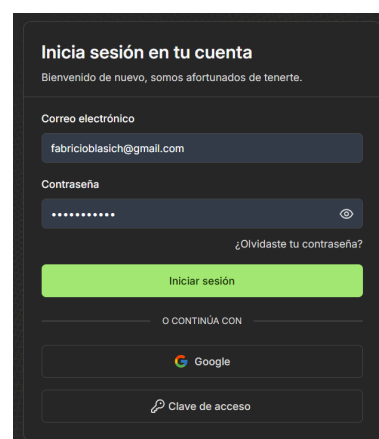
Se implementó un flujo OAuth 2.0 restringido al ámbito *Interno* de Google Workspace, lo cual garantiza que solo los usuarios del dominio institucional puedan autenticarse.

Para completar la restricción de acceso, se deshabilitó el registro local de nuevas cuentas mediante la configuración correspondiente en las variables de entorno de la aplicación. Con esta medida, el único mecanismo de ingreso quedó limitado a la autenticación mediante Google. El detalle de las variables utilizadas se documenta en el Apéndice B.8.

La Figura 3.9 muestra el cambio visible en la interfaz de acceso. Con el registro local habilitado aparece el enlace “¿No tienes una cuenta? Regístrate”; al deshabilitarlo, ese enlace desaparece y el ingreso queda limitado a cuentas institucionales.



(A) Pantalla de acceso con registro local habilitado.



(B) Pantalla de acceso con registro local deshabilitado.

FIGURA 3.9: Comparación de la interfaz de acceso de Documenso con el registro local habilitado y deshabilitado. La eliminación del enlace de registro evidencia que el ingreso al sistema queda limitado a cuentas institucionales.

Capítulo 4

Ensayos y Resultados

El capítulo reúne las pruebas que validan la infraestructura desplegada en el Capítulo 3 frente a los objetivos funcionales, de seguridad, de rendimiento y de continuidad operativa definidos en la Sección 1.3. Las verificaciones avanzan por capas: despliegue de cada servicio, integración entre componentes, control de acceso, flujo funcional completo, métricas de consumo de recursos y restauración de respaldos. Cada ensayo contrasta el comportamiento observado en producción contra el diseño previsto, y el capítulo cierra con una matriz de trazabilidad que vincula los requisitos con la evidencia obtenida.

4.1. Metodología de Validación

La validación abarca el comportamiento de los componentes desplegados, sus integraciones y el sistema en su conjunto bajo condiciones controladas de producción. Las verificaciones se organizan en cinco capas progresivas:

- Capa de servicio: verificación de que cada contenedor inicia, supera su *healthcheck* y mantiene disponibilidad en el tiempo.
- Capa de integración: verificación de que los canales de comunicación entre servicios retornan respuestas exitosas y no registran errores en producción (Documenso con PostgreSQL, Browserless, MinIO y el *relay* SMTP de Brevo).
- Capa de sistema: ejecución del flujo de firma completo de extremo a extremo en producción, desde la carga del documento hasta la descarga del PDF sellado.
- Capa de rendimiento: observación del consumo de CPU y RAM durante operación normal y bajo carga controlada.
- Capa de continuidad: verificación de que los respaldos automáticos se almacenan correctamente y que un volcado puede restaurarse sin pérdida de datos.

Las herramientas empleadas para la recolección de evidencia fueron: la interfaz de Coolify (logs de contenedores y estado de servicios), las herramientas de desarrollo del navegador (trazas HTTP), Adobe Reader (validación de firma en PDF), Gmail (headers técnicos de autenticación SMTP) y Netdata (métricas de CPU y RAM por contenedor).

4.2. Validación de Despliegue de Servicios

Esta sección verifica que la capa de servicio descrita en la Sección 4.1 es operativa: cada contenedor del *stack* alcanzó el estado *Healthy* y superó las pruebas de disponibilidad configuradas en el Capítulo 3. La validación se presenta en dos niveles complementarios: la verificación de los *healthchecks* a nivel de infraestructura desde la interfaz de Coolify y la consulta del *endpoint* de diagnóstico interno de Documenso.

4.2.1. Estado de los contenedores en Coolify

El archivo de composición del despliegue, presentado en la Sección 3.3 del Capítulo 3, define una directiva *healthcheck* para cada servicio del *stack*. Las Figuras 4.1 y 4.2 reflejan visualmente la resolución exitosa del árbol de dependencias del orquestador (*depends_on*) y la confirmación de los *healthchecks* internos: los servicios *documenso*, *database*, *browserless* y *socat* se observan en estado *Healthy*, en coherencia con el diseño de despliegue.

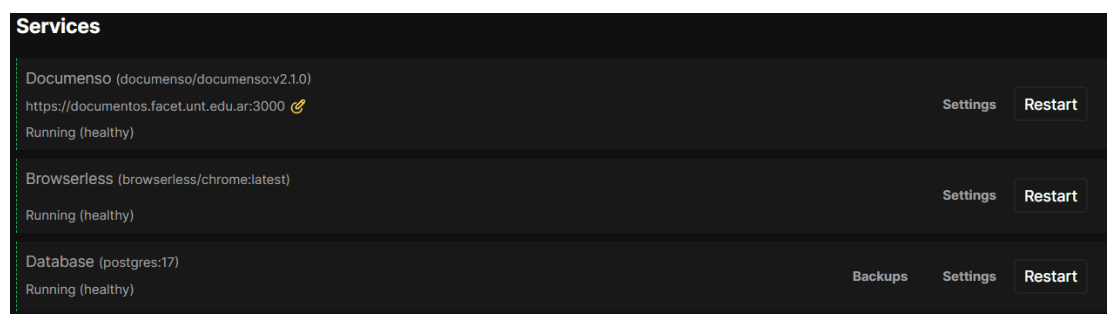


FIGURA 4.1: Evidencia del estado *Healthy* de los servicios *documenso*, *database* y *browserless*, consistente con la resolución de dependencias y la ejecución satisfactoria de *healthchecks* internos.



FIGURA 4.2: Evidencia del estado *Healthy* del servicio *socat*, consistente con la validación operativa del *proxy* TCP hacia MinIO en el puerto 9000.

El estado *Healthy* de los cuatro servicios confirma que el *stack* completo superó sus pruebas de disponibilidad. Dado que *documenso* declara *depends_on* sobre *database* con la condición *service_healthy*, su propio estado *Healthy* implica además que PostgreSQL completó sus verificaciones antes de que la aplicación iniciara.

4.2.2. Verificación del estado interno de la aplicación

Documenso expone además el *endpoint* `/api/health`, que extiende la verificación de infraestructura hacia los subsistemas internos de la aplicación. A diferencia del *healthcheck* de Docker

—que únicamente comprueba accesibilidad de red—, este *endpoint* reporta el estado de la conectividad con la base de datos y la disponibilidad del certificado de firma.

El Código 4.1 muestra la respuesta obtenida al consultar <https://documentos.facet.unt.edu.ar/api/health>.

CÓDIGO 4.1: Respuesta del endpoint /api/health de Documenso

```
{
  "status": "ok",
  "timestamp": "2026-04-12T15:01:14.367Z",
  "checks": {
    "database": { "status": "ok" },
    "certificate": { "status": "ok" }
  }
}
```

Los tres campos con valor ok confirman que la aplicación está operativa, que Documenso estableció conectividad con PostgreSQL y que el certificado de firma institucional está cargado y disponible.

4.3. Verificación de la Comunicación entre Servicios

Esta sección valida que los canales de comunicación entre los componentes del despliegue operan según el diseño descrito en el Capítulo 3, mediante tres verificaciones: la conectividad con el almacenamiento de objetos MinIO a través del *proxy* Socat, la comunicación con Browserless para el renderizado del certificado de firma y la entregabilidad del correo transaccional a través del *relay* de Brevo.

4.3.1. Conectividad con el almacenamiento de objetos (MinIO)

Los logs HTTP del servidor MinIO registran los dos flujos de acceso al *bucket* documenso-prod: el directo, originado desde el navegador del usuario, y el indirecto, originado desde el *backend* de la aplicación.

La Figura 4.3 muestra una operación PUT 200 OK cuyo campo User-Agent identifica al cliente como un navegador Microsoft Edge con IP de origen externa a la red institucional, lo que confirma que la solicitud atravesó el recorrido completo: *reverse proxy* perimetral, Traefik, contenedor Socat y servidor físico TrueNAS.

La Figura 4.4 muestra el registro de una operación GET con código 200 OK sobre ese bucket. En este caso, el campo User-Agent indica node, lo que identifica al cliente como el proceso Node.js del *backend* de Documenso. Este registro corresponde al flujo indirecto: la aplicación recupera el documento PDF desde MinIO a través del *proxy* Socat para procesarlo internamente.

```

minio.facet.unt.edu.ar [REQUEST s3.PutObject] [2026-04-24T01:37:33.059] [Client IP: 186.12.188.90]
minio.facet.unt.edu.ar PUT /documenso-prod/oz3y072e34n1/facet-branding.jpg?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Content-Sha256=UNSIGNED-PAYLOAD&X-Amz-Credential=DhzphIaLgVpctk8P36Sp%2F20260424%2Fus-east-1%2Fs3%2Faws4_request&X-Amz-Date=20260424T013732Z&X-Amz-Expires=3600&X-Amz-Signature=f620c26d0c3400dd4fa187133a30e66fa0a57a3b6ed4b3b60e2ba02599f12133&X-Amz-SignedHeaders=host&x-amz-checksum-crc32=AAAAA%3D%3D&x-amz-sdk-checksum-algorithm=CRC32&x-id=PutObject
minio.facet.unt.edu.ar Proto: HTTP/1.1
minio.facet.unt.edu.ar Host: minio.facet.unt.edu.ar
minio.facet.unt.edu.ar X-Amz-Signature-Age: 1060
minio.facet.unt.edu.ar X-Forwarded-Port: 443
minio.facet.unt.edu.ar X-Forwarded-Proto: https
minio.facet.unt.edu.ar X-Forwarded-Server: 832a2aeed10b
minio.facet.unt.edu.ar Accept: */*
minio.facet.unt.edu.ar Accept-Language: en-US,en;q=0.9
minio.facet.unt.edu.ar Origin: https://documentos.facet.unt.edu.ar
minio.facet.unt.edu.ar Sec-Ch-Ua: "Microsoft Edge";v="147", "Not.A/Brand";v="8", "Chromium";v="147"
minio.facet.unt.edu.ar Sec-Fetch-Mode: cors
minio.facet.unt.edu.ar User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36 Edg/147.0.0.0

```

FIGURA 4.3: Registro de acceso HTTP del servidor MinIO mostrando una operación PUT 200 OK originada desde un navegador Microsoft Edge. El campo User-Agent identifica el flujo directo documentado en la Sección 3.4.

```

minio.facet.unt.edu.ar [REQUEST s3.GetObject] [2026-04-24T01:25:22.802] [Client IP: 200.45.169.126]
minio.facet.unt.edu.ar GET /documenso-prod/1qqnmzpuj7dc/hoja-blanco-prueba-signed.pdf?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Content-Sha256=UNSIGNED-PAYLOAD&X-Amz-Credential=DhzphIaLgVpctk8P36Sp%2F20260424%2Fus-east-1%2Fs3%2Faws4_request&X-Amz-Date=20260424T012522Z&X-Amz-Expires=3600&X-Amz-Signature=c6cac3790bae3070378aeb04e2d8c23cbfe4bef72797a491f45afd&X-Amz-SignedHeaders=host&x-amz-checksum-mode=ENABLED&x-id=GetObject
minio.facet.unt.edu.ar Proto: HTTP/1.1
minio.facet.unt.edu.ar Host: minio.facet.unt.edu.ar
minio.facet.unt.edu.ar X-Amz-Signature-Age: 804
minio.facet.unt.edu.ar X-Forwarded-For: 200.45.169.126, 10.10.2.100
minio.facet.unt.edu.ar X-Forwarded-Proto: https
minio.facet.unt.edu.ar Accept-Encoding: br, gzip, deflate
minio.facet.unt.edu.ar Content-Length: 0
minio.facet.unt.edu.ar Sec-Fetch-Mode: cors
minio.facet.unt.edu.ar User-Agent: node
minio.facet.unt.edu.ar X-Forwarded-Host: minio.facet.unt.edu.ar
minio.facet.unt.edu.ar X-Forwarded-Port: 443
minio.facet.unt.edu.ar X-Forwarded-Server: 832a2aeed10b
minio.facet.unt.edu.ar X-Real-IP: 200.45.169.126

```

FIGURA 4.4: Registro de acceso HTTP del servidor MinIO mostrando una operación GET 200 OK con User-Agent: node. El identificador confirma que el cliente es el proceso *backend* de Documenso, correspondiente al flujo indirecto.

Ambos registros, observados desde el servidor de almacenamiento, confirman que los dos flujos definidos en el diseño operan correctamente en producción: el canal directo navegador-MinIO y el canal indirecto backend-MinIO reciben respuestas 200 OK del servidor, lo que valida la cadena completa de conectividad a través del proxy Socat.

4.3.2. Comunicación con Browserless (renderizado del certificado de firma)

Los registros de los contenedores documenso y browserless se analizaron para verificar que el renderizado del certificado de firma opera sin errores en producción.

El Código 4.2 contiene el fragmento de los registros de Documenso al momento en que se disparó el job de sellado para el documento 12.

CÓDIGO 4.2: Registros del contenedor Documenso: disparo del job
internal.seal-document

```
2026-Apr-11 17:48:46
[JOBS]: Triggering job internal.seal-document with payload
{ documentId: 12, requestMetadata: { ipAddress:
  '148.222.222.66',
  userAgent: 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) ...' }
}
2026-Apr-11 17:48:46
Submitting job to endpoint:
http://documenso:3000/api/jobs/internal.seal-document/[JOB_ID]
```

El Código 4.3 registra la respuesta de Browserless, comprendida entre los instantes 17:48:52 y 17:49:11 UTC.

CÓDIGO 4.3: Registros del contenedor Browserless: recepción y completado del job de renderizado

```

17:48:52 browserless:job      [...]: Inbound WebSocket request.
17:48:52 browserless:job      [...]: Adding new job to queue.
17:48:52 browserless:system Generating fresh chrome browser
17:48:52 browserless:chrome-helper Launching Chrome with args:
      { "executablePath": "/usr/bin/google-chrome" }
17:49:06 browserless:system Got chrome instance
17:49:11 browserless:server  [...]: Recording successful stat
      and cleaning up.
17:49:11 browserless:chrome-helper Browser process [...] has
      closed, cleaning up.

```

Los registros evidencian que Browserless recibió la solicitud de renderizado por WebSocket, lanzó una instancia de Chrome desde `/usr/bin/google-chrome` y completó el procesamiento en aproximadamente 18 segundos. El mensaje `Recording successful stat and cleaning up` confirma que el job concluyó sin errores. Este resultado confirma que la limitación documentada en la Subsección 3.3.2 del Capítulo 3 quedó resuelta en producción y que el sellado criptográfico del documento puede completarse.

4.3.3. Entregabilidad del correo transaccional (Brevo SMTP)

La Figura 4.5 presenta la evidencia recolectada en Gmail mediante la inspección de encabezados SMTP de un correo transaccional emitido por Documenso, donde se observan los mecanismos SPF, DKIM y DMARC. Estos resultados (PASS) garantizan que los proveedores de correo reconozcan la legitimidad de las notificaciones emitidas por la plataforma. En la práctica, esto evita que los avisos de firma sean bloqueados o marcados como *spam*,

Mensaje original	
ID del mensaje	<9dc25ead-cae5-43bc-91a6-e16fb8d0bd09@smtp-relay.sendinblue.com>
Creado el:	11 de abril de 2026 a las 2:45 p.m. (Entregado tras 4 segundos)
De:	Firmas FACET <notificaciones@documentos.facet.unt.edu.ar>
Para:	Fabricio - Admin <fblasich0@gmail.com>
Asunto:	Please sign this document
SPF:	PASS con el IP 77.32.148.23 Más información
DKIM:	'PASS' con el dominio documentos.facet.unt.edu.ar Más información
DMARC:	'PASS' Más información

FIGURA 4.5: Captura de encabezados SMTP obtenida en Gmail para un correo transaccional de Documenso. Se visualizan los resultados de validación de SPF, DKIM y DMARC sobre el mensaje emitido a través del *relay* de Brevo.

4.4. Validación de Acceso y Control de Seguridad

Esta sección verifica que el mecanismo de acceso a la plataforma opera según lo configurado en la Subsección 3.6.2: solo cuentas pertenecientes al dominio institucional pueden autenticarse, y el registro libre de nuevos usuarios está deshabilitado.

4.4.1. Restricción del inicio de sesión con Google OAuth 2.0

Al intentar iniciar sesión con una cuenta externa al dominio institucional, Google interrumpió el flujo de OAuth antes de emitir el código de autorización: el servidor de autorización devolvió el código Error 403: `org_internal` con el mensaje Acceso bloqueado: Sistema FirmasFACET solo se puede usar dentro de su organización (Figura 4.6). Este comportamiento es consecuencia directa de haber configurado el tipo de aplicación como *Interna* en Google Cloud Console, tal como se describió en la Subsección 3.6.2: la restricción opera en la capa de autorización de Google y no depende de ninguna validación adicional por parte de la aplicación.

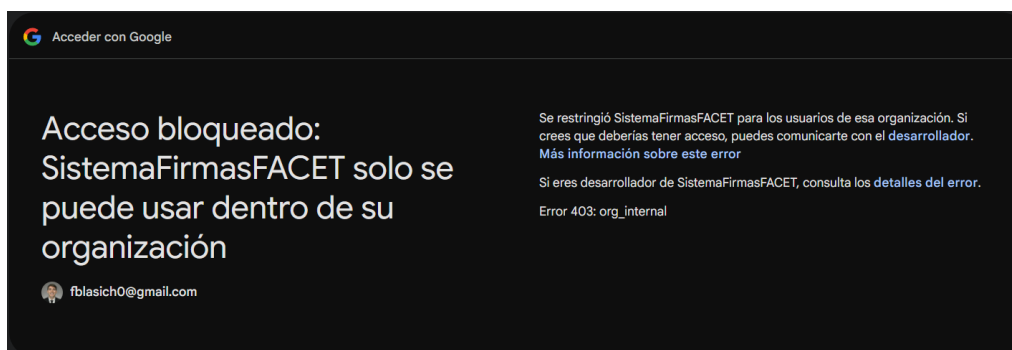


FIGURA 4.6: Pantalla de error devuelta por Google al intentar autenticarse con una cuenta @gmail.com. El código `org_internal` confirma que la restricción opera en la capa de autorización de Google.

Con una cuenta @herrera.unt.edu.ar, Google presentó el modal de consentimiento y, una vez aceptado, el servidor emitió el código de autorización; el backend de Documenso completó el intercambio y el usuario accedió al panel de trabajo.

4.4.2. Deshabilitación del registro local

El registro de usuarios mediante credenciales locales fue deshabilitado a través de la variable de entorno `NEXT_PUBLIC_DISABLE_SIGNUP=true`, cuyo efecto en la interfaz se documentó en la Subsección 3.6.2 (Figura 3.9).

Se realizó una solicitud GET directa a `https://documentos.facet.unt.edu.ar/signup` para verificar que la restricción opera a nivel de servidor. La solicitud retornó 302 Found con Location: `/signin` (Figura 4.7): la redirección es forzada por el backend antes de renderizar cualquier contenido de registro, lo que confirma que la restricción opera en la lógica de enrutamiento del servidor y no se limita a ocultar el formulario en el frontend.

▼ General	
Request URL	https://documentos.facet.unt.edu.ar/signup
Request Method	GET
Status Code	● 302 Found
Remote Address	200.45.169.100:443
Referrer Policy	strict-origin-when-cross-origin
▼ Response headers	
Alt-Svc	h3=":443"; ma=2592000
Content-Length	0
Content-Type	text/plain; charset=UTF-8
Date	Sat, 11 Apr 2026 23:02:21 GMT
Location	/signin
Server	nginx
Vary	Accept-Encoding

FIGURA 4.7: Captura de las herramientas de desarrollo del navegador mostrando la respuesta 302 Found con redirección a /signin al intentar acceder directamente a la ruta de registro.

4.5. Validación del Flujo Funcional de Extremo a Extremo

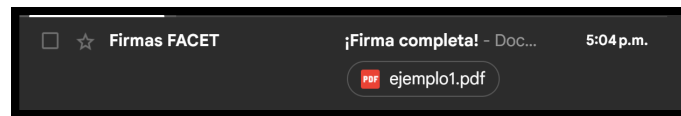
Esta sección valida la capa de sistema definida en la Sección 4.1: el flujo completo de firma documental se ejecutó en producción con dos firmantes. La validación se presenta en dos partes: el ciclo operativo de extremo a extremo y la verificación de la firma electrónica incrustada en el documento resultante.

4.5.1. Ciclo completo de firma documental

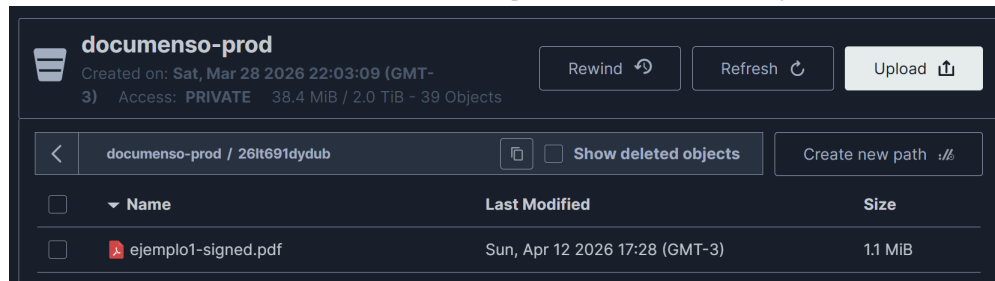
Se ejecutó un ciclo completo de firma sobre el documento `ejemplo1.pdf` con dos destinatarios: una cuenta institucional (@herrera.unt.edu.ar) y una cuenta externa al dominio (@gmail.com). El flujo recorrió las etapas de carga del documento, asignación de firmantes, posicionamiento de campos, notificación por correo electrónico y aplicación de la firma. El destinatario externo accedió mediante el enlace recibido por correo y completó la acción sin requerir cuenta local en la plataforma, lo que confirmó la operación del acceso por enlace documentado en la Subsección 3.6.2. Al completarse ambas firmas, la plataforma disparó el job de sellado criptográfico descrito en la Subsección 4.3.2 y el ciclo concluyó sin errores.

El procedimiento paso a paso de este ciclo completo se documenta en el Apéndice D.

La Figura 4.8 registra el cierre del ciclo. Al completarse ambas firmas, la plataforma emitió el correo de notificación con el PDF firmado adjunto y persistió el artefacto `_signed.pdf` en el bucket `documenso-prod` de MinIO. Esta persistencia cerró el ciclo de integración validado en la Sección 4.3: el documento sellado criptográficamente quedó almacenado en el bucket verificado en la Subsección 4.3.1.



(A) Correo de notificación de completado con PDF firmado adjunto.



(B) Bucket documento-prod: artefacto _signed.pdf persistido en MinIO.

FIGURA 4.8: Correo de notificación emitido al concluir el ciclo con el PDF firmado adjunto (A); registro del artefacto _signed.pdf en el bucket documento-prod de MinIO (B).

4.5.2. Verificación de la firma electrónica en el documento

El PDF descargado al concluir el ciclo documentado en la Subsección 4.5.1 se analizó en Adobe Acrobat para confirmar que el sellado criptográfico realizado por Documenso es verificable desde un visor estándar.

El panel de firmas (Figura 4.9) indica que no hubo modificaciones en el documento desde que se firmó; la advertencia de validez desconocida responde a que el certificado es autofirmado y no pertenece a una cadena de confianza comercial, comportamiento previsto en el diseño y documentado en la Subsección 2.3.

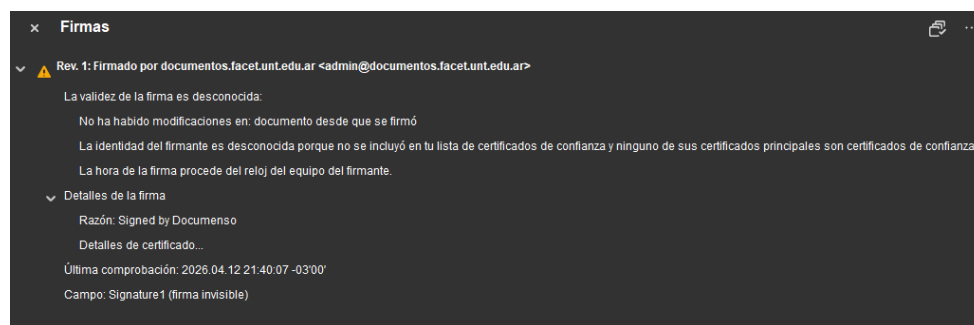


FIGURA 4.9: Panel de firmas de Adobe Acrobat. Se indica que no hubo modificaciones en el documento desde que se firmó y se identifican los detalles del firmante institucional.

La Figura 4.10 detalla el certificado utilizado: emisor y sujeto son documentos.facet.unt.edu.ar (FACET), con vigencia del 28/03/2026 al 25/03/2036. Este intervalo corresponde al parámetro -days 3650 utilizado en la generación del certificado. La concordancia entre la vigencia exhibida por Microsoft Edge y el parámetro de generación confirma que el visor reconoce el material

criptográfico institucional producido durante el despliegue (Apéndice B.5).

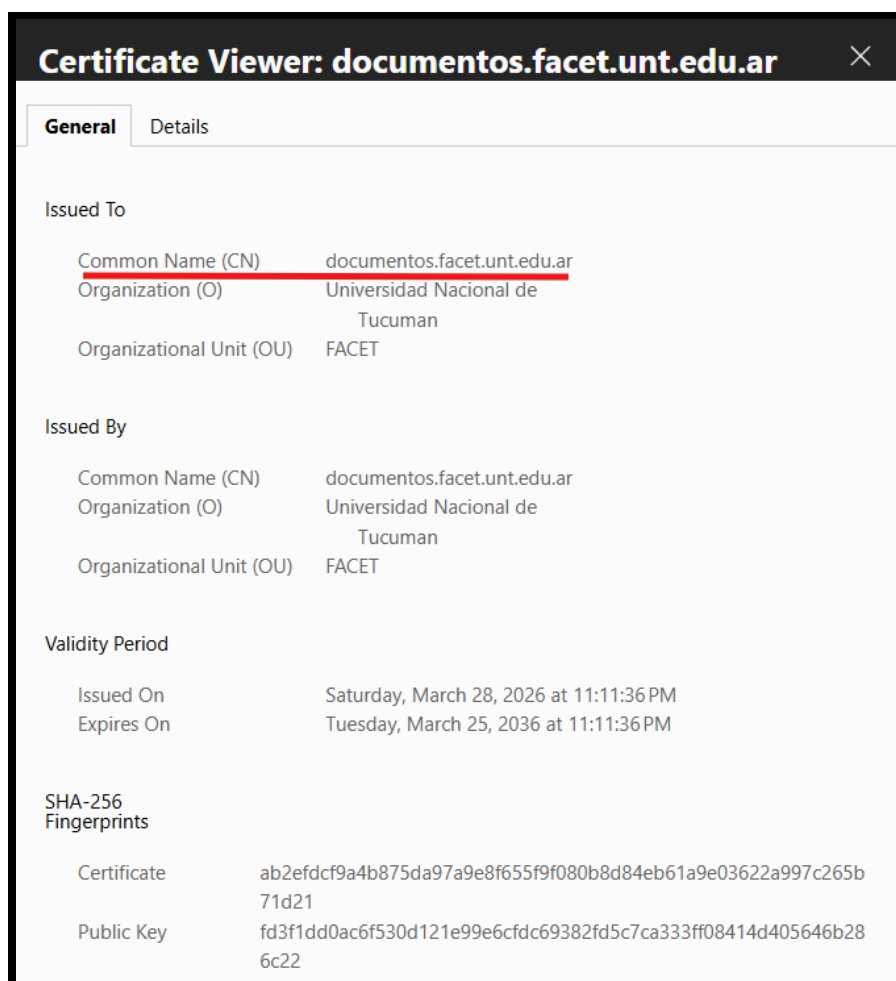


FIGURA 4.10: Visor de certificados de Microsoft Edge. Emisor y sujeto son documentos.facet.unt.edu.ar (FACET); vigencia del 28/03/2026 al 25/03/2036.

4.6. Métricas de Rendimiento del Sistema

Las métricas de consumo de CPU y RAM se registraron por contenedor mediante Netdata. Los valores se contrastan a continuación con la capacidad auditada del LXC: 8 vCPUs y 8 GiB de RAM (Subsección 3.2). En todas las gráficas de esta sección, la curva amarilla corresponde a Documenso, la naranja a Browserless, la azul a PostgreSQL y la rosa al proxy Socat.

4.6.1. Consumo de recursos en operación normal (baseline)

El baseline se registró el 24 de abril de 2026 entre las 16:17 y las 16:20 UTC bajo condiciones de navegación estándar y descarga de archivos, sin ejecución de ciclos de firma concurrentes.

Memoria RAM

La Figura 4.11 registra el consumo de RAM de los cuatro contenedores del *stack* durante el período de observación. Las curvas se mantuvieron planas a lo largo del período, lo que descartó fugas de memoria en operación normal.

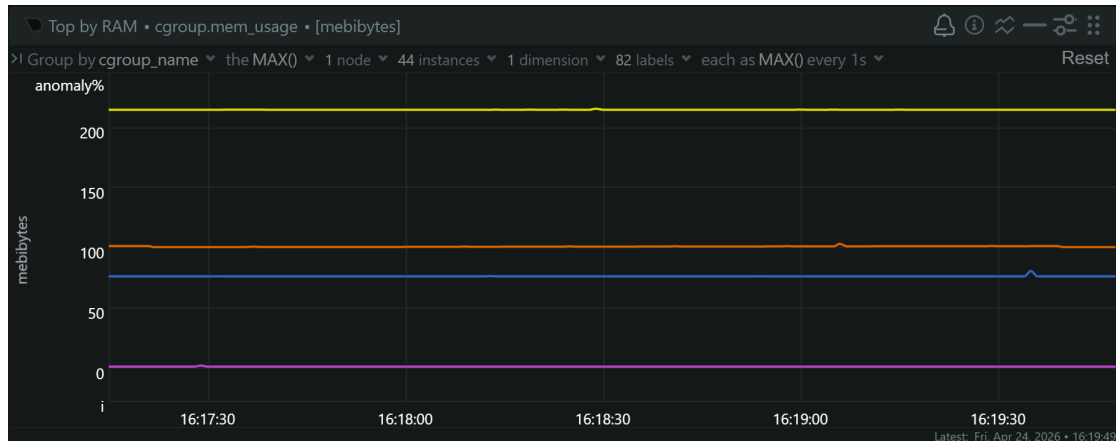


FIGURA 4.11: Consumo de RAM (en MiB) de los contenedores del *stack* durante parte del período de observación en baseline. Documentso: $\approx 215,5$ MiB; Browserless: $\approx 102,0$ MiB; PostgreSQL: $\approx 77,0$ MiB; *proxy Socat*: $\approx 2,2$ MiB. Total del *stack*: $\approx 396,7$ MiB sobre 8 GiB asignados al LXC.

El consumo total del *stack* en reposo fue de aproximadamente 396,7 MiB, equivalente al 5 % de la capacidad del LXC. Documentso concentró la mayor parte de ese consumo (215,5 MiB), seguido de Browserless (102,0 MiB). PostgreSQL y Socat aportaron conjuntamente menos de 80 MiB.

CPU

En condición de reposo, los cuatro contenedores registraron consumos de CPU estrictamente puntuales (Figura 4.12).

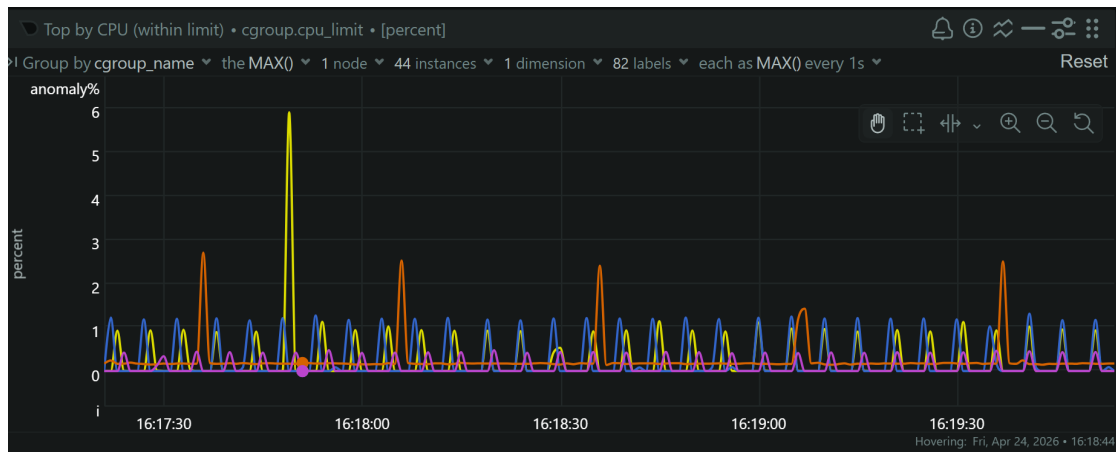


FIGURA 4.12: Consumo de CPU (% del límite del cgroup) de los contenedores del *stack* durante parte del período de observación en baseline. Picos máximos: Documenso: 5,9 %; Browserless: 2,6 %; PostgreSQL: 1,2 %; *proxy* Socat: 0,43 %.

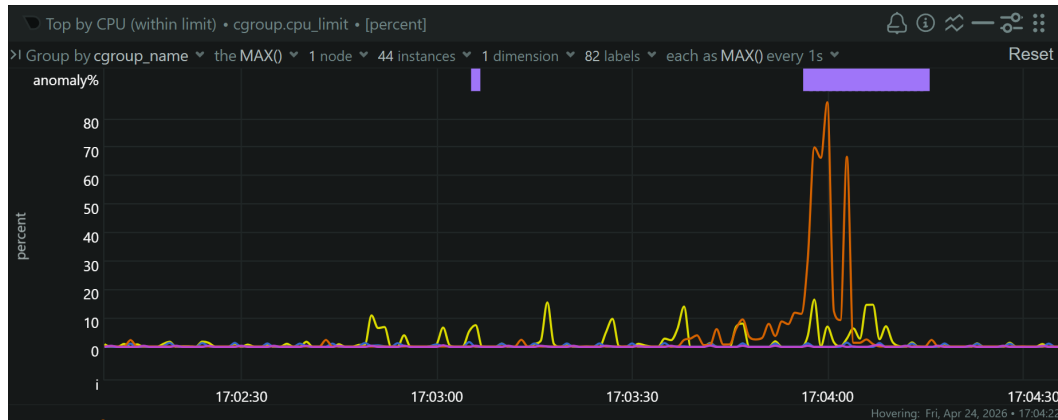
Los picos de Documenso (máximo 5,9 %) correspondieron a solicitudes de renderizado de página y no a procesamiento sostenido. Entre los instantes de actividad, todos los contenedores retornaron a valores próximos a cero. La holgura disponible respecto a los 8 vCPUs del LXC resultó amplia bajo esta carga.

4.6.2. Consumo de recursos bajo carga controlada

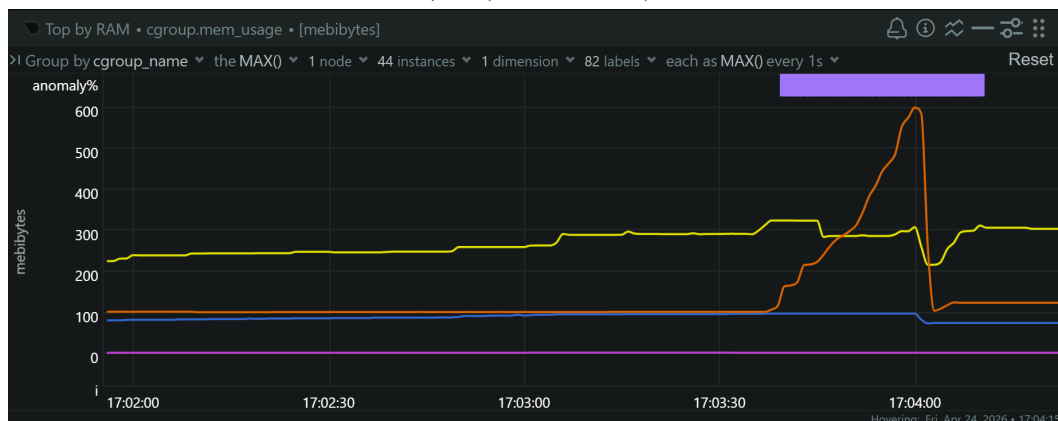
Se realizaron dos ensayos de carga sobre el *stack* en producción: el sellado de un documento único y una ráfaga de cinco documentos enviados en intervalos breves.

Ensayo 1: sellado de un documento

Las Figuras 4.13A y 4.13B registran el comportamiento del *stack* durante el sellado de un documento individual.



(A) Consumo de CPU (% del límite del cgroup) durante el sellado de un documento. Pico máximo: Browserless 86,00 %; Documentso 16,66 %.



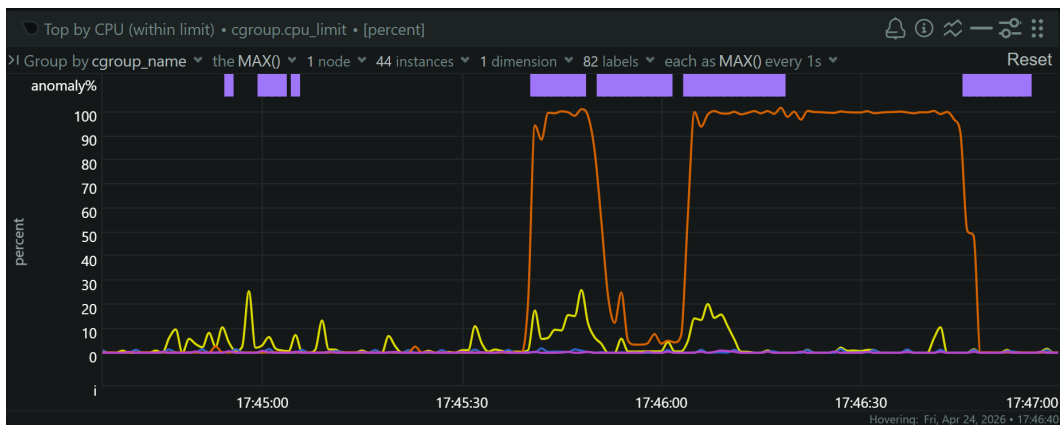
(B) Consumo de RAM (en MiB) durante el sellado de un documento. Browserless: 102 MiB → 600 MiB (pico, +498 MiB), estabilizado en 124,3 MiB (+22,3 MiB retenidos). Documentso: 215,5 MiB → 308,8 MiB (+93,3 MiB retenidos). PostgreSQL: pico transitorio de 97 MiB (+20 MiB).

FIGURA 4.13: Consumo de CPU y RAM del *stack* durante el sellado de un documento. Browserless alcanzó un pico máximo de 86,00 % de CPU y 600 MiB de RAM; al concluir el job, la CPU retornó al baseline mientras la RAM se estabilizó en 124,3 MiB. Documentso retuvo 93,3 MiB adicionales de forma permanente tras el evento.

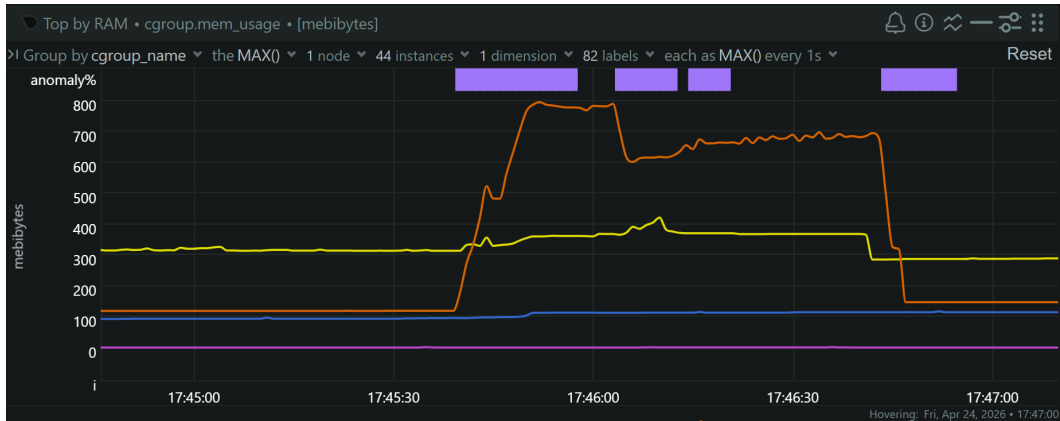
En CPU, el evento se manifestó como un pico aislado de 2 segundos de duración: Browserless alcanzó un máximo de 86,00 % mientras Documenso no superó el 16,66 %. El resto del *stack* permaneció en valores de baseline. En RAM, Browserless escaló abruptamente desde 102 MiB hasta 600 MiB (+498 MiB) para instanciar el proceso de Chrome encargado del renderizado. Al concluir el job, el proceso fue destruido y el sistema operativo recuperó la memoria; Browserless se estabilizó en 124,3 MiB (+22,3 MiB sobre su baseline) y Documenso en 308,8 MiB (+93,3 MiB). El impacto sobre PostgreSQL fue transitorio (pico de 97 MiB, +20 MiB) y retornó a su valor de baseline al concluir el job.

Ensayo 2: ráfaga de cinco documentos

Las Figuras 4.14A y 4.14B registran el comportamiento del *stack* durante la ráfaga concurrente.



(A) Consumo de CPU (% del límite del cgroup) durante la ráfaga de cinco documentos. Pico máximo sostenido: Browserless 100 %; Documenso 25 % al inicio del proceso.



(B) Consumo de RAM (en MiB) durante la ráfaga de cinco documentos. Pico máximo de Browserless: 797 MiB. Total del *stack* en el pico: $\approx 1\,338$ MiB sobre 8 GiB asignados al LXC.

FIGURA 4.14: Consumo de CPU y RAM del *stack* durante la ráfaga de cinco documentos. La ventana de actividad se extendió entre las 17:45:38 y las 17:46:50 UTC (≈ 1 min 12 s). Browserless alcanzó un pico sostenido de 100 % de CPU y un pico de RAM de 797 MiB antes de liberar ambos recursos al completarse todos los jobs.

En este ensayo, el bloque de actividad se extendió entre las 17:45:38 y las 17:46:50 UTC, aproximadamente 1 minuto 12 segundos: Browserless procesó los cinco jobs en forma secuencial, lo que explica la forma escalonada de la curva de RAM (Figura 4.14B), donde cada peldaño corresponde al lanzamiento de un proceso de Chrome adicional. El pico máximo de RAM de Browserless fue de 797 MiB. Al completarse el último job, la curva cayó a los 298 MiB de baseline en un único intervalo de muestreo.

En CPU, Browserless registró un pico agudo inicial de 100 % a las 17:45:38, un breve descenso hacia el baseline a las 17:45:52 y luego una actividad sostenida al 100 % durante aproximadamente 1 minuto hasta las 17:46:38. El pico de 100 % corresponde a 1 núcleo virtual sobre los 8 vCPUs del LXC (el 12,5 % de la capacidad total). Documenso registró un pico de 25 % al inicio del proceso, asociado al despacho inicial de los jobs de sellado, retornando luego al baseline.

Tabla comparativa de rendimiento

La Tabla 4 consolida los valores registrados en ambas subsecciones frente a la capacidad auditada del LXC.

TABLA 4: Comparativa de consumo de recursos en baseline y bajo carga controlada (ráfaga de 5 documentos). El límite total del LXC es de 8 vCPUs y 8 192 MiB de RAM.

Componente	Baseline	Pico (ráfaga)	Impacto sobre LXC
Uso de CPU (capacidad total: 8 vCPUs)			
Browserless	2,6 %	100 %	≈ 12,5 % (≈ 1 vCPU)
Documenso	5,9 %	25 %	≈ 3,1 % (≈ 0,25 vCPU)
Uso de RAM (capacidad total: 8 192 MiB)			
Browserless	≈ 102,0 MiB	≈ 797 MiB	≈ 9,7 %
Stack total	≈ 396,7 MiB (5 %)	≈ 1 338 MiB	≈ 16,3 %

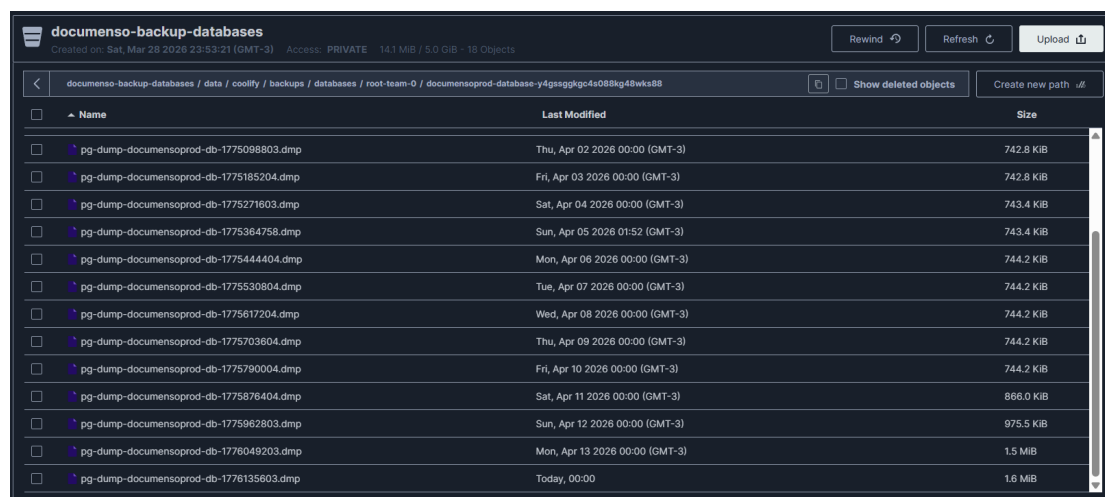
El recurso con mayor variación bajo carga fue la RAM de Browserless, que se multiplicó por ≈ 7,8 respecto al baseline. Aun en el peor caso registrado, el *stack* consumió el 16,3 % de la RAM del LXC, lo que dejó un margen de aproximadamente 6 854 MiB (≈ 6,7 GiB) disponibles. La CPU no representó un cuello de botella en ninguno de los dos ensayos.

4.7. Validación del Respaldo y Recuperación

Esta sección verifica la capa de continuidad definida en la Sección 4.1: los mecanismos de respaldo automático configurados en la Sección 3.5 del Capítulo 3 generan y persisten los artefactos esperados en el almacenamiento externo. La validación se apoya en la inspección del *bucket* de MinIO y en los metadatos de los objetos almacenados; el procedimiento operativo de restauración se documenta en el Apéndice C.

4.7.1. Respaldo de la base de datos

El *job* de respaldo de PostgreSQL fue configurado en Coolify para ejecutarse diariamente a las 03:00 y persistir los volcados en el *bucket* `documenso-backup-databases` del servidor MinIO. La Figura 4.15 muestra el contenido del bucket al 14 de abril de 2026.



Name	Last Modified	Size
pg-dump-documensoprod-db-1775098803.dmp	Thu, Apr 02 2026 00:00 (GMT-3)	742.8 KiB
pg-dump-documensoprod-db-1775185204.dmp	Fri, Apr 03 2026 00:00 (GMT-3)	742.8 KiB
pg-dump-documensoprod-db-1775271603.dmp	Sat, Apr 04 2026 00:00 (GMT-3)	743.4 KiB
pg-dump-documensoprod-db-1775364758.dmp	Sun, Apr 05 2026 01:52 (GMT-3)	743.4 KiB
pg-dump-documensoprod-db-1775444404.dmp	Mon, Apr 06 2026 00:00 (GMT-3)	744.2 KiB
pg-dump-documensoprod-db-1775530804.dmp	Tue, Apr 07 2026 00:00 (GMT-3)	744.2 KiB
pg-dump-documensoprod-db-1775617204.dmp	Wed, Apr 08 2026 00:00 (GMT-3)	744.2 KiB
pg-dump-documensoprod-db-1775703604.dmp	Thu, Apr 09 2026 00:00 (GMT-3)	744.2 KiB
pg-dump-documensoprod-db-1775790004.dmp	Fri, Apr 10 2026 00:00 (GMT-3)	744.2 KiB
pg-dump-documensoprod-db-1775876404.dmp	Sat, Apr 11 2026 00:00 (GMT-3)	866.0 KiB
pg-dump-documensoprod-db-1775962803.dmp	Sun, Apr 12 2026 00:00 (GMT-3)	975.5 KiB
pg-dump-documensoprod-db-1776049203.dmp	Mon, Apr 13 2026 00:00 (GMT-3)	1.5 MiB
pg-dump-documensoprod-db-1776135603.dmp	Today, 00:00	1.6 MiB

FIGURA 4.15: Consola de MinIO mostrando el *bucket* `documenso-backup-databases`. Se registran 18 objetos con nomenclatura `pg-dump-documensoprod-db-{timestamp}.dmp`, generados diariamente desde el 28 de marzo de 2026. El tamaño total de los artefactos es de 14,1 MiB.

El listado registra 18 volcados con una cadencia estrictamente diaria: la marca de tiempo UNIX incluida en cada nombre de archivo garantiza la unicidad del artefacto e impide colisiones de nombres ante reinicios del *job*. Los volcados se almacenan bajo una ruta que contiene el identificador único del contenedor de base de datos.

El tamaño de los artefactos refleja el crecimiento orgánico de la base de datos: los volcados del 2 al 10 de abril se mantuvieron en el rango de 742–744 KiB, período que coincide con el despliegue inicial sin carga de documentos. A partir del 11 de abril, coincidiendo con los ensayos de firma documentados en la Sección 4.5, el tamaño escaló progresivamente hasta 1,6 MiB al 14 de abril, lo que evidencia que cada ciclo de firma incorporó registros nuevos a la base de datos.

La validación de retención para este dominio se realizó mediante inspección directa de metadatos sobre un volcado generado el 7 de abril dentro del *bucket* `documenso-backup-databases`. En ese objeto, el atributo `X-Amz-Object-Lock-Mode` registró el valor `GOVERNANCE` y el atributo `X-Amz-Object-Lock-Retain-Until-Date` fijó el vencimiento para el 7 de mayo de 2026. Esta evidencia confirma una retención efectiva de 30 días, consistente con la política definida en la Sección 3.5.

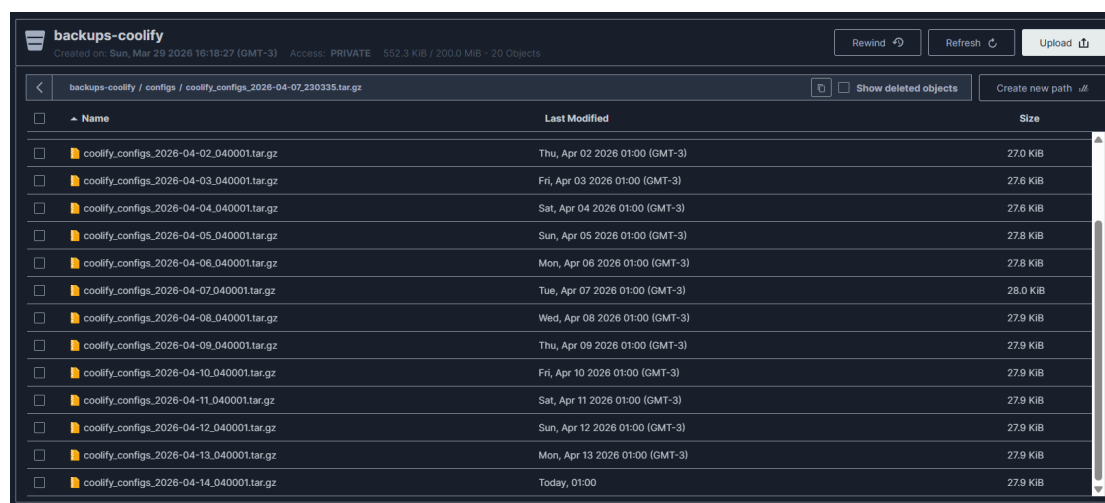
4.7.2. Prueba de restauración de la base de datos

Se ejecutó un simulacro de restauración a partir de uno de los volcados almacenados en el bucket `documenso-backup-databases`. Coolify orquestó el comando `pg_restore --clean` sobre el contenedor de PostgreSQL, lo que forzó el reemplazo completo del estado previo de la

base de datos. La integridad del resultado se verificó accediendo a la aplicación y confirmando que los registros de documentos y firmantes se encontraban íntegros. El procedimiento detallado para importar y restaurar los respaldos se documenta en el Apéndice C.1.

4.7.3. Respaldo de la configuración del *stack*

El *script* `backup-coolify.sh`, configurado mediante *cron* para ejecutarse diariamente a las 04:00, empaqueta los manifiestos y secretos del despliegue y transfiere el resultado al *bucket* `backups-coolify` de MinIO. La Figura 4.16 muestra el contenido del bucket al 14 de abril de 2026.



Name	Last Modified	Size
coolify_configs_2026-04-02_040001.tar.gz	Thu, Apr 02 2026 01:00 (GMT-3)	27.0 KiB
coolify_configs_2026-04-03_040001.tar.gz	Fri, Apr 03 2026 01:00 (GMT-3)	27.6 KiB
coolify_configs_2026-04-04_040001.tar.gz	Sat, Apr 04 2026 01:00 (GMT-3)	27.6 KiB
coolify_configs_2026-04-05_040001.tar.gz	Sun, Apr 05 2026 01:00 (GMT-3)	27.8 KiB
coolify_configs_2026-04-06_040001.tar.gz	Mon, Apr 06 2026 01:00 (GMT-3)	27.8 KiB
coolify_configs_2026-04-07_040001.tar.gz	Tue, Apr 07 2026 01:00 (GMT-3)	28.0 KiB
coolify_configs_2026-04-08_040001.tar.gz	Wed, Apr 08 2026 01:00 (GMT-3)	27.9 KiB
coolify_configs_2026-04-09_040001.tar.gz	Thu, Apr 09 2026 01:00 (GMT-3)	27.9 KiB
coolify_configs_2026-04-10_040001.tar.gz	Fri, Apr 10 2026 01:00 (GMT-3)	27.9 KiB
coolify_configs_2026-04-11_040001.tar.gz	Sat, Apr 11 2026 01:00 (GMT-3)	27.9 KiB
coolify_configs_2026-04-12_040001.tar.gz	Sun, Apr 12 2026 01:00 (GMT-3)	27.9 KiB
coolify_configs_2026-04-13_040001.tar.gz	Mon, Apr 13 2026 01:00 (GMT-3)	27.9 KiB
coolify_configs_2026-04-14_040001.tar.gz	Today, 01:00	27.9 KiB

FIGURA 4.16: Consola de MinIO mostrando el *bucket* `backups-coolify`. Se registran 20 objetos con nomenclatura `coolify_configs_YYYY-MM-DD_HH:mm:ss.tar.gz`, generados diariamente desde el 28 de marzo de 2026. El tamaño total de los artefactos es de 552,3 KiB.

Los 20 artefactos presentan un tamaño estable en el rango de 27,0–28,0 KiB, lo que indica que las configuraciones del conjunto de todos los proyectos administrados por Coolify (incluyendo Documenso) se mantuvieron sin variaciones significativas. La inspección interna del archivo confirmó la presencia de los manifiestos `docker-compose.yml` y los secretos `.env` correspondientes, garantizando que el respaldo contiene la información suficiente para reconstruir el entorno ante un escenario de recuperación de desastres. El procedimiento de inspección y extracción de estos respaldos se detalla en el Apéndice C.

En el dominio de configuraciones, la comprobación se efectuó sobre un respaldo del 7 de abril almacenado en el *bucket* `backups-coolify`. La lectura de metadatos mostró nuevamente `X-Amz-Object-Lock-Mode=GOVERNANCE`, mientras que `X-Amz-Object-Lock-Retain-Until-Date` proyectó la retención hasta el 4 de octubre de 2026. Esta combinación válida que la política aplicada en este dominio corresponde a 180 días. El procedimiento de restauración del *stack* a partir de estos artefactos se documenta en el Apéndice C.2.

4.7.4. Validación de las políticas de ciclo de vida

Las políticas de ciclo de vida configuradas en la Sección 3.5 del Capítulo 3 se verificaron mediante el comando `mc ilm ls` del cliente de MinIO, ejecutado sobre cada uno de los tres *buckets* del sistema. Esta verificación es crítica porque, aunque la aplicación no explota versiones de objetos en su operación diaria, la activación de *Object Lock* fuerza el versionado a nivel de infraestructura. En este contexto, la salida del comando demuestra que el motor ILM opera como mecanismo de *Garbage Collection* para purgar *Delete Markers* y evitar el colapso de las cuotas de almacenamiento. La Figura 4.17 presenta la salida del comando.

Las reglas registradas son coherentes con el diseño: el *bucket* documental (*documenso-prod*) conserva la versión vigente de cada objeto durante 1 825 días (cinco años) y elimina las versiones anteriores tras un día, mientras que los *buckets* de respaldo aplican retenciones de 30 y 180 días respectivamente. En los tres casos, el campo `KEEP VERSIONS` registra el valor 0, lo que indica que las versiones no vigentes no se acumulan más allá del plazo configurado.

```
fblasich@fblasich-Lenovo-IdeaPad-S145-15IWL:~$ mc ilm ls truenas-minio/documenso-prod
```

Expiration for latest version (Expiration)					
ID	STATUS	PREFIX	TAGS	DAYS TO EXPIRE	EXPIRE DELETEMARKER
d75eh5i8t83mr5ith4lg	Enabled	-	-	1825	false

Expiration for older versions (NoncurrentVersionExpiration)					
ID	STATUS	PREFIX	TAGS	DAYS TO EXPIRE	KEEP VERSIONS
d75eh5i8t83mr5ith4lg	Enabled	-	-	1	0

(A) *Bucket* *documenso-prod*: expiración de la versión vigente a los 1 825 días y eliminación de versiones anteriores tras 1 día.

```
fblasich@fblasich-Lenovo-IdeaPad-S145-15IWL:~$ mc ilm ls truenas-minio/documenso-backup-databases
```

Expiration for latest version (Expiration)					
ID	STATUS	PREFIX	TAGS	DAYS TO EXPIRE	EXPIRE DELETEMARKER
d75dto28t83l0bgf0h50	Enabled	-	-	30	false

Expiration for older versions (NoncurrentVersionExpiration)					
ID	STATUS	PREFIX	TAGS	DAYS TO EXPIRE	KEEP VERSIONS
d75dto28t83l0bgf0h50	Enabled	-	-	30	0

(B) *Bucket* *documenso-backup-databases*: expiración y eliminación de versiones anteriores a los 30 días.

```
fblasich@fblasich-Lenovo-IdeaPad-S145-15IWL:~$ mc ilm ls truenas-minio/backups-coolify
```

Expiration for latest version (Expiration)					
ID	STATUS	PREFIX	TAGS	DAYS TO EXPIRE	EXPIRE DELETEMARKER
d75dur28t83l0v3kj41g	Enabled	-	-	180	false

Expiration for older versions (NoncurrentVersionExpiration)					
ID	STATUS	PREFIX	TAGS	DAYS TO EXPIRE	KEEP VERSIONS
d74q8628t83o1ie2a77g	Enabled	-	-	180	0

(C) *Bucket backups-coolify*: expiración y eliminación de versiones anteriores a los 180 días.

FIGURA 4.17: Salida del comando `mc ilm ls` sobre los tres *buckets* del despliegue. Cada regla indica el tipo de transición, el período de retención y el estado activo de la política.

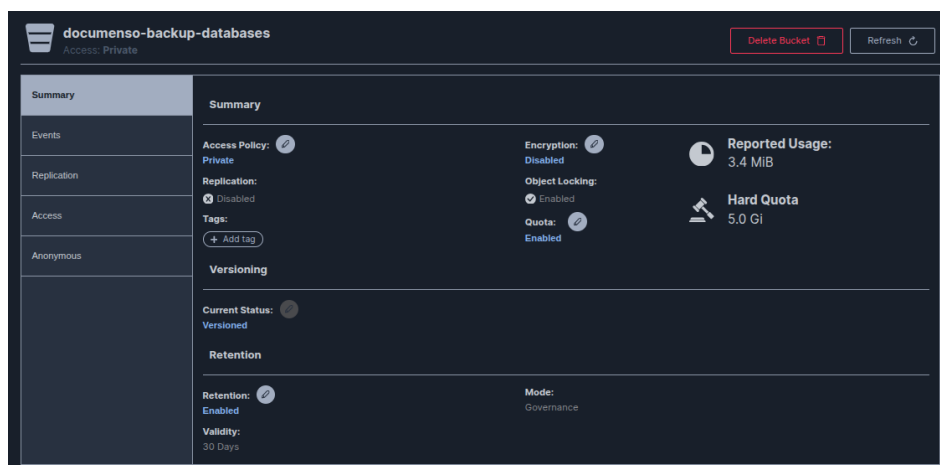
La interfaz web de MinIO permite corroborar la configuración de cada *bucket* desde una perspectiva complementaria. La Figura 4.18 muestra el panel de administración con el estado del versionado, el bloqueo de objetos y la política de retención activa.

The screenshot displays the MinIO web console for the 'documenso-prod' bucket. The left sidebar contains navigation links: Summary, Events, Replication, Access, and Anonymous. The main content area is divided into several sections:

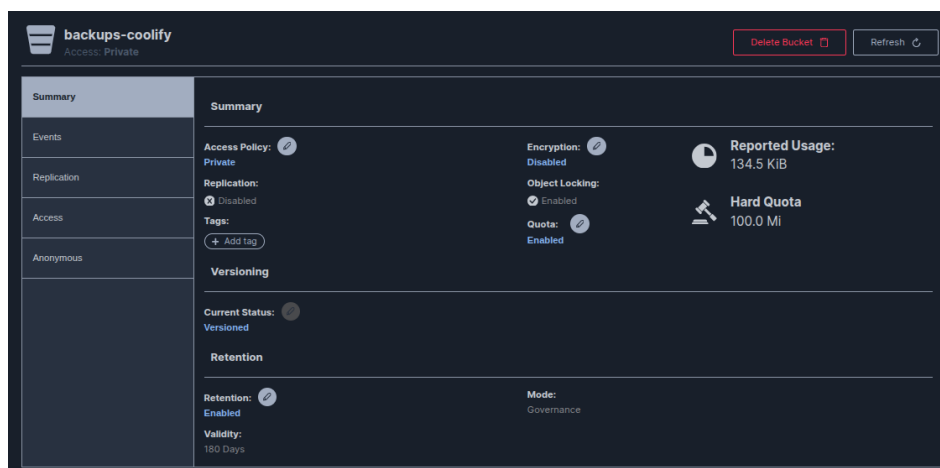
- Summary:** Includes 'Access Policy: Private', 'Replication: Disabled', 'Tags: + Add tag', 'Encryption: Disabled', 'Object Locking: Enabled', 'Quota: Enabled', 'Reported Usage: 4.4 MiB', and 'Hard Quota: 2.0 Ti'.
- Versioning:** Shows 'Current Status: Versioned'.
- Retention:** Shows 'Retention: Enabled', 'Validity: 1825 Days', and 'Mode: Governance'.

 At the top right, there are buttons for 'Delete Bucket' and 'Refresh'.

(A) *Bucket documenso-prod*: Object Locking habilitado, versionado activo, retención Governance de 1 825 días, cuota de 2,0 TiB.



(B) Bucket `documenso-backup-databases`: Object Locking habilitado, versionado activo, retención Governance de 30 días, cuota de 5,0 GiB.



(C) Bucket `backups-coolify`: Object Locking habilitado, versionado activo, retención Governance de 180 días, cuota de 100,0 MiB.

FIGURA 4.18: Panel de administración de MinIO para los tres *buckets* del despliegue. Se observan el estado del versionado, el bloqueo de objetos, la política de retención y la cuota asignada.

Los resultados confirman que las tres capas de protección —reglas ILM, bloqueo de objetos en modo Governance y cuotas de almacenamiento— se encuentran activas y operativas en cada *bucket* del sistema.

4.8. Trazabilidad y Cumplimiento de Requisitos

Esta sección cierra el ciclo de validación. La Tabla 5 vincula cada requisito del alcance definido en el Capítulo 1 con la sección del Capítulo 3 donde fue abordado y con la evidencia empírica que lo valida. A continuación, la Tabla 6 retoma el análisis del estado del arte para contrastar las condiciones de las plataformas comerciales con los resultados de la infraestructura desplegada.

4.8.1. Matriz de trazabilidad

TABLA 5: Trazabilidad entre los requisitos del Capítulo 1, el diseño del Capítulo 3 y la validación del presente capítulo. Las columnas “Diseño” y “Validación” indican el número de la sección correspondiente.

Requisito	Diseño	Validación	Resultado
Selección de plataforma de código abierto	3.3	4.5	Cumplido. Flujo <i>end-to-end</i> validado con usuarios reales.
Despliegue contenerizado sobre infraestructura institucional (Proxmox, Colify)	3.2	4.2	Cumplido. Estado <i>Healthy</i> alcanzado; restricción AVX mitigada.
Autenticación restringida al dominio institucional	3.6.2	4.4	Cumplido. Acceso institucional validado; dominios externos bloqueados con 403.
Integración con almacenamiento de objetos (MinIO) mediante <i>proxy</i> Socat	3.4	4.3.1	Cumplido. Lectura y escritura por túnel TCP con HTTP 200 OK.
Correo transaccional con entregabilidad verificada (Brevo SMTP)	3.6.1	4.3.3	Cumplido. SPF, DKIM y DMARC en <i>pass</i> ; entregabilidad verificada.
Firma electrónica con certificado institucional	3.3.3	4.5.2	Cumplido. Sello validado en Adobe Acrobat; limitación de confianza documentada.
Estrategia de respaldo y retención de datos	3.5	4.7	Cumplido. ILM y <i>Object Lock</i> activos.
Consumo de recursos dentro de la capacidad del LXC	3.2	4.6	Cumplido. RAM pico 16,3 %; CPU dentro de umbrales operativos.
Acceso web <i>clientless</i> (usabilidad)	3.3.3	4.5	Cumplido. Operatividad desde navegadores estándar sin software adicional.
Elaboración del manual de mantenimiento	N/A	Apéndice C	Cumplido. Procedimientos técnicos de mantenimiento documentados.
Elaboración del manual de usuarios	N/A	Apéndice D	Cumplido. Procedimientos operativos de uso documentados.

4.8.2. Comparativa con el estado del arte

La Tabla 6 retoma el análisis de mercado presentado en la Sección 1.4 del Capítulo 1 y lo contrasta con los resultados de la solución implementada. El propósito no es establecer una superioridad absoluta sobre los proveedores comerciales, sino verificar que la alternativa de código abierto resolvió los condicionantes que motivaron su adopción en el contexto institucional de la FACET.

TABLA 6: Contraste entre las condiciones de las plataformas comerciales relevadas en la Sección 1.4 y los resultados de la infraestructura desplegada.

Aspecto	Plataformas <i>SaaS</i>	Solución implementada
Costo de licenciamiento (20 usuarios)	≈ 5 326 USD anuales recurrentes	Costo de licencia nulo
Soberanía de los datos	Retención y ubicación dependientes del proveedor	<i>On-Premise</i> (servidores propios de la UNT)
Gestión de identidad	SSO condicionado a plan <i>Enterprise</i>	SSO con Google OAuth 2.0 sin costo <i>Enterprise</i> adicional
Límites de uso	Límites por usuario/año (p. ej., 100 documentos)	Limitado solo por hardware institucional (2 TB)
Inmutabilidad y retención	Retención opaca y dependiente de políticas del proveedor	Control total mediante <i>Object Lock</i> e ILM a nivel de <i>bucket</i>
Certificado de firma	Emitido y administrado por el proveedor	Certificado institucional generado y controlado por la FACET
Carga operativa y disponibilidad	Cubierta por SLA del proveedor	Asumida por la institución (<i>trade-off</i> operativo)

La adopción del esquema *On-Premise* implica que el área de TI de la facultad asume la carga operativa que en los servicios comerciales queda cubierta por contrato. Como contrapartida, la solución eliminó el costo recurrente de 5 326 USD anuales proyectado en el estudio inicial, mantuvo los documentos dentro de la infraestructura propia de la UNT y integró el acceso al directorio institucional sin dependencia de un proveedor externo.

Capítulo 5

Conclusiones

5.1. Conclusiones Generales

La solución implementada consolidó una arquitectura de nivel empresarial para firma electrónica sobre infraestructura institucional de la FACET. El despliegue integró Documenso como núcleo de aplicación, PostgreSQL para persistencia transaccional, Browserless para renderizado del certificado de firma, MinIO como almacenamiento de objetos y el *proxy* Socat para enlazar la red Docker con el servidor físico de almacenamiento, bajo la orquestación de Coolify y Traefik en un contenedor LXC sobre Proxmox VE. Las verificaciones del Capítulo 4 —desde *healthchecks* de servicios hasta ensayos de extremo a extremo y restauración de respaldos— validaron el cumplimiento de los objetivos funcionales, de seguridad, de rendimiento y de continuidad operativa definidos en la Sección 1.3, en concordancia con la matriz de trazabilidad de la Subsección 4.8.1.

La adopción de una alternativa de código abierto auditada y desplegada en entorno propio entregó un retorno de inversión concreto: eliminó el costo recurrente de licencias comerciales, preservó la soberanía total de los datos y evitó la dependencia contractual de proveedores externos de *software*. Frente al escenario de suscripción anual proyectado en el Capítulo 1, la arquitectura implementada resultó financieramente sustentable para una institución pública, al trasladar la inversión hacia capacidades internas reutilizables de integración, seguridad y operación.

Este resultado constituyó, además, uno de los primeros servicios productivos desplegados íntegramente sobre la infraestructura del DEEC. La operación sostenida confirmó que los recursos disponibles de virtualización, almacenamiento y red —Proxmox VE, TrueNAS/MinIO y Coolify— soportan cargas reales con criterios de disponibilidad, seguridad y trazabilidad equivalentes a un entorno operativo formal. En términos institucionales, la implementación establece una base operativa concreta sobre la que la FACET puede desplegar futuros servicios sin partir de cero.

La integración de este ecosistema puso en juego conocimientos transversales de la carrera: la orquestación de la topología requirió aplicar conceptos de sistemas operativos y protocolos de comunicación TCP/IP para segmentar el tráfico mediante túneles y *proxies* inversos; el diseño de la retención de datos se sustentó en principios de bases de datos y continuidad operativa; el diagnóstico de las limitaciones del procesador demostró que la arquitectura de computadoras es determinante en decisiones de compatibilidad; y la evaluación de alternativas y el análisis de impacto económico se apoyaron en criterios de gestión de proyectos y planificación tecnológica.

Lo que este proyecto enseñó con mayor claridad es que los registros, las métricas y los respaldos no son un complemento del sistema, sino su columna vertebral. Sin los registros de contenedores no habría sido posible diagnosticar el fallo de sellado de Browserless ni identificar los flujos de acceso a MinIO; sin las métricas de la Sección 4.6, los picos de RAM de Browserless bajo carga habrían sido invisibles hasta convertirse en un problema real. Y lo que quedó más claro de todo es que ninguna intervención sobre el entorno productivo es aceptable sin un respaldo verificado previo: la prueba de restauración documentada en la Sección 4.7 no fue un ejercicio opcional, fue la única manera de saber con certeza que los volcados almacenados en MinIO realmente servían para algo.

En cuanto a la continuidad operativa, la estrategia de respaldo automatizado hacia el almacenamiento externo —junto con los procedimientos de restauración del Apéndice C y la guía de inicio rápido del Apéndice D— garantizan que el sistema pueda ser recuperado y operado por el personal de la facultad sin depender del autor original. Esa transferencia de conocimiento es, en última instancia, el criterio más exigente de cualquier despliegue productivo: que el servicio sobreviva a quien lo construyó.

5.2. Trabajo Futuro

Si bien el sistema cumple con los objetivos planteados y se encuentra operativo en producción, la arquitectura actual admite tres ejes de evolución prioritarios: resiliencia, escalabilidad horizontal y extensión institucional.

- Respaldo *off-site* y replicación. Tal como se advirtió en la Sección 3.5.2 del diseño, el servidor TrueNAS actual constituye un punto único de fallo (*SPOF*) para el repositorio documental. La siguiente etapa debe implementar *Cross-Site Replication* en MinIO hacia un nodo geográficamente separado o hacia un proveedor de nube pública, para asegurar capacidad efectiva de *Disaster Recovery* ante incidentes de sitio completo.
- Alta Disponibilidad del nodo de cómputo. Si bien el repositorio documental queda salvaguardado mediante la replicación descrita en el punto anterior, el servidor físico actual constituye un *SPOF* a nivel de cómputo: una interrupción del *hardware* del nodo provoca la caída inmediata del servicio. La arquitectura debe evolucionar hacia un clúster de Alta Disponibilidad gestionado mediante *Docker Swarm* o *Kubernetes/K3s*, de modo que la carga se redistribuya ante la falla de un nodo y el servicio mantenga continuidad operativa.
- Migración de *hardware* y actualización de versión. El servidor actual utiliza procesadores sin soporte de instrucciones *AVX*, lo que restringe el despliegue a Documenso v2.1.0. La migración a *hardware* compatible habilitará la adopción de ramas superiores con nuevas funcionalidades y parches de seguridad vigentes; el procedimiento operativo de actualización se encuentra documentado en el Apéndice C.4.
- Extensión del servicio a otras dependencias de la universidad. El modelo de organizaciones y equipos de Documenso permite incorporar nuevas facultades o áreas de la UNT sobre la misma plataforma, preservando aislamiento lógico por unidad organizativa y evitando duplicación innecesaria de infraestructura.

- Obtención de certificado emitido por una autoridad certificante de confianza. La infraestructura desplegada soporta de forma nativa certificados X.509 emitidos por autoridades certificantes licenciadas. La incorporación de un certificado anclado en una cadena de confianza pública permitirá evolucionar de firma electrónica a firma digital conforme a la Ley 25.506, sin rediseñar la arquitectura base.

Apéndice A

Diagramas y Topologías de Red

En este apéndice se presentan versiones ampliadas de los diagramas incluidos en el Capítulo 3. Su propósito es facilitar la lectura de etiquetas, relaciones de red y recorridos de tráfico que, por razones de escala, en el cuerpo principal del documento se muestran de forma reducida. Los diagramas fueron elaborados con la herramienta FossFLOW [38].

A.1. Topología General de la Infraestructura

La figura reproduce en mayor tamaño la topología general de la plataforma. Esta vista permite distinguir con claridad las diferentes zonas de red involucradas en el despliegue —red externa (WAN), LAN institucional, contenedor LXC y servicios contenerizados—, así como el recorrido completo del tráfico hasta alcanzar la aplicación.

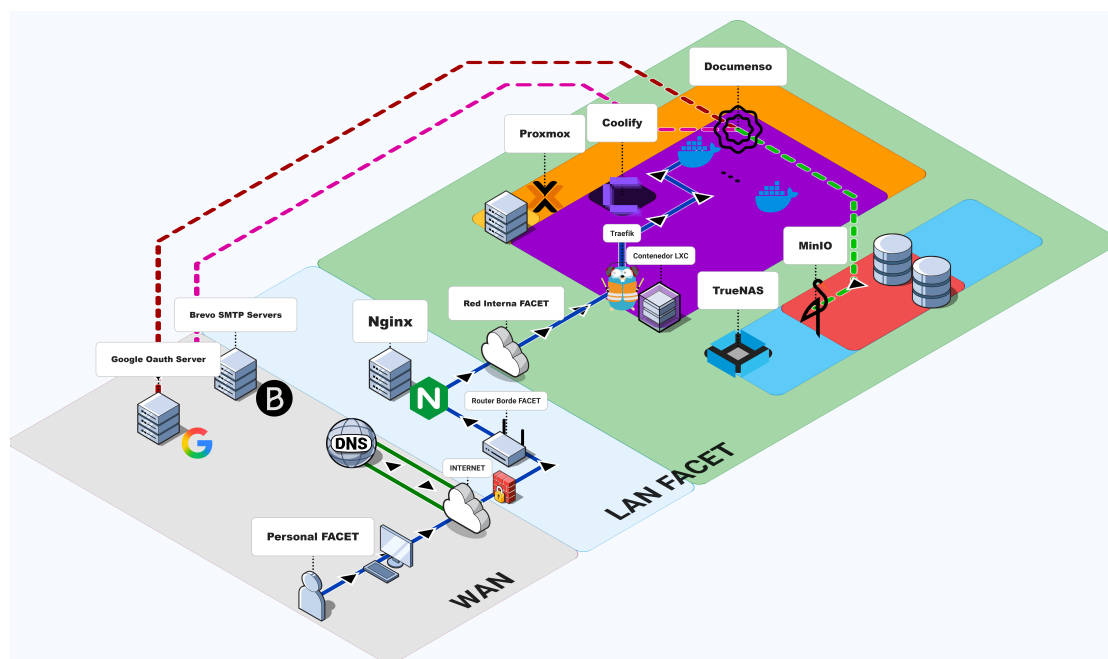


FIGURA A.1: Vista ampliada de la topología general de la infraestructura y del flujo de tráfico entre WAN, LAN institucional y entorno contenerizado. Versión a escala completa de la Fig. 3.1 del Capítulo 3.

A.2. Arquitectura Interna de la PaaS y Servicios Contenerizados

Profundizando en la capa de orquestación, la figura detalla la arquitectura interna de la plataforma Coolify. En ella se aprecia el aislamiento provisto por el contenedor LXC base, la segmentación mediante redes virtuales de Docker y la separación lógica entre el plano de control de la PaaS y los servicios de usuario desplegados sobre ella.

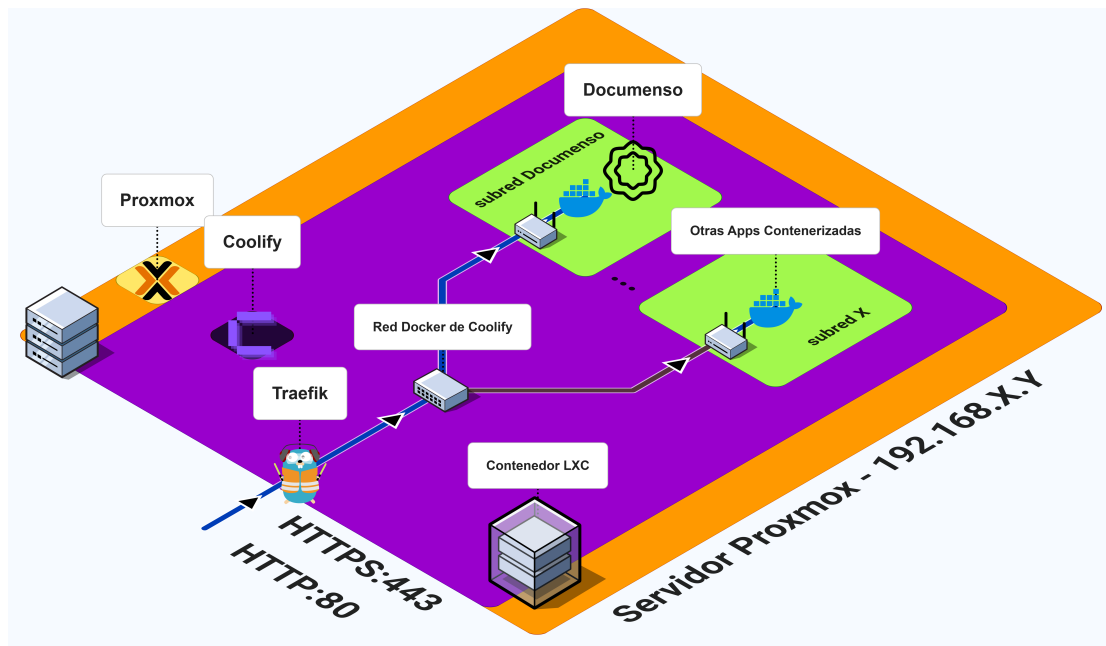


FIGURA A.2: Vista ampliada de la arquitectura interna de Coolify y de la segmentación de redes entre los servicios contenerizados desplegados en el LXC base. Versión a escala completa de la Fig. 3.4 del Capítulo 3.

A.3. Topología Interna de la Red Docker del Stack Documenso

A nivel de aplicación, la figura expone la topología interna específica del despliegue de Documenso. La vista identifica con precisión los componentes del *stack*, sus dependencias internas y el rol fundamental de Traefik como punto único de entrada a la red privada administrada por Coolify.

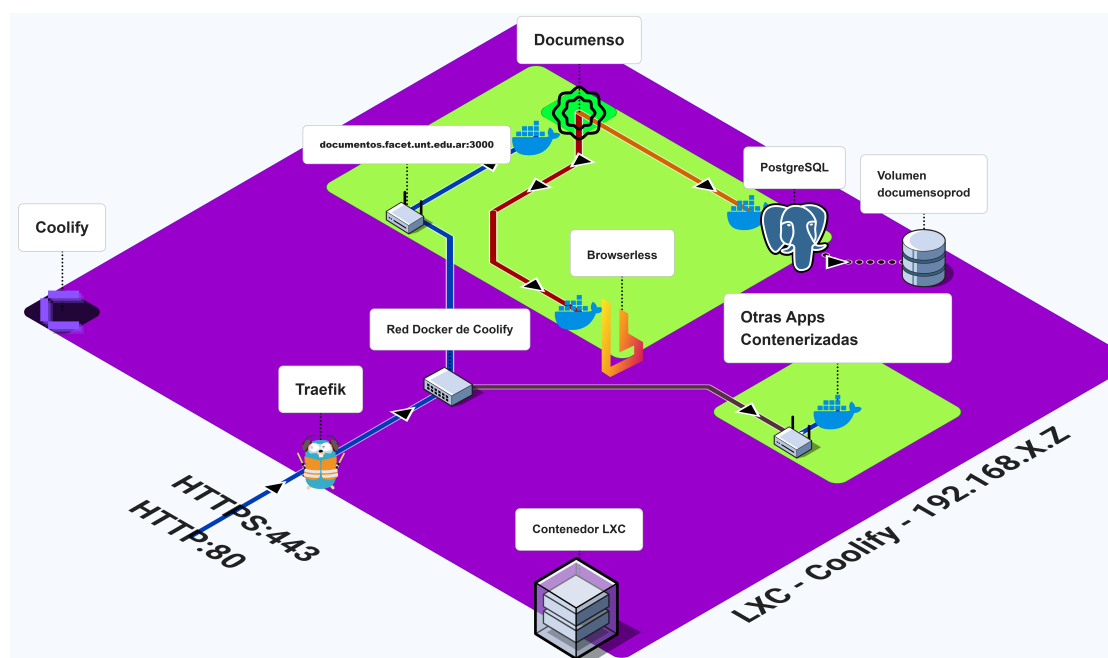


FIGURA A.3: Vista ampliada de la comunicación entre servicios contenerizados y del enrutamiento de Traefik dentro de la red privada del *stack* desplegado por Coolify. Versión a escala completa de la Fig. 3.5 del Capítulo 3.

A.4. Flujos de Acceso a MinIO a Través del Proxy Socat

Finalmente, la figura desglosa el esquema de conectividad hacia el almacenamiento descrito en la Sección 3.4. Las líneas discontinuas representan los dos flujos convergentes hacia el dominio público: en verde, el flujo directo originado en el navegador para los recursos gráficos; y en naranja, el flujo indirecto originado en el *backend* de Documenso para la gestión de documentos PDF. A partir de su convergencia en el *reverse proxy* perimetral, el recorrido unificado se ilustra mediante líneas continuas a través de Traefik, el túnel TCP del contenedor socat y, finalmente, el servidor físico TrueNAS/MinIO.

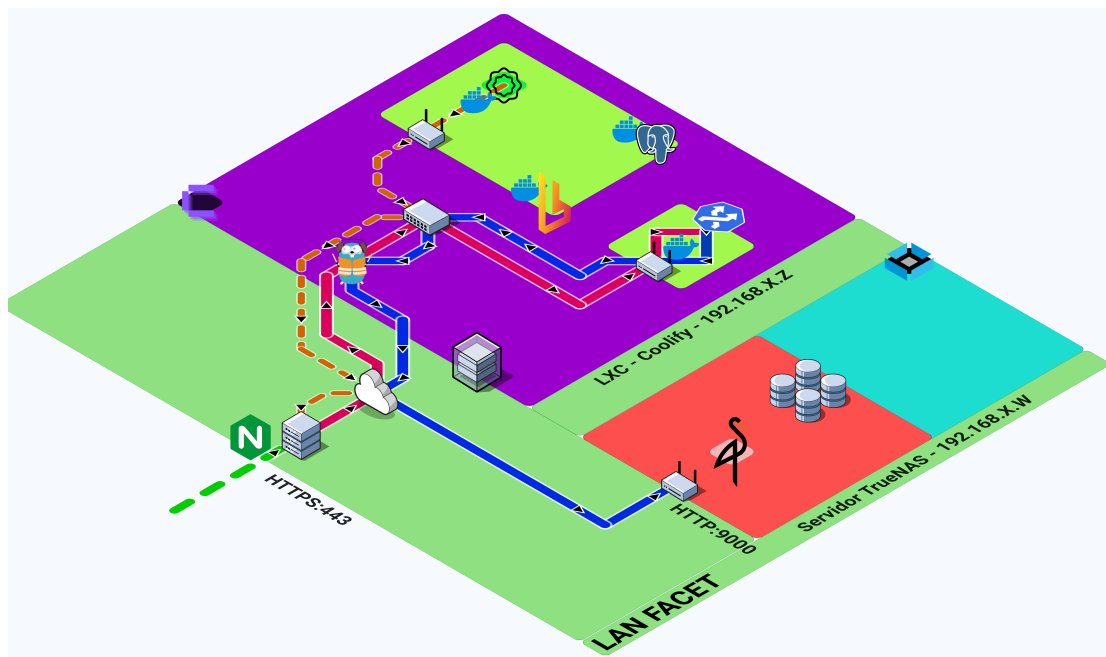


FIGURA A.4: Vista ampliada de la convergencia del flujo directo del navegador y del flujo indirecto del *backend* hacia el servidor físico TrueNAS/MinIO, a través del reverse proxy perimetral, Traefik y el contenedor Socat. Versión a escala completa de la Fig. 3.8 del Capítulo 3.

Apéndice B

Composición de Servicios y Scripts de Despliegue

Este apéndice consolida los artefactos de configuración referenciados en el Capítulo 3: las composiciones Docker del despliegue, los comandos de administración inicial y los *scripts* de operación y mantenimiento. Se documentan además las variables de entorno del despliegue productivo, la generación del material criptográfico institucional y la parametrización de los servicios externos integrados en la solución.

B.1. Composición Docker Completa del Despliegue

En este apartado se incluyen las composiciones Docker utilizadas en el despliegue de la plataforma. El Listado B.1 presenta el manifiesto principal del proyecto Documenso en Coolify, con la definición conjunta de los servicios documenso, database y browserless, sus dependencias, verificaciones de estado y volumen persistente. El Listado B.2 corresponde al proyecto independiente del *proxy* Socat, desplegado como servicio separado en Coolify para la conectividad con MinIO.

```

1 services:
2   documenso:
3     image: 'documenso/documenso:v2.1.0'
4     depends_on:
5       database:
6         condition: service_healthy
7     environment:
8       - NEXT_PRIVATE_JOBS_PROVIDER=local
9       - 'NEXT_PRIVATE_INTERNAL_WEBAPP_URL=http://documenso:3000'
10      - 'NEXT_PRIVATE_BROWSERLESS_URL=ws://browserless:3000'
11      - SERVICE_URL_DOCUMENSO_3000
12      - 'NEXTAUTH_URL=${SERVICE_URL_DOCUMENSO}'
13      - 'NEXTAUTH_SECRET=${SERVICE_BASE64_AUTHSECRET}'
14      - 'NEXT_PRIVATE_ENCRYPTION_KEY=${SERVICE_BASE64_ENCRYPTIONKEY}'
15    },
16    - 'NEXT_PRIVATE_ENCRYPTION_SECONDARY_KEY=${SERVICE_BASE64_SECONDARYENCRYPTIONKEY}'
17    - 'NEXT_PUBLIC_WEBAPP_URL=${SERVICE_URL_DOCUMENSO}'
18    - 'NEXT_PRIVATE_RESEND_API_KEY=${NEXT_PRIVATE_RESEND_API_KEY}'

```



```

18         - 'NEXT_PRIVATE_SMTP_TRANSPORT=${NEXT_PRIVATE_SMTP_TRANSPORT}'
19     ,
20     - 'NEXT_PRIVATE_SMTP_HOST=${NEXT_PRIVATE_SMTP_HOST}'
21     - 'NEXT_PRIVATE_SMTP_PORT=${NEXT_PRIVATE_SMTP_PORT}'
22     - 'NEXT_PRIVATE_SMTP_USERNAME=${NEXT_PRIVATE_SMTP_USERNAME}'
23     - 'NEXT_PRIVATE_SMTP_PASSWORD=${NEXT_PRIVATE_SMTP_PASSWORD}'
24     - 'NEXT_PRIVATE_SMTP_FROM_NAME=${NEXT_PRIVATE_SMTP_FROM_NAME}'
25     ,
26     - 'NEXT_PRIVATE_SMTP_FROM_ADDRESS=${
NEXT_PRIVATE_SMTP_FROM_ADDRESS}'
27     - 'NEXT_PRIVATE_DATABASE_URL=postgresql://${
SERVICE_USER_POSTGRES}:${SERVICE_PASSWORD_POSTGRES}@database/${
POSTGRES_DB:-documensoprod-db}?schema=public'
28     - 'NEXT_PRIVATE_DIRECT_DATABASE_URL=postgresql://${
SERVICE_USER_POSTGRES}:${SERVICE_PASSWORD_POSTGRES}@database/${
POSTGRES_DB:-documensoprod-db}?schema=public'
29     - NEXT_PRIVATE_SIGNING_TRANSPORT=local
30     - 'NEXT_PRIVATE_SIGNING_PASSPHRASE=${
SERVICE_PASSWORD_DOCUMENSO}'
31     - 'NEXT_PRIVATE_SIGNING_LOCAL_FILE_CONTENTS=${
CERTIFICADO_P12_BASE64}'
32     - 'NEXT_PUBLIC_DISABLE_SIGNUP=${DISABLE_LOGIN:-false}'
33     healthcheck:
34     test:
35     - CMD-SHELL
36     - "wget -q -O - http://documenso:3000/ | grep -q 'Sign in
to your account'"
37     interval: 5s
38     timeout: 10s
39     retries: 20
40     database:
41     image: 'postgres:17'
42     environment:
43     - 'POSTGRES_USER=${SERVICE_USER_POSTGRES}'
44     - 'POSTGRES_PASSWORD=${SERVICE_PASSWORD_POSTGRES}'
45     - 'POSTGRES_DB=${POSTGRES_DB:-documensoprod-db}'
46     volumes:
47     - 'documensoprod-postgresql-data:/var/lib/postgresql/data'
48     healthcheck:
49     test:
50     - CMD-SHELL
51     - 'pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}'
52     interval: 5s
53     timeout: 20s
54     retries: 10
55     browserless:
56     image: 'browserless/chrome:1.61-chrome-stable'
57     restart: always
58     deploy:
59     resources:
60     limits:
61     cpus: '2.0'
62     memory: 2G
63     environment:
64     - MAX_CONCURRENT_SESSIONS=5
65     - MAX_QUEUE_LENGTH=20

```

```

64         - TIMEOUT=60000
65         - DEFAULT_IGNORE_HTTPS_ERRORS=true
66     extra_hosts:
67         - 'documentos.facet.unt.edu.ar:host-gateway'
68     healthcheck:
69         test:
70             - CMD
71             - curl
72             - '-f'
73             - 'http://localhost:3000/'
74         interval: 30s
75         timeout: 10s
76         retries: 3
77 volumes:
78     documensoprod-postgresql-data:
79         external: true

```

CÓDIGO B.1: Archivo docker-compose.yml completo utilizado como base del despliegue

```

1 services:
2     minio-proxy:
3         image: alpine/socat
4         command: 'TCP4-LISTEN:9000,fork,reuseaddr TCP4:192.168.X.W:9000'
5         expose:
6             - '9000'
7         healthcheck:
8             test: ["CMD", "nc", "-z", "127.0.0.1", "9000"]
9             interval: 5s
10            timeout: 2s
11            retries: 5

```

CÓDIGO B.2: Archivo docker-compose.yml del proxy Socat hacia MinIO

B.2. Asignación Inicial del Rol Administrador

La documentación oficial de Documenso establece que el primer usuario con rol ADMIN debe asignarse directamente en la base de datos [31]. En este despliegue, la operación se realizó desde la terminal de gestión de PostgreSQL expuesta por Coolify, mediante la conexión y la sentencia SQL que se presentan en el Listado B.3.

```

1 psql -U $POSTGRES_USER -d $POSTGRES_DB
2 UPDATE "User" SET roles = '{ADMIN}' WHERE email = 'fblasich0@gmail.com';

```

CÓDIGO B.3: Conexión a PostgreSQL y sentencia SQL para asignar el rol administrador al primer usuario

B.3. Automatización del Resguardo de Configuración

El *script* presentado en el Listado B.4 automatiza el resguardo de toda la configuración exportable de Coolify: localiza los archivos `.env` y `docker-compose` activos, los empaqueta y los transfiere a MinIO mediante `rclone`.

```

1 #!/bin/bash
2 # --- CONFIGURACION ---
3 DATE=$(date +%Y-%m-%d_%H%M%S)
4 BACKUP_NAME="coolify_configs_${DATE}.tar.gz"
5 BACKUP_TEMP_FILE="/tmp/${BACKUP_NAME}"
6 REMOTE_MINIO="minio:backups-coolify/configs"
7
8 echo "Buscando archivos .env y docker-compose..."
9
10 find /data/coolify/applications /data/coolify/services \
11     -name ".env" \
12     -o -name "docker-compose.yml" \
13     -o -name "docker-compose.yaml" \
14     | tar -czf "${BACKUP_TEMP_FILE}" -P -T -
15
16 echo "Subiendo backup a MinIO..."
17 rclone copy "${BACKUP_TEMP_FILE}" "$REMOTE_MINIO"
18
19 if [ $? -eq 0 ]; then
20     echo "Backup enviado correctamente."
21     rm "${BACKUP_TEMP_FILE}"
22 else
23     echo "ERROR: Fallo la subida con rclone."
24     exit 1
25 fi

```

CÓDIGO B.4: Script de respaldo selectivo de configuraciones exportables hacia MinIO

Su ejecución se programó con la expresión `cron 04***/bin/bash/root/backup_configs.sh>>/var/log/coolify_backup.log2>&1`. Desde el punto de vista de diseño, el uso de `rclone` en lugar de una utilidad específica de MinIO mantiene el procedimiento portable hacia otros destinos S3 compatibles sin modificar la lógica principal del respaldo.

B.4. Variables Mágicas de Coolify (SERVICE_###)

Coolify incorpora un mecanismo de variables mágicas que permite inyectar automáticamente credenciales, usuarios y otros valores sensibles generados de forma segura durante el despliegue [39]. En lugar de codificar secretos en texto plano dentro del archivo `docker-compose.yml`, la PaaS expone variables derivadas del contexto del servicio, las cuales se resuelven en tiempo de ejecución.

En este despliegue, Coolify resolvió automáticamente `SERVICE_PASSWORD_DOCUMENTSO`, `SERVICE_USER_POSTGRES` y `SERVICE_PASSWORD_POSTGRES` sin requerir valores explícitos en la composición. Con esta estrategia, el repositorio de configuración nunca almacena credenciales en texto plano: los secretos los genera y gestiona la propia PaaS.

Además de las variables generadas automáticamente, desde el mismo panel se definieron variables manuales. En particular, `DOCUMENTSO_DISABLE_TELEMETRY=true` desactiva la recopilación de datos de telemetría anónima que Documentso habilita por defecto en instancias autoalojadas,

cuyo propósito original es reunir estadísticas de uso del producto. Su inclusión responde al criterio de evitar comunicaciones hacia servidores externos no contemplados en el diseño de la infraestructura institucional.

B.5. Generación de Material Criptográfico Institucional

La configuración de firma local de Documenso admite el uso de un certificado PKCS#12 suministrado como contenido codificado en Base64, en lugar de montarlo como archivo físico dentro del contenedor [40]. Para este despliegue se generó un certificado institucional autofirmado, destinado al sellado criptográfico de documentos dentro del dominio operativo de la FACET.

El Listado B.5 sintetiza el procedimiento: primero se genera la clave privada RSA de 2048 bits; sobre ella se emite el certificado X.509 autofirmado con los atributos institucionales; ambos artefactos se consolidan en un contenedor PKCS#12 con el modificador `-legacy`, necesario para la compatibilidad con Documenso, y el resultado se codifica en Base64 para su inyección como variable de entorno.

```
1 openssl genrsa -out private.key 2048
2 openssl req -new -x509 -key private.key -out certificate.crt -days 3650 -
  subj "/C=AR/ST=Tucuman/L=San Miguel de Tucuman/O=Universidad Nacional
  de Tucuman/OU=FACET/CN=documentos.facet.unt.edu.ar/emailAddress=
  admin@documentos.facet.unt.edu.ar"
3 openssl pkcs12 -export -legacy -out certificate_facet.p12 -inkey private.
  key -in certificate.crt -passout pass:${SERVICE_PASSWORD_DOCUMENSO}
4 base64 -w 0 certificate_facet.p12 > cert_base64_facet.txt
```

CÓDIGO B.5: Generación del certificado institucional PKCS#12 y codificación en Base64

El contenido generado en `cert_base64_facet.txt` se asigna posteriormente a la variable `CERTIFICADO_P12_BASE64`. En términos operativos, esto significa que el despliegue no almacena el certificado en texto plano dentro del repositorio, sino que lo inyecta desde Coolify como un secreto externo. La contraseña utilizada para proteger el archivo PKCS#12 es la provista dinámicamente por la variable `SERVICE_PASSWORD_DOCUMENSO`, generada por la propia PaaS. Aunque la variable que Documenso consume internamente para la firma local es `NEXT_PRIVATE_SIGNING_LOCAL_FILE_CONTENTS`, en esta implementación el valor se administra en Coolify mediante `CERTIFICADO_P12_BASE64` y luego se referencia en la composición con la asignación `NEXT_PRIVATE_SIGNING_LOCAL_FILE_CONTENTS=${CERTIFICADO_P12_BASE64}`. En la composición, `CERTIFICADO_P12_BASE64` almacena el certificado codificado y se inyecta directamente en `NEXT_PRIVATE_SIGNING_LOCAL_FILE_CONTENTS`, la variable que Documenso consume al iniciar. Respaldo el archivo de variables de Coolify equivale a respaldar el certificado institucional: su contenido completo está embebido en `CERTIFICADO_P12_BASE64`, de modo que no se requiere un procedimiento de respaldo independiente para el material criptográfico. La documentación oficial de Documenso enfatiza que tanto este certificado como su frase de paso deben considerarse parte del esquema de respaldo de la plataforma, ya que su pérdida impediría continuar emitiendo nuevas firmas con la identidad institucional configurada [29].

B.6. Configuración de Almacenamiento S3 (MinIO)

Documenso permite delegar el almacenamiento de documentos a un *backend* compatible con S3 mediante variables específicas de configuración [30]. En el despliegue institucional, este mecanismo se vinculó con MinIO expuesto detrás del *reverse proxy* institucional.

Las variables relevantes son las siguientes:

- NEXT_PUBLIC_UPLOAD_TRANSPORT: fijado en s3 para delegar la persistencia de archivos al *backend* de objetos en lugar del disco local.
- NEXT_PRIVATE_UPLOAD_ENDPOINT: URL base del servidor de almacenamiento (https://minio.facet.unt.edu.ar).
- NEXT_PRIVATE_UPLOAD_BUCKET: nombre del *bucket* lógico (documenso-prod) que conserva los documentos.
- NEXT_PRIVATE_UPLOAD_REGION: región genérica requerida por el cliente S3 (fijada en us-east-1).
- NEXT_PRIVATE_UPLOAD_FORCE_PATH_STYLE: fijado en true para forzar URLs basadas en ruta, un ajuste vital para que el *proxy* Traefik enrute el tráfico correctamente hacia MinIO.
- NEXT_PRIVATE_UPLOAD_ACCESS_KEY_ID: credencial pública de acceso a MinIO.
- NEXT_PRIVATE_UPLOAD_SECRET_ACCESS_KEY: clave secreta de autenticación S3.

Además de las variables anteriores, la operación correcta de la carga de archivos exigió un ajuste explícito en el *reverse proxy* perimetral de la facultad. Durante las pruebas iniciales, toda solicitud de carga superior a 1 MB era rechazada con el código HTTP 413 Content Too Large, evidenciando que el límite efectivo no estaba siendo impuesto por Documenso sino por la capa de publicación externa. Para resolverlo, se ampliaron los valores de `client_max_body_size` a 50 MB para documentos.facet.unt.edu.ar y a 75 MB para minio.facet.unt.edu.ar. Esta diferencia permitió mantener un margen operativo sobre el límite visible configurado en la aplicación y contemplar el hecho de que los artefactos firmados pueden resultar ligeramente más pesados que el PDF original.

B.7. Configuración de Transporte de Correo (SMTP)

La plataforma requiere un transporte SMTP para el envío de invitaciones de firma, notificaciones de estado y comunicaciones transaccionales del sistema [41]. En esta implementación se utilizó el *relay* de Brevo mediante autenticación SMTP clásica.

Las variables principales son las siguientes:

- NEXT_PRIVATE_SMTP_TRANSPORT: mecanismo de conexión (fijado en smtp-auth).
- NEXT_PRIVATE_SMTP_HOST: servidor *relay* remoto (smtp-relay.brevo.com).
- NEXT_PRIVATE_SMTP_PORT: puerto de conexión TCP (587).

- NEXT_PRIVATE_SMTP_SECURE: fijado en false para delegar el cifrado al mecanismo STARTTLS en lugar de exigir TLS implícito desde el inicio.
- NEXT_PRIVATE_SMTP_USERNAME: usuario de autenticación del *relay*.
- NEXT_PRIVATE_SMTP_PASSWORD: contraseña del servicio SMTP.
- NEXT_PRIVATE_SMTP_FROM_ADDRESS: correo emisor de las notificaciones institucionales (notificaciones@documentos.facet.unt.edu.ar).
- NEXT_PRIVATE_SMTP_FROM_NAME: remitente visible para el destinatario (Firmas FACET).

B.8. Autenticación con Google OAuth 2.0

Documenso soporta autenticación mediante Google OAuth 2.0 a través de variables de entorno específicas para el proveedor [42]. En el entorno institucional, este mecanismo se utilizó para delegar el proceso de autenticación al dominio administrado por Google Workspace, evitando la gestión local de contraseñas dentro de la plataforma.

Las variables principales son las siguientes:

- NEXT_PRIVATE_GOOGLE_CLIENT_ID: identificador único de la aplicación registrada en Google Cloud.
- NEXT_PRIVATE_GOOGLE_CLIENT_SECRET: secreto criptográfico para el intercambio seguro de *tokens* con la API de Google.
- NEXT_PUBLIC_DISABLE_SIGNUP: fijado en true para bloquear el registro local y limitar el ingreso exclusivamente a las identidades del dominio institucional preautorizado.

Estas variables son insuficientes sin la configuración del cliente OAuth en Google Cloud, donde deben registrarse los URI de redirección autorizados para el dominio institucional. La desactivación del registro local complementa este esquema, ya que evita la creación autónoma de cuentas dentro de la plataforma y refuerza el criterio de que solo usuarios con identidad institucional validada puedan iniciar trámites desde la interfaz web.

Apéndice C

Manual de Mantenimiento del Sistema

Este apéndice documenta los procedimientos operativos de mantenimiento del sistema de firma electrónica desplegado en la FACET. Su propósito es complementar la Sección 4.7 del Capítulo 4, concentrando el detalle paso a paso de las tareas de restauración en un único documento de referencia para el personal de TI.

Los procedimientos descritos asumen que los artefactos de respaldo (volcados `.dmp` y archivos `.tar.gz`) se encuentran disponibles en los *buckets* correspondientes del servidor MinIO, tal como se verifica en la Sección 4.7.

C.1. Restauración de la Base de Datos

Este procedimiento permite restaurar PostgreSQL a partir de un volcado `.dmp` almacenado en el *bucket* `documenso-backup-databases` de MinIO. La operación se ejecuta íntegramente desde la interfaz de Coolify y no requiere acceso directo por terminal al contenedor de la base de datos.

Prerrequisitos: acceso al panel de Coolify con permisos de administrador y conectividad con el servidor MinIO.

1. **Localizar el respaldo.** Dentro de Coolify, navegar al proyecto Documenso siguiendo la ruta Documenso → production → DocumensoProd. Seleccionar el servicio de la base de datos (`database (postgres:17)`) y acceder a la pestaña **Backups**. En el listado de ejecuciones, identificar el último respaldo con estado `Success` y copiar la ruta completa indicada en el campo `Location` (Figura C.1).

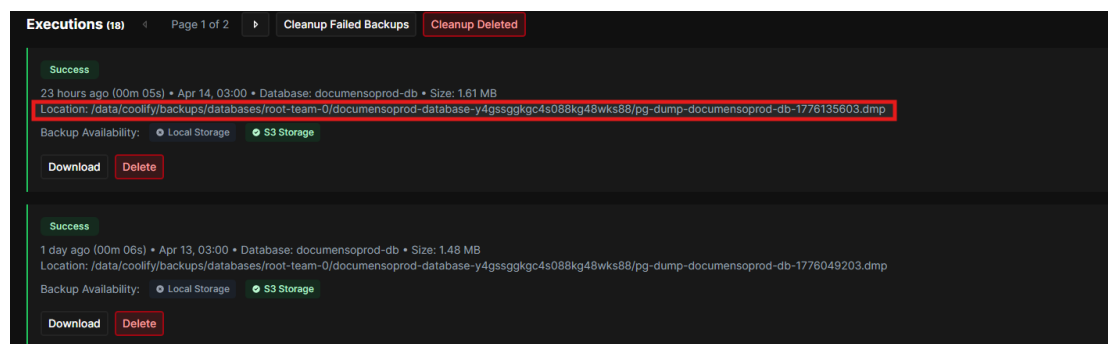


FIGURA C.1: Pestaña **Backups** del servicio database en Coolify. Se observan las ejecuciones diarias con estado **Success**, el tamaño de cada volcado y el campo **Location** que contiene la ruta del objeto en MinIO.

2. **Configurar la importación.** Acceder a la pestaña **Import Backup** del mismo servicio. En la sección **Choose Restore Method**, seleccionar la opción **Restore from S3**.
3. **Seleccionar el origen S3.** En el menú desplegable **S3 Storage**, elegir la conexión previamente configurada en Coolify: *Backup Sistemas de Firmas – Respaldo del Sistema de Firmas Documenso*.
4. **Validar el archivo.** En el campo **S3 File Path** (within bucket), pegar la ruta (Location) copiada en el paso 1. Presionar el botón **Check File** y verificar que la sección **File Information** muestre el nombre y el tamaño del volcado. Esta comprobación confirma que el objeto existe en MinIO y que la conexión S3 es funcional.
5. **Modificar el comando de restauración.** Este es el paso más crítico del procedimiento. El comando de importación que Coolify genera por defecto es:

```
pg_restore -U $POSTGRES_USER -d ${POSTGRES_DB:-
${POSTGRES_USER:-postgres}}
```

Advertencia: si la base de datos de destino ya contiene datos —que es el caso habitual en un entorno de producción activo—, la ejecución sin modificaciones fallará por colisión de objetos existentes. El administrador **debe** agregar la bandera `--clean` al final del campo **Custom Import Command** para que `pg_restore` elimine los objetos de la base antes de recrearlos. El comando final debe quedar de la siguiente forma:

```
pg_restore -U $POSTGRES_USER -d ${POSTGRES_DB:-
${POSTGRES_USER:-postgres}} --clean
```

6. **Ejecutar la restauración.** Una vez configurados los campos anteriores, presionar el botón **Restore Database from S3** (Figura C.2). Coolify descargará el volcado desde MinIO y ejecutará `pg_restore --clean` contra el contenedor de PostgreSQL.

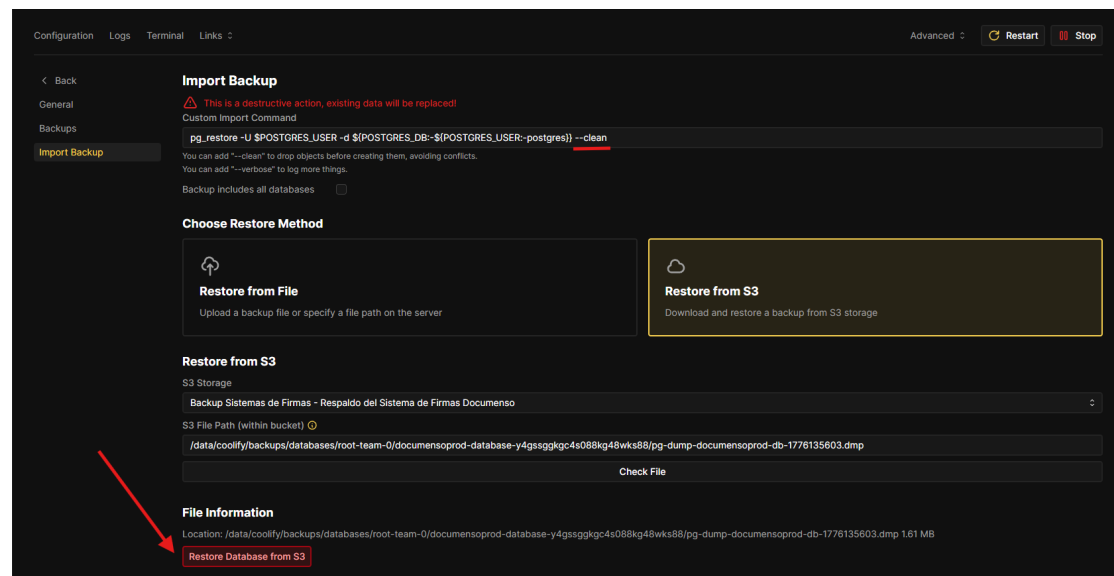


FIGURA C.2: Pestaña **Import Backup** con la opción **Restore from S3** seleccionada. Se observa el comando de restauración modificado con la bandera `--clean`, la conexión S3 configurada, la ruta del volcado validada mediante **Check File** y el botón **Restore Database from S3**.

7. **Verificación posterior.** Una vez completada la operación, verificar la integridad del sistema:
 - Consultar el *endpoint* `https://documentos.facet.unt.edu.ar/api/health` y confirmar que los campos `status`, `database` y `certificate` reportan "ok".
 - Iniciar sesión en la interfaz web y comprobar que los documentos, firmantes y equipos previos se encuentran íntegros.

C.2. Restauración de la Configuración del Stack

Este procedimiento describe cómo reconstruir la configuración del despliegue —archivos `docker-compose.yml` y variables de entorno— a partir de los respaldos almacenados en el *bucket* `backups-coolify` de MinIO. El flujo de trabajo combina la terminal integrada de Coolify con su interfaz gráfica.

Prerrequisitos: acceso al panel de Coolify con permisos de administrador. La utilidad `rc1one` ya se encuentra instalada en el LXC como parte de la automatización de respaldos documentada en el Apéndice B.3.

C.2.1. Acceso a la terminal del servidor

En la barra lateral izquierda de Coolify, seleccionar la pestaña **Terminal** y, en el selector central, elegir el servidor **localhost**. Esta consola opera sobre el LXC anfitrión del despliegue, por lo que no se requiere una conexión SSH externa.

C.2.2. Recuperación y extracción de archivos

Desde la terminal, listar los respaldos disponibles, copiar el deseado y extraer su contenido:

```

1 # 1. Listar respaldos disponibles en MinIO
2 rclone ls minio:backups-coolify
3
4 # 2. Copiar el ultimo respaldo a una carpeta temporal
5 rclone copy minio:backups-coolify/configs/coolify_configs_YYYY-MM-DD.tar.gz /tmp/
6
7 # 3. Extraer el contenido
8 tar -xzf /tmp/coolify_configs_YYYY-MM-DD.tar.gz -C /tmp/
9
10 # 4. Verificar el contenido de un artefacto extraido
11 cat /tmp/data/coolify/services/y4gssggkgc4s088kg48wks88/.env

```

CÓDIGO C.1: Descarga y extracción del respaldo de configuración desde MinIO

El valor YYYY-MM-DD debe reemplazarse por la fecha del respaldo que corresponda restaurar, según lo observado en la salida del paso 1. La Figura C.3 ilustra la ejecución exitosa de los comandos de listado y copia dentro de la terminal de Coolify.

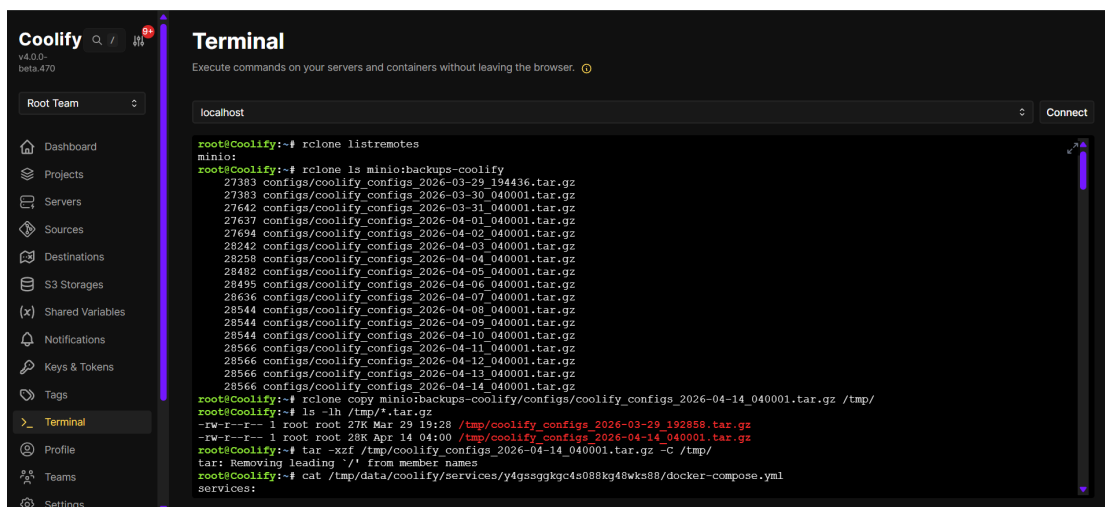


FIGURA C.3: Terminal integrada de Coolify mostrando el listado de respaldos disponibles en MinIO y la copia del archivo seleccionado al directorio temporal del servidor.

C.2.3. Identificación de servicios

Coolify organiza los archivos de cada servicio bajo identificadores únicos generados en el momento del despliegue. Las rutas críticas para el despliegue institucional actual son:

- **Stack Documenso** (aplicación, base de datos y Browserless): `/data/coolify/services/y4gssggkgc4s088kg48wks88/`
- **Servicio Socat**: `/data/coolify/services/i040ggc0c84c0sc0ss4cs0co/`

Cada directorio contiene un archivo `docker-compose.yml` y un archivo `.env` con la definición de servicios y los secretos necesarios para su operación. Estos identificadores corresponden al despliegue vigente al momento de redactar este documento; ante una reinstalación o migración de Coolify, los identificadores se regeneran y las rutas cambian. En ese caso, se debe localizar el directorio correcto consultando el panel de Coolify o listando el contenido de `/data/coolify/services/` en el servidor.

C.2.4. Restauración desde la interfaz de Coolify

Navegar en el panel de Coolify hacia Proyecto Documenso → `production` → `DocumensoProd` y seguir los pasos indicados a continuación.

1. **Restaurar el archivo Compose.** Presionar el botón **Edit Compose File**. En el editor emergente (Figura C.4), reemplazar todo el contenido con el texto del archivo `docker-compose.yml` extraído en la ruta del identificador correspondiente y presionar **Save**.

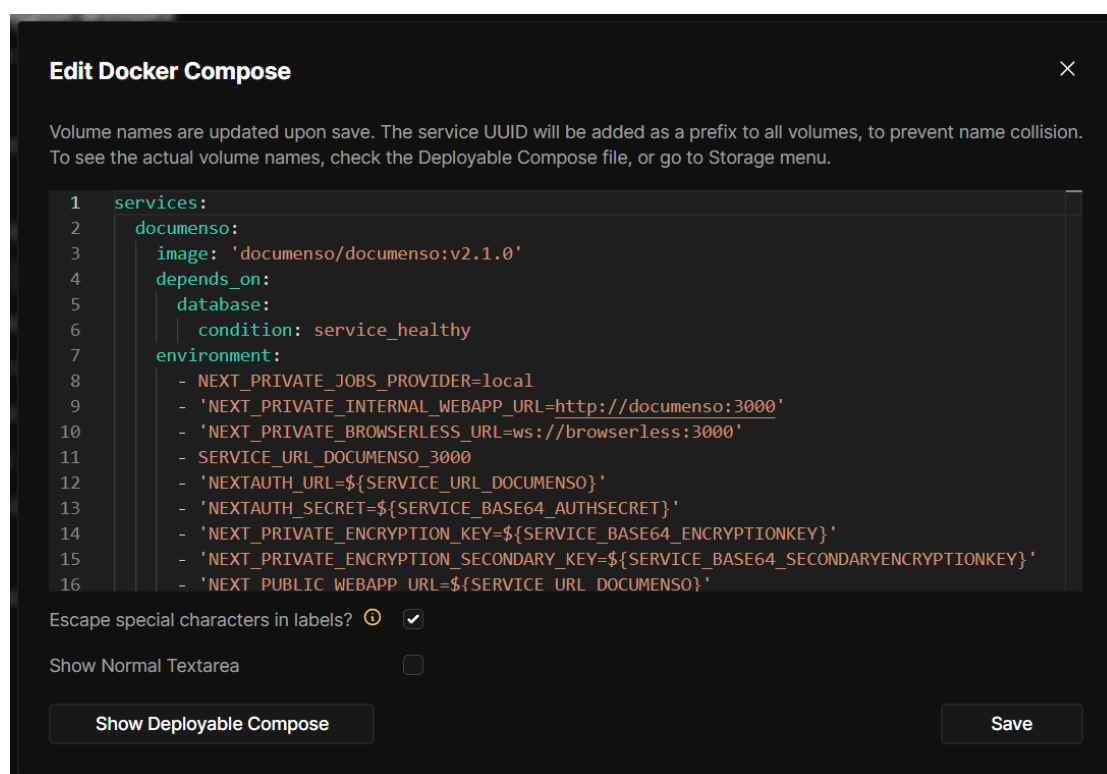


FIGURA C.4: Editor de la composición Docker accesible mediante **Edit Compose File** en Coolify, donde se reemplaza el contenido con la configuración recuperada del respaldo.

2. **Restaurar las variables de entorno.** Ir a la pestaña **Environment Variables**, activar **Developer View** para obtener el editor de texto plano y pegar el contenido completo del archivo `.env` restaurado. Presionar **Save**.

3. **Repetir para el servicio Socat.** Navegar al servicio del *proxy* Socat en Coolify y aplicar los pasos 1 y 2 con los archivos de su identificador (i040ggc0c84c0sc0ss4cs0co).
4. **Desplegar y verificar.** Presionar **Deploy** en cada servicio. Tras el redespliegue, confirmar que los cuatro contenedores alcancen el estado `Healthy` en el panel de Coolify (Subsección 4.2.1) y que el *endpoint* `https://documentos.facet.unt.edu.ar/api/health` reporte "ok" en todos sus campos.

C.3. Renovación del Certificado de Firma Institucional

Cuando el certificado autofirmado se aproxima a su fecha de caducidad, se debe generar un nuevo par de claves y reemplazar la variable de entorno correspondiente en Coolify. El certificado actual, generado durante el despliegue inicial, es válido hasta el 25 de marzo de 2036 (Apéndice B.5).

1. **Generar el nuevo material criptográfico.** Abrir una terminal en la computadora local del administrador y ejecutar los comandos de generación de clave privada, certificado X.509 y empaquetado PKCS#12 detallados en la Sección B.5. Al finalizar, se obtendrá un archivo de texto con la cadena codificada en Base64 del nuevo certificado. No es necesario ni recomendable ejecutar este procedimiento dentro del servidor de producción.
2. **Actualizar la variable de entorno.** En la interfaz de Coolify, navegar a Proyecto Documenso → `production` → `DocumensoProd` y acceder a la pestaña **Environment Variables**. Ubicar la variable `CERTIFICADO_P12_BASE64` (Figura C.5) y reemplazar su valor por la nueva cadena en Base64 generada en el paso anterior. Presionar **Update**.

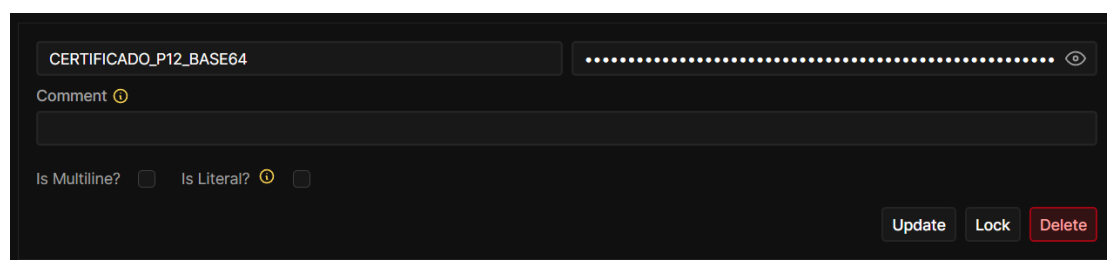


FIGURA C.5: Variable `CERTIFICADO_P12_BASE64` en la pestaña **Environment Variables** de Coolify. El valor se muestra enmascarado por tratarse de un secreto.

3. **Actualizar la contraseña (condicional).** Si al generar el nuevo PKCS#12 se utilizó una contraseña distinta a la original, también se debe actualizar el valor de la variable `SERVICE_PASSWORD_DOCUMENSO` en la misma pestaña.
4. **Aplicar los cambios.** Presionar **Save** y luego **Deploy** para reiniciar el contenedor de Documenso con el nuevo certificado. El éxito de la operación se comprueba consultando el *endpoint* `https://documentos.facet.unt.edu.ar/api/health` y verificando que el campo `certificate` reporte "ok".

La renovación del certificado **no invalida ni corrompe** los documentos que ya fueron firmados con el certificado anterior. Cada PDF sellado conserva incrustada la firma y el certificado con el que fue emitido, por lo que su verificabilidad no depende del certificado activo en la plataforma. El nuevo certificado se utiliza exclusivamente para el sellado de los documentos que se procesen a partir del redesplicue.

C.4. Actualización de la Versión de Documenso

Este procedimiento describe cómo actualizar Documenso a una versión más reciente dentro del entorno administrado por Coolify. Dado que la aplicación ejecuta migraciones de esquema de base de datos durante cada arranque, la operación exige un respaldo previo y la verificación de compatibilidad con el hardware del servidor.

C.4.1. Prerrequisito crítico de hardware

Advertencia: el servidor físico original de la FACET utiliza procesadores Intel Xeon E5620, que **no soportan instrucciones AVX**. Tal como se detalla en la Subsección 3.3.3, a partir de la versión v2.2.0 Documenso requiere AVX, por lo que el despliegue opera actualmente con v2.1.0, última versión compatible con este hardware.

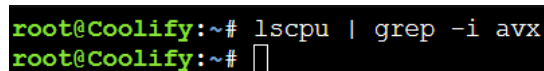
El procedimiento de actualización descrito en esta sección solo debe aplicarse si la infraestructura ha sido migrada a un servidor con CPU compatible con AVX. Para verificar la compatibilidad del nuevo servidor, ejecutar el siguiente comando en la terminal del LXC:

```
lscpu | grep -i avx
```

CÓDIGO C.2: Verificación de soporte AVX en el procesador del servidor

Si el comando no devuelve ninguna línea con las *flags* *avx* o *avx2*, el servidor **no es compatible** con versiones iguales o superiores a v2.2.0. La Figura C.6 muestra el resultado de este comando en el hardware actual, donde la salida vacía confirma la ausencia de soporte AVX.

Nota sobre virtualización: si el nuevo servidor utiliza máquinas virtuales (por ejemplo, en Proxmox VE), el hipervisor puede ocultar las instrucciones AVX del procesador físico al usar perfiles de CPU genéricos como *kvm64*. Para evitar este problema, se debe configurar el modelo de CPU de la VM como *host*.



```
root@Coolify:~# lscpu | grep -i avx
root@Coolify:~#
```

FIGURA C.6: Ejecución de `lscpu | grep -i avx` en el LXC del despliegue. La salida vacía confirma que el procesador Intel Xeon E5620 no soporta instrucciones AVX, lo que impide actualizar Documenso a versiones iguales o superiores a v2.2.0.

C.4.2. Preparación

Antes de modificar cualquier archivo del despliegue, se deben completar dos tareas de preparación:

1. **Consultar las notas de versión.** Revisar las *Release Notes* oficiales en <https://github.com/documenso/documenso/releases> para la versión objetivo y todas las versiones intermedias entre la actual y la deseada. Identificar posibles *breaking changes*, variables de entorno nuevas o eliminadas y requisitos adicionales de infraestructura.
2. **Realizar un respaldo manual de la base de datos.** Documenso utiliza Prisma como ORM y ejecuta migraciones de esquema de forma automática durante el arranque de cada nueva versión. **Estas migraciones no son reversibles:** una vez aplicadas, el esquema de la base de datos queda adaptado a la versión nueva y no puede volver al estado anterior de forma automática. Por este motivo, es **estrictamente obligatorio** crear un respaldo manual de PostgreSQL desde Coolify antes de proceder. El procedimiento de respaldo y restauración se detalla en la Sección C.1.

C.4.3. Procedimiento de actualización en Coolify

Una vez verificada la compatibilidad de hardware y completado el respaldo, la actualización se ejecuta desde la interfaz de Coolify:

1. **Editar la composición Docker.** Navegar a Proyecto Documenso → production → DocumensoProd y presionar **Edit Compose File**. Localizar la línea de la imagen del servicio principal y modificar el *tag* de versión:

```
image: 'documenso/documenso:vX.Y.Z'
```

donde vX.Y.Z corresponde a la versión objetivo identificada en las notas de versión. Presionar **Save**.

2. **Actualizar variables de entorno (condicional).** Si las notas de versión indican variables nuevas o modificadas, agregarlas o ajustarlas en la pestaña **Environment Variables** antes del despliegue.
3. **Desplegar.** Presionar el botón **Deploy**. Coolify descargará la nueva imagen y recreará el contenedor. Durante el arranque, Prisma detectará las migraciones pendientes y las aplicará de forma automática sobre PostgreSQL. No se requiere intervención manual en la base de datos.

C.4.4. Verificación

Tras el despliegue, el administrador debe confirmar que la actualización se completó correctamente:

1. **Revisar los registros del servicio.** En Coolify, acceder a la pestaña **Logs** del servicio Documenso y verificar que las migraciones de Prisma hayan finalizado sin errores. Los mensajes indicativos de éxito incluyen líneas del tipo `All migrations have been successfully applied`.
2. **Consultar el endpoint de salud.** Acceder a <https://documentos.facet.unt.edu.ar/api/health> y verificar que los campos `status`, `database` y `certificate` reporten "ok".
3. **Validación funcional.** Iniciar sesión en la interfaz web y comprobar que los documentos, firmantes y equipos existentes se muestren correctamente. Se recomienda ejecutar un

ciclo de firma de prueba de extremo a extremo para confirmar la operación completa del sistema.

C.4.5. Procedimiento de rollback

Si la actualización falla o produce un comportamiento inesperado, las migraciones de esquema aplicadas por Prisma **no se pueden revertir de forma automática**. Por este motivo, la primera acción debe ser detener el servicio antes de cualquier operación de restauración.

1. **Detener el servicio fallido.** En el panel de Coolify, acceder a DocumensoProd y presionar el botón **Stop**. Esto detiene el contenedor de la aplicación y evita que siga intentando aplicar migraciones sobre la base de datos. **No detener el contenedor de PostgreSQL**, ya que debe permanecer activo para el paso siguiente.
2. **Restaurar la base de datos.** Con el contenedor de la aplicación detenido, acceder al servicio database en Coolify y ejecutar el procedimiento de importación desde S3 descrito en la Sección C.1, utilizando el respaldo creado en la etapa de preparación.
3. **Revertir la imagen.** Volver al servicio DocumensoProd, presionar **Edit Compose File** y restaurar el *tag* de la imagen a la versión estable anterior (por ejemplo, documenso/documenso:v2.1.0). Presionar **Save**.
4. **Redesplegar.** Presionar el botón **Deploy**. Coolify recreará el contenedor de la aplicación con la versión correcta y lo reconectará a la base de datos ya restaurada. Verificar la operación mediante el *endpoint* `/api/health` y una prueba funcional en la interfaz web.

C.5. Diagnóstico de Fallas y Referencia Rápida

Esta sección reúne los puntos de inspección y las referencias necesarias para diagnosticar las fallas más frecuentes del despliegue.

C.5.1. Monitoreo y registros

El estado de salud y los registros de ejecución de los tres servicios del *stack* —Documenso, Browserless y PostgreSQL— se consultan directamente desde la pestaña **Logs** de cada contenedor, accesible en Coolify mediante la ruta Documenso → production → DocumensoProd. Desde allí se pueden filtrar los mensajes por servicio para aislar el componente afectado.

C.5.2. Tabla de resolución de problemas

La Tabla 7 resume los síntomas más comunes, el componente involucrado y la referencia al procedimiento o configuración correspondiente.

TABLA 7: Síntomas frecuentes del despliegue Documenso, componente a revisar y referencia al procedimiento o configuración correspondiente.

Síntoma / Error	Componente a revisar / Referencia
El <i>endpoint</i> /api/health reporta error en database	Estado del contenedor database en Coolify; verificar credenciales en el <code>.env</code> .
El <i>endpoint</i> /api/health reporta error en certificate	El certificado cadució o el BASE64 es inválido (Sección C.3).
Los correos de invitación no llegan	Configuración SMTP y registros del contenedor Documenso (Sección B.7).
Error al procesar la firma o al cargar el editor de documentos	Registros del contenedor Browserless; verificar estado Healthy (Subsección 3.3.2).
Illegal instruction (core dumped)	Incompatibilidad de CPU con instrucciones AVX (Sección C.4).
Documentos quedan en estado PENDING sin completarse	Registros del <i>job</i> <code>internal.seal-document</code> (Subsección 3.3.2).
Carga de archivos falla con 413 Content Too Large	Límite de <code>client_max_body_size</code> en Nginx (Sección 3.4).
Access Denied al subir o descargar documentos	Credenciales S3 o política del <i>bucket</i> en MinIO (Sección B.6).
Carga de archivos falla con error de almacenamiento o Quota Exceeded	Verificar y ampliar la cuota del <i>bucket</i> desde la interfaz de MinIO (Buckets → Quota).

Para diagnósticos fuera de los escenarios listados, se puede consultar la guía oficial de resolución de problemas de Documenso en <https://docs.documenso.com/docs/self-hosting/maintenance/troubleshooting>.

C.6. Migración a Certificado Emitido por Autoridad Certificante (CA)

Tal como se establece en la Sección 2.3, la arquitectura del sistema soporta de forma nativa certificados X.509 emitidos por Autoridades Certificantes licenciadas. Esta capacidad permite, sin modificar la infraestructura subyacente, migrar desde el esquema actual de firma electrónica con certificado autofirmado hacia un esquema de firma digital que cumpla el Artículo 9 de la Ley 25.506 [12], si la institución lo requiriera en el futuro.

Las CA comerciales o gubernamentales suelen entregar el material criptográfico en archivos separados: clave privada, certificado del titular y cadena de confianza intermedia. Documenso requiere que estos componentes se empaqueten en un único contenedor PKCS#12. El siguiente comando realiza esa operación:

```
1 openssl pkcs12 -export -out certificate.p12 \
2   -inkey private.key \
3   -in certificate.crt \
4   -certfile chain.crt
```

CÓDIGO C.3: Empaquetado de certificado emitido por CA en formato PKCS#12

Una vez obtenido el archivo `certificate.p12`, el procedimiento operativo es idéntico al de la Sección C.3: codificar el archivo en Base64, reemplazar el valor de la variable `CERTIFICADO_P12_BASE64` en Coolify y, si la frase de paso difiere de la actual, actualizar también `SERVICE_PASSWORD_DOCUMENTSO`. Tras presionar **Deploy**, los documentos firmados a partir de ese momento llevarán un certificado reconocido por las cadenas de confianza de los lectores PDF estándar.

Para conversiones desde otros formatos de certificado (como DER o PEM sin cadena), se puede consultar la guía oficial de Documenso en <https://docs.documenso.com/docs/self-hosting/configuration/signing-certificate/local#using-an-existing-certificate>.

Apéndice D

Guía de Inicio Rápido para Usuarios

Este apéndice constituye una guía operativa dirigida al personal docente y administrativo de la FACET para el uso cotidiano de la plataforma de firma electrónica Documenso. Se describen los pasos necesarios para firmar documentos institucionales dentro del contexto organizacional correcto. Para funcionalidades avanzadas —plantillas, campos especiales, configuración de equipos— se recomienda consultar la documentación oficial del proyecto, disponible en <https://docs.documenso.com/docs/users>.

D.1. Acceso al Sistema

El acceso se realiza mediante autenticación Google OAuth con la cuenta institucional @hererra.unt.edu.ar. Al iniciar sesión por primera vez, el sistema posiciona al usuario en su organización **Personal**, que es un espacio individual no vinculado a la estructura institucional. Los documentos creados en este espacio no se asocian a la FACET.

D.2. Selección de Organización y Equipo

Este es el paso más importante del flujo de trabajo. Antes de cargar cualquier documento oficial, se debe verificar el contexto organizacional:

1. En el selector de organización (ubicado en la barra superior derecha), cambiar de **Personal** a **FACET** (Figura D.1).
2. Una vez dentro de la organización FACET, seleccionar el **Equipo** (Departamento) correspondiente al departamento del usuario (Figura D.2).

Todo documento subido fuera del contexto FACET/Equipo quedará desvinculado de la estructura institucional y no será visible para los demás miembros del departamento.

Si la organización FACET no aparece en el selector, cerrar la sesión actual, abrir el enlace de invitación a la organización FACET recibido por correo electrónico e iniciar sesión nuevamente con la cuenta institucional. Tras este paso, la organización FACET quedará disponible en el selector.

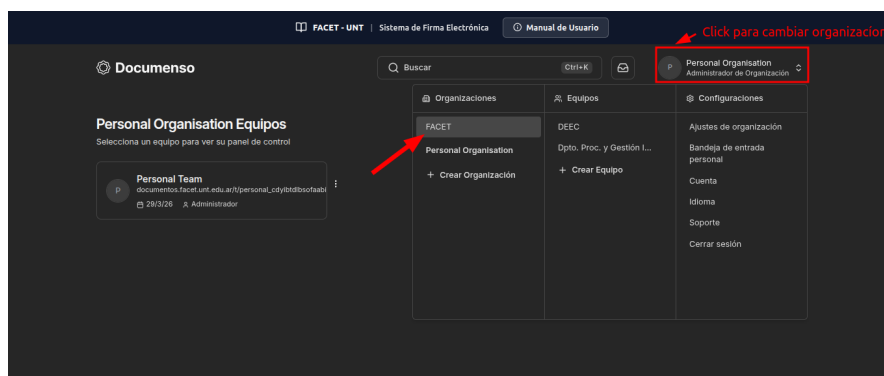


FIGURA D.1: Selector de organización desplegado. Se debe cambiar de *Personal* a *FACET* antes de operar con documentos institucionales.

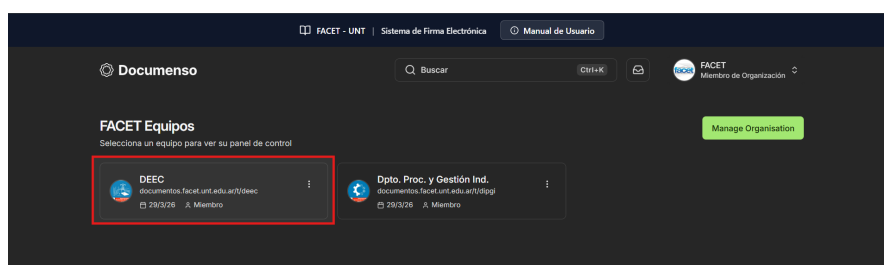


FIGURA D.2: Listado de equipos dentro de la organización FACET. Cada equipo corresponde a un departamento o área de la facultad.

D.3. Carga y Preparación de un Documento

D.3.1. Carga del Archivo

Desde el panel de documentos del equipo, utilizar el botón **Cargar Documento** para subir el archivo PDF a firmar (Figura D.3). El sistema presenta dos opciones: se debe seleccionar **Cargar Documento** e ignorar la opción **Documento (Legado)**. El tamaño máximo admitido es de 50 MB y el formato aceptado es exclusivamente PDF.

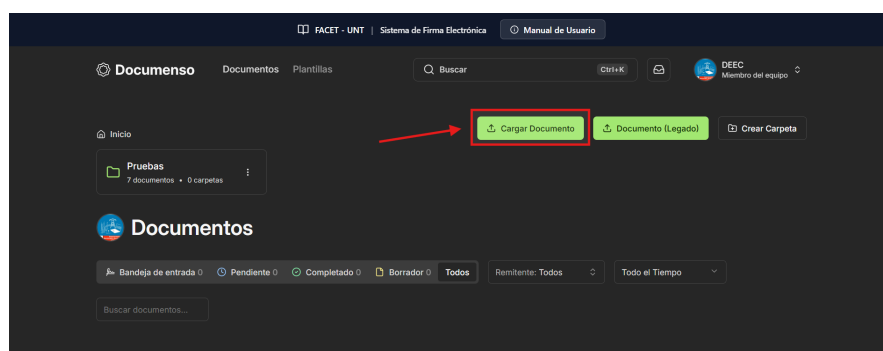


FIGURA D.3: Opciones de carga. Seleccionar siempre **Cargar Documento**; la opción **Documento (Legado)** no debe utilizarse.

Una vez cargado, el documento ingresa en estado **Borrador** y se accede al editor de preparación.

D.3.2. Destinatarios y Roles

En el editor, se agregan los destinatarios indicando para cada uno su dirección de correo electrónico, nombre y rol. La Tabla 8 resume los roles disponibles.

TABLA 8: Roles de destinatario disponibles en Documenso.

Rol	¿Acción requerida?	Descripción
Firmante	Sí	Debe completar los campos de firma asignados.
Aprobador	Sí	Debe aprobar el documento; la firma es opcional.
Visor	Sí	Debe confirmar que ha revisado el documento.
CC	No	Recibe una copia del documento una vez completado.

Opcionalmente, se puede habilitar el **orden de firma** para que los destinatarios actúen en secuencia (por ejemplo, un aprobador en posición 1 y los firmantes en posición 2).

D.3.3. Colocación de Campos de Firma

En la sección **Agregar Campos** del editor, se selecciona el destinatario en el panel lateral y se arrastra al menos un campo de tipo **Firma** sobre la zona del PDF donde debe estamparse (Figura D.4). Los campos se identifican por color según el destinatario asignado. Adicionalmente, se pueden colocar campos de nombre, fecha, texto, iniciales u otros tipos según la necesidad del documento.

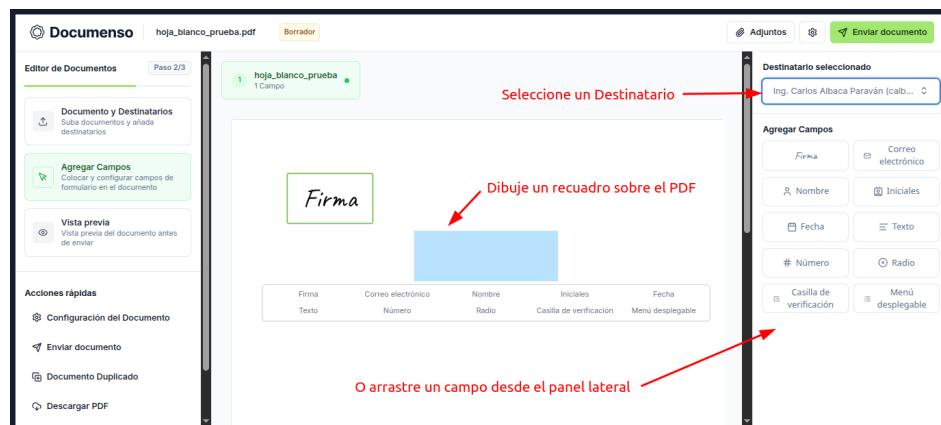


FIGURA D.4: Editor de documento con un campo de firma colocado sobre el PDF. El panel lateral derecho muestra los tipos de campo disponibles.

El documento no podrá enviarse hasta que cada firmante tenga asignado al menos un campo de firma.

D.4. Envío del Documento

Una vez preparado el documento, se procede al envío. La plataforma ofrece dos modalidades de distribución:

1. **Correo electrónico:** cada destinatario recibe una notificación por correo con un enlace único de firma. Se puede personalizar el asunto y el cuerpo del mensaje.
2. **Enlace directo:** se genera un enlace de firma sin enviar correo. Esta modalidad es útil para distribuir el documento por WhatsApp, mensajería interna u otros canales.

La Figura D.5 muestra ambas opciones en la pantalla de envío.

Enviar documento
Los destinatarios podrán firmar el documento una vez enviado

☐ Email
 ☐ None

Responder al correo (Optional)

Asunto (Optional)

Mensaje (Optional) ⓘ

Enviar documento
Los destinatarios podrán firmar el documento una vez enviado

☐ Email
 ☐ None

No enviaremos nada para notificar a los destinatarios.
Generaremos enlaces de firma para ti, que podrás enviar a los destinatarios a través de tu método preferido.

(A) Distribución por correo electrónico.

(B) Generación de enlace directo de firma.

FIGURA D.5: Pantalla de envío del documento. (a) Distribución por correo electrónico. (b) Generación de enlace directo de firma.

Tras el envío, el documento pasa del estado **Borrador** a **Pendiente**.

D.5. Seguimiento y Finalización

El progreso de cada documento se puede consultar desde el panel de documentos del equipo. La Tabla 9 describe los estados posibles.

TABLA 9: Estados del ciclo de vida de un documento.

Estado	Descripción
Borrador	El documento está en preparación; aún no ha sido enviado.
Pendiente	El documento fue enviado y aguarda la acción de los destinatarios.
Completado	Todos los destinatarios completaron sus acciones. El documento queda sellado con un certificado digital.
Rechazado	Un destinatario rechazó el documento (si la opción de rechazo fue habilitada). El documento no puede modificarse ni reactivarse; es necesario crear uno nuevo.

Al completarse, todas las partes reciben una copia del PDF firmado y sellado digitalmente.

Si un destinatario no ha actuado sobre el documento, se le puede reenviar un recordatorio desde el menú de acciones (tres puntos) del documento en estado **Pendiente**.

Para cancelar un envío erróneo, se puede eliminar el documento desde el mismo menú de acciones. Esta operación es irreversible: se anulan todas las firmas pendientes y los destinatarios reciben una notificación de cancelación.

D.6. Experiencia del Destinatario

El destinatario accede al documento a través del enlace recibido por correo o por enlace directo. Al abrirlo, visualiza el PDF con sus campos asignados resaltados. Para los campos de firma, puede optar por dibujar la firma manualmente, escribir su nombre y seleccionar un estilo tipográfico, o cargar una imagen de su firma. Una vez completados todos los campos obligatorios, confirma la acción.

Nota aclaratoria sobre la validación del certificado: al abrir el documento firmado en Adobe Acrobat, es posible que el lector indique que la validez del certificado es desconocida. Este comportamiento es normal y esperado, ya que la plataforma utiliza un certificado criptográfico emitido internamente por la FACET. La integridad del documento está asegurada y el sello es plenamente válido para el uso institucional interno.

D.7. Referencia Rápida

A modo de síntesis, el flujo completo para documentos institucionales es:

1. Iniciar sesión con la cuenta @herrera.unt.edu.ar.
2. Cambiar a la organización **FACET** y seleccionar el equipo del departamento.
3. Cargar el PDF mediante **Cargar Documento**.
4. Agregar destinatarios con sus roles y colocar los campos de firma.
5. Enviar por correo electrónico o generar un enlace directo.
6. Consultar el estado desde el panel de documentos.

Para funcionalidades adicionales como la creación de plantillas reutilizables, configuración o uso de campos distintos al de firma o la gestión de equipos, consultar la documentación oficial de Documenso disponible en <https://docs.documenso.com/docs/users>.

Bibliografía

- [1] DocuSign. *DocuSign Plans and Pricing*. 2025. URL: <https://ecom.docusign.com/plans-and-pricing/esignature> (visitado 05-11-2025).
- [2] Dropbox Sign. *Dropbox Sign Pricing*. 2025. URL: <https://sign.dropbox.com/products/dropbox-sign/pricing> (visitado 05-11-2025).
- [3] Adobe. *Acrobat Sign Pricing for Business*. 2025. URL: <https://www.adobe.com/acrobat/pricing/business.html> (visitado 05-11-2025).
- [4] DocuSeal. *DocuSeal On-Premises*. 2026. URL: <https://www.docuseal.com/on-premises> (visitado 03-04-2026).
- [5] Documenso. *Community Edition*. 2026. URL: <https://docs.documenso.com/docs/policies/community-edition> (visitado 03-04-2026).
- [6] Documenso Contributors. *documenso/documenso*. 2026. URL: <https://github.com/documenso/documenso> (visitado 03-04-2026).
- [7] FOSSA. *Open Source Software Licenses 101: The AGPL License*. 2021. URL: <https://fossa.com/blog/open-source-software-licenses-101-agpl-license/> (visitado 03-04-2026).
- [8] Ultralytics. *Licencia de Software AGPL 3.0*. 2026. URL: <https://www.ultralytics.com/es/legal/agpl-3-0-software-license> (visitado 03-04-2026).
- [9] Eduardo Saade y Cátedra de Protocolos de Comunicación TCP/IP. «Módulo Núm. 7: Seguridad en Redes». Material de clases. Facultad de Ciencias Exactas y Tecnología, Universidad Nacional de Tucumán. 2024.
- [10] Documenso. *Signing Certificates - Documenso Documentation*. 2026. URL: <https://docs.documenso.com/docs/concepts/signing-certificates> (visitado 17-03-2026).
- [11] Documenso. *Signing Certificate Configuration*. 2026. URL: <https://docs.documenso.com/docs/self-hosting/configuration/signing-certificate> (visitado 22-03-2026).
- [12] Congreso de la Nación Argentina. *Ley 25.506 - Firma Digital*. <https://servicios.infoleg.gob.ar/infolegInternet/anexos/70000-74999/70749/norma.htm>. Accedido por última vez: 2026-03-16. 2001.
- [13] Proxmox Server Solutions GmbH. *Introduction to Proxmox VE*. 2026. URL: <https://pve.proxmox.com/pve-docs/chapter-pve-intro.html> (visitado 17-03-2026).
- [14] Bilbao y Lutz. «Taller de Virtualización de Sistemas Operativos en Proxmox». Material de taller. Facultad de Ciencias Exactas y Tecnología, Universidad Nacional de Tucumán. 2024.
- [15] Docker Inc. *Docker Overview*. 2026. URL: <https://docs.docker.com/get-started/docker-overview/> (visitado 17-03-2026).
- [16] Coolify. *Introduction to Coolify*. 2026. URL: <https://coolify.io/docs/get-started/introduction> (visitado 17-03-2026).

- [17] Traefik Labs. *Traefik Concepts*. 2026. URL: <https://doc.traefik.io/traefik/> (visitado 17-03-2026).
- [18] Inc. iXsystems. *TrueNAS SCALE - Open Source Hyperconverged Storage*. 2024. URL: <https://www.truenas.com/truenas-scale/> (visitado 05-11-2025).
- [19] MinIO Inc. *MinIO - High Performance Object Storage*. 2024. URL: <https://min.io/> (visitado 05-11-2025).
- [20] Kim Hefner. *RAID 6*. 2023. URL: <https://www.techtarget.com/searchstorage/definition/RAID-6-redundant-array-of-independent-disks> (visitado 08-04-2026).
- [21] Brien Posey, Erin Sullivan y Paul Crocetti. *Backup Retention Policy Best Practices: A Guide with Template*. 2025. URL: <https://www.techtarget.com/searchdatabackup/answer/What-are-some-data-retention-policy-best-practices> (visitado 02-04-2026).
- [22] MinIO. *Object Locking and Immutability*. 2025. URL: <https://docs.min.io/enterprise/aistor-object-store/administration/object-locking-and-immutability/> (visitado 02-04-2026).
- [23] EnterpriseDB. *Barman: Retention Policies*. 2025. URL: https://docs.pgbarman.org/release/3.17.0/user_guide/retention_policies.html (visitado 02-04-2026).
- [24] IETF. *The OAuth 2.0 Authorization Framework*. 2012. URL: <https://oauth.net/2/> (visitado 05-11-2025).
- [25] Coolify. *Installation and Requirements*. 2026. URL: <https://coolify.io/docs/get-started/installation> (visitado 23-03-2026).
- [26] Documenso. *Self-Hosting: Requirements*. 2026. URL: <https://docs.documenso.com/docs/self-hosting/getting-started/requirements> (visitado 23-03-2026).
- [27] Coolify. *Documenso Template for Coolify*. 2026. URL: <https://github.com/coolabsio/coolify/blob/v4.x/templates/compose/documenso.yaml> (visitado 30-03-2026).
- [28] Documenso. *Docker Compose - Self-Hosting Deployment*. 2026. URL: <https://docs.documenso.com/docs/self-hosting/deployment/docker-compose> (visitado 30-03-2026).
- [29] Documenso. *Backups*. 2026. URL: <https://docs.documenso.com/docs/self-hosting/maintenance/backups> (visitado 02-04-2026).
- [30] Documenso. *Storage Configuration*. 2026. URL: <https://docs.documenso.com/docs/self-hosting/configuration/storage> (visitado 29-03-2026).
- [31] Documenso. *Quick Start*. 2026. URL: <https://docs.documenso.com/docs/self-hosting/getting-started/quick-start#granting-admin-access> (visitado 04-04-2026).
- [32] browserless. *Browsers as a Service - Start*. 2026. URL: <https://docs.browserless.io/baas/start> (visitado 04-04-2026).
- [33] Documenso Contributors. *Playwright/Chromium not included in Docker image - Document sealing fails after signing environment Issue #2060*. 2025. URL: <https://github.com/documenso/documenso/issues/2060> (visitado 30-03-2026).

- [34] Documenso Contributors. *No signing certificate in the document in v1.9.0 Issue #1634*. 2025. URL: <https://github.com/documenso/documenso/issues/1634> (visitado 30-03-2026).
- [35] browserless. *browserless/chrome*. 2026. URL: <https://hub.docker.com/r/browserless/chrome> (visitado 30-03-2026).
- [36] Documenso Contributors. *Crash after upgrading from 2.0.14 to 2.2.4 (Illegal instruction) Issue #2292*. 2025. URL: <https://github.com/documenso/documenso/issues/2292> (visitado 30-03-2026).
- [37] MinIO. *mc ilm rule add*. 2026. URL: <https://docs.min.io/enterprise/aistor-object-store/reference/cli/mc-ilm-rule/mc-ilm-rule-add/> (visitado 02-04-2026).
- [38] Stan Smith. *FossFLOW*. 2026. URL: <https://github.com/stan-smith/FossFLOW> (visitado 11-04-2026).
- [39] Coolify. *Magic Environment Variables*. 2026. URL: <https://coolify.io/docs/knowledge-base/environment-variables#magic-environment-variables> (visitado 29-03-2026).
- [40] Documenso. *Signing Certificate: Using Base64 Encoded Contents*. 2026. URL: <https://docs.documenso.com/docs/self-hosting/configuration/signing-certificate/local#using-base64-encoded-contents> (visitado 29-03-2026).
- [41] Documenso. *Email Configuration*. 2026. URL: <https://docs.documenso.com/docs/self-hosting/configuration/email> (visitado 29-03-2026).
- [42] Documenso. *OAuth Providers: Google OAuth*. 2026. URL: <https://docs.documenso.com/docs/self-hosting/configuration/advanced/oauth-providers#google-oauth-gmail> (visitado 29-03-2026).